

Brian D'Andrade, PhD
Editor

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
p_ob.select= 1  
ler_ob.select=1  
context.scene.objects.active  
["Selected" + str(modifier.name)]  
mirror_ob.select = 0  
bpy.context.selected_objects  
data.objects[one.name].select  
print("please select exact")
```

```
--- OPERATOR CLASSES ---
```

Software Engineering

Artificial Intelligence, Compliance, and Security

Computer Science, Technology and Applications

NOVA

COURSE CODE BCSE301L

SOFTWARE ENGINEERING

Modules 4 & 5

Software Design · *Software Testing*

Comprehensive Academic Study Guide

A reference text and examination-preparation manual

Built on the official course materials for Module 4 (Software Design) and Module 5 (Software Testing), with fully worked numerical problems, examination answer banks, practice questions and two mock examinations with answer keys.

Preface: How to Use This Book

This book is a study text for Modules 4 and 5 of the course *Software Engineering* (BCSE301L). Module 4 treats *software design*: the translation of a requirements model into a design model, the fundamental design concepts, component-level design through cohesion and coupling, architectural mapping from data flow diagrams, and user-interface design. Module 5 treats *software testing*: testing objectives and strategies, white-box techniques built on control flow graphs and cyclomatic complexity, graph and connection matrices, black-box test-design techniques, regression and mutation testing, object-oriented testing, and the testing of web-based systems.

Source Authority

The two official course files—the Module 4 lecture document and the Module 5 slide deck—are the primary authority for this book. Every topic they contain appears here, in their terminology, with obvious typographical slips corrected. Where the course material states a result tersely (for example, a cyclomatic-complexity value on a slide), the book reproduces the result and then shows the complete working behind it.

Explanations that go beyond the course files are included only where they are needed for understanding or for answering application questions. Such material is placed in boxes headed SUPPLEMENTARY EXPLANATION, or is introduced in the text with the phrase “as a supplementary note”. Examples labelled (*course example*) are taken from the course material; worked examples without that label are additional practice problems constructed for this book, and every one of their numerical results has been checked independently.

Organisation

Chapters 1–5 cover Module 4 and Chapters 6–13 cover Module 5. The end matter is designed for revision: master comparison tables, a formula handbook, a definitions bank, a catalogue of diagrams that examiners expect students to draw, an examination answer bank with model 2-, 5- and 10-mark answers, a graded practice bank, two complete mock examinations with separate answer keys, and a final revision handbook containing mnemonics, examination traps and one-page summary sheets.

Conventions

- Definitions to be learnt verbatim are set in DEFINITION boxes; points that decide marks are set in EXAM TIP boxes; frequent errors are set in COMMON MISTAKE boxes.
- In all cyclomatic-complexity work the book uses *E* for edges, *N* for nodes, *P* for connected components, *D* for decision (predicate) nodes, and *R* for regions. The course slides occasionally reuse the letter *P* for predicate nodes; this is pointed out where it occurs.
- Figures, tables and formulas are numbered by chapter (Figure 8.3 is the third figure of Chapter 8). All entries in the contents lists are hyperlinked.

Suggested Study Plan

A first reading should follow the chapters in order, because Chapters 3 and 4 depend on the concepts of Chapter 2, and Chapters 8 and 9 depend on the coverage ideas of Chapter 7. Numerical skills—cyclomatic complexity, connection matrices, boundary values and mutation scores—should be practised with pen and paper using the worked examples before attempting Section D of the practice bank. In the final days before the examination, the revision handbook at the end of the book, followed by one timed mock paper, gives the most efficient review.

Contents

Preface: How to Use This Book	i
List of Figures	viii
List of Tables	x
List of Formulas	xii
1 Software Design	1
1.1 The Role of Software Design	1
1.1.1 Intuition	1
1.1.2 Position of Design in the Software Process	1
1.2 Design Models and the Flow of Information	2
1.2.1 The Four Design Models in Detail	3
1.3 Importance of Software Design	4
1.4 Software Quality Guidelines and Attributes	4
1.5 Design Principles	4
1.5.1 Avoid Tunnel Vision	5
1.5.2 Traceable to the Analysis Model	5
1.5.3 Do Not Reinvent the Wheel	5
1.5.4 Minimise Intellectual Distance	5
1.5.5 Uniformity and Integration	6
1.5.6 Accommodate Change	6
1.5.7 Graceful Degradation	6
1.5.8 Design Is Not Coding; Coding Is Not Design	6
1.5.9 Assess Quality During Design	6
1.5.10 Review to Minimise Conceptual Errors	6
1.6 The Design Process	6
1.7 Design Concepts: The Questions They Answer	7
1.8 Design versus Coding	7
2 Design Concepts	9
2.1 Abstraction	9
2.1.1 Intuition	9
2.1.2 Types of Abstraction	9
2.1.3 Course Example: A Calculator in C++	9
2.2 Design Patterns	11
2.3 Separation of Concerns	12
2.4 Modularity	12
2.4.1 Course Example: A Modular Calculator in Python	13
2.4.2 Key Benefits of Modularity	14
2.4.3 Criteria for Evaluating Modularity	14
2.4.4 The Balancing Act: Undermodularity and Overmodularity	15
2.5 Refinement	15

2.5.1	The Refinement Process	16
2.5.2	Course Examples	16
2.5.3	Refinement and Partitioning	17
2.6	Refactoring	17
2.6.1	Purpose	17
2.6.2	Identifying the Need for Refactoring	17
2.6.3	Benefits	17
3	Component-Level Design: Cohesion and Coupling	19
3.1	Cohesion	19
3.1.1	Intuition	19
3.1.2	The Cohesion Scale	19
3.1.3	Types of Cohesion	20
3.1.4	Comprehensive Cohesion Table	21
3.1.5	Why Higher Cohesion Is Desirable	22
3.2	Coupling	22
3.2.1	Intuition	22
3.2.2	The Coupling Scale	23
3.2.3	Types of Coupling	23
3.2.4	Comprehensive Coupling Table	25
3.3	High Cohesion and Low Coupling	26
3.3.1	Why the Principle Matters	26
4	Architectural Design	28
4.1	Software Architecture	28
4.2	Architectural Models	28
4.2.1	Course Example: Library Management System	28
4.3	Control Hierarchy	30
4.4	Structural Partitioning	32
4.4.1	Horizontal Partitioning	32
4.4.2	Vertical Partitioning (Factoring)	33
4.5	Logical Data Representation	34
4.6	Architectural Mapping Using Data Flow	34
4.7	Transform Mapping	35
4.7.1	Concept	35
4.7.2	Steps in Transform Mapping	35
4.7.3	Course Example: Payroll System	36
4.8	Transaction Mapping	38
4.8.1	Concept	38
4.8.2	Steps in Transaction Mapping	38
4.8.3	Course Example: ATM System	38
4.8.4	Transform Mapping versus Transaction Mapping	40
5	User Interface Design	42
5.1	The Golden Rules of Interface Design	42
5.1.1	Rule 1: Place the User in Control	42
5.1.2	Rule 2: Reduce the User's Memory Load	42
5.1.3	Rule 3: Make the Interface Consistent	42

5.2	Interface Design Models	43
5.2.1	Categories of Users	43
5.3	The User Interface Design Process	45
5.4	Interface Design Issues	46
5.4.1	System Response Time	46
5.4.2	Help Facilities	46
5.4.3	Error Handling	46
5.4.4	Menu and Command Labelling	47
5.4.5	Application Accessibility	47
5.4.6	Internationalisation	47
5.5	Characteristics of a Good User Interface	47
5.6	User Interface Design Principles	48
6	Software Testing Fundamentals	50
6.1	What Is Software Testing?	50
6.2	Objectives of Testing	50
6.3	The Testing Strategy	51
6.4	Testing Strategies Listed in the Course Material	51
6.5	Types of Testing	52
6.5.1	Static and Dynamic Techniques	53
6.5.2	White-, Black- and Grey-Box Technique Families	53
7	White-Box Testing	55
7.1	Concept	55
7.1.1	Intuition	55
7.1.2	Objectives	55
7.1.3	Types of Defects Identified	55
7.2	Code Coverage	56
7.2.1	Statement Coverage	56
7.2.2	Branch Coverage	56
7.2.3	Path Coverage	56
7.3	Control-Structure Testing: Errors in Conditions	58
8	Basis Path Testing and Cyclomatic Complexity	60
8.1	Basis Path Testing	60
8.2	Control Flow Graphs	60
8.2.1	Terminology	61
8.2.2	Flow-Graph Notation for Structured Constructs	61
8.2.3	Rules for Drawing a CFG	61
8.3	Cyclomatic Complexity	62
8.3.1	Three Methods of Computation	62
8.3.2	Meaning of M	63
8.3.3	The Basis Path Testing Procedure	63
8.4	Worked Examples	63
8.5	Summary of Results	69
9	Graph Matrices and Connection Matrices	71
9.1	Graph Matrices	71
9.2	Connection Matrices	71

9.3	Computing Cyclomatic Complexity from a Connection Matrix	72
9.4	Producing Paths from a Graph Matrix	75
10	Black-Box Testing	76
10.1	Concept	76
10.2	Equivalence Class Partitioning	77
10.3	Boundary Value Analysis	78
10.4	Decision-Table-Based Testing	80
10.5	State Transition Testing	82
11	Regression Testing and Mutation Testing	86
11.1	Regression Testing	86
11.1.1	Why Regression Testing Is Needed	86
11.1.2	Key Characteristics	86
11.1.3	When Regression Testing Is Performed	86
11.1.4	Steps in Regression Testing	87
11.1.5	Selecting Test Cases	87
11.1.6	Advantages and Challenges	87
11.1.7	Regression Testing versus Retesting	88
11.2	Mutation Testing	88
11.2.1	Intuition	88
11.2.2	Terminology	88
11.2.3	How Mutation Testing Works	89
11.2.4	Mutation Score	89
11.2.5	Advantages and Challenges	91
12	Object-Oriented Testing	92
12.1	Why OO Software Needs a Different Approach	92
12.2	Levels and Kinds of OO Testing	92
12.2.1	Class Testing	92
12.2.2	Inheritance Testing	93
12.2.3	Polymorphism Testing	93
12.2.4	Scenario-Based Testing	94
12.3	Challenges in OO Testing	94
12.4	Techniques for OO Testing	94
12.5	Advantages and Challenges	94
13	Testing Web-Based Systems	96
13.1	Why Web Testing Is Distinctive	96
13.2	Types of Web Testing	96
A	Master Comparison Tables	99
B	Formula Handbook	104
C	Definitions Bank	107
D	Important Diagrams	111
E	Examination Answer Bank	113

E.1	2-Mark Answers	113
E.2	5-Mark Answers	114
E.3	10-Mark Answers	115
F	Practice Problems	118
G	Mock Examinations	122
G.1	Mock Examination 1	122
G.2	Mock Examination 2	123
H	Answer Keys	125
H.1	Mock Examination 1 — Answer Key	125
H.2	Mock Examination 2 — Answer Key	126
H.3	Answers to Section D (Practice Problems)	127
I	Final Revision Handbook	129
I.1	Mnemonics and Memory Hooks	129
I.2	Do Not Confuse These	129
I.3	Common Examination Traps	130
I.4	Final-Day Revision Sheet	130
I.5	One-Page Formula Sheet	131
I.6	One-Page Ultra-Fast Revision	132
	Index	133

List of Figures

1.1	Design is the last modelling activity before construction.	1
1.2	Translating the requirements model into the design model (adapted from the course material).	2
1.3	The design process: generate, document and evaluate at three levels; dashed arrows indicate iteration after evaluation.	7
2.1	Modularity and software cost: an optimal number of modules M balances development and integration cost.	15
2.2	Stepwise refinement of a login system (course example).	16
3.1	Cohesion scale used in the course material (lowest to highest).	19
3.2	Coupling scale used in the course material (strongest to weakest).	23
3.3	Left: elements of unrelated concerns scattered across modules with many cross-dependencies. Right: each module groups related elements and communicates through few, narrow connections.	26
4.1	Structural model of the Library Management System.	29
4.2	Dynamic model: sequence diagram for borrowing a book (dashed arrows are replies).	30
4.3	Control hierarchy of the Library Management System (course example).	30
4.4	Horizontal partitioning of the Library Management System into layers (course example).	32
4.5	Vertical partitioning: independent subsystems, with control at the top and workers at the bottom (course example).	33
4.6	Logical data representation for the LMS: a transaction associates a user with a book.	34
4.7	Transform flow is linear through a transform centre; transaction flow fans out from a transaction centre into alternative action paths.	35
4.8	Payroll DFD with flow boundaries (course example redrawn).	36
4.9	First-level factoring of the payroll system.	37
4.10	Program structure for the payroll system after second-level factoring.	37
4.11	ATM DFD showing transaction flow (course example redrawn).	39
4.12	Program structure for the ATM system after transaction mapping.	39
5.1	The four models of interface design.	44
5.2	The UI design process as a spiral of four activities; each pass elaborates the interface further.	45
6.1	Testing strategy: each testing level (upper half) verifies the corresponding development level (lower half).	51
6.2	Three dimensions for classifying testing.	52
6.3	Static and dynamic testing techniques (course taxonomy redrawn).	53
8.1	Flow-graph notation for structured constructs; shaded nodes are predicate nodes.	61

10.1	Equivalence classes for a password length that must be 6 to 12 characters. . . .	77
10.2	Boundary values for a range $[a, b]$: just below, on and just above each boundary, plus a nominal value.	79
10.3	State transition diagram for PIN entry (course example redrawn).	83
10.4	State transition diagram for the login system.	84
11.1	The regression testing cycle.	87
13.1	Tiers of a web-based system and the testing types that target each.	96

List of Tables

1.1	Correspondence between requirements-model elements and design models. . .	3
1.2	The ten design principles.	5
1.3	Design compared with coding.	8
2.1	Types of abstraction.	10
2.2	Criteria for evaluating modularity.	14
3.1	Types of cohesion: definition, example, relatedness, problems and recognition clue.	21
3.2	Types of coupling: definition, example, dependency, problems and recognition clue.	25
4.1	Architectural models.	29
4.2	Measures and relationships in a control hierarchy.	31
4.3	Horizontal versus vertical partitioning.	34
4.4	Transform mapping compared with transaction mapping.	40
5.1	Guidelines for placing the user in control.	43
5.2	Guidelines for reducing the user's memory load.	43
5.3	Guidelines for making the interface consistent.	44
5.4	The four interface design models.	44
5.5	The sixteen user interface design principles.	48
6.1	Testing strategies and their purpose.	51
6.2	Manual versus automated testing.	53
6.3	Functional versus non-functional testing.	54
7.1	Common errors in conditions.	58
8.1	Summary of the worked cyclomatic-complexity examples.	70
10.1	White-box versus black-box testing.	76
10.2	Equivalence partitioning versus boundary value analysis.	80
10.3	Decision-table testing versus state transition testing.	85
11.1	Regression testing versus retesting.	88
11.2	Common mutation operators.	89
12.1	Kinds of object-oriented testing.	92
12.2	Class testing versus integration testing in OO systems.	95
13.1	Types of testing for web-based systems.	96
13.2	Cross-browser versus cross-platform testing.	98
13.3	Accessibility, localisation and internationalisation testing.	98
A.1	Design versus coding.	99

A.2	Abstraction versus modularity.	99
A.3	Modularity versus separation of concerns.	99
A.4	Cohesion versus coupling.	100
A.5	High versus low cohesion.	100
A.6	High versus low coupling.	100
A.7	Horizontal versus vertical partitioning.	100
A.8	Transform versus transaction mapping.	101
A.9	Refactoring versus debugging.	101
A.10	White-box versus black-box testing.	101
A.11	Manual versus automated testing.	101
A.12	Functional versus non-functional testing.	102
A.13	Statement versus branch versus path coverage.	102
A.14	Equivalence partitioning versus boundary value analysis.	102
A.15	Decision table versus state transition testing.	102
A.16	Regression testing versus retesting.	102
A.17	Class testing versus integration testing (OO).	103
A.18	Cross-browser versus cross-platform testing.	103
A.19	Accessibility versus localisation versus internationalisation.	103
D.1	Diagrams to be able to draw.	111
I.1	Mnemonics.	129

List of Formulas

7.1	Statement coverage	56
7.2	Branch coverage	56
7.3	Path coverage	56
8.1	Cyclomatic complexity from edges and nodes	62
8.2	Cyclomatic complexity from decision nodes	62
8.3	Cyclomatic complexity from regions	62
9.1	Cyclomatic complexity from a connection matrix	72
10.1	Number of boundary-value test cases (supplementary)	79
10.2	Number of rules in a complete decision table	81
11.1	Mutation score	89

Software Design

Software design is the activity in which the *what* of a system, captured during requirements analysis, is converted into the *how* of a buildable solution. This chapter explains the role of design in the software process, the four design models produced from the requirements model, the quality attributes a design must satisfy, the principles that guide good design, and the iterative design process described in the course material.

1.1 The Role of Software Design

1.1.1 Intuition

Before a building is constructed, an architect produces drawings that show the structure, the rooms, the services and the way people will move through the space. Construction workers do not improvise the layout; they follow the drawings. Software design plays the same role. It is the blueprint that tells programmers which components to build, how those components communicate, what data they hold and how users will interact with them.

DEFINITION – Software Design

Software design is an iterative process that translates analysed requirements into a blueprint for constructing the software. It begins with a high-level, abstract representation and becomes progressively more detailed, while every refinement remains traceable to the requirements.

1.1.2 Position of Design in the Software Process

The course material identifies three facts about the position of design:

1. Design is a **core part of software engineering** and is applied **regardless of the software process model** in use. Whether a team follows a waterfall, incremental, spiral or agile process, some form of design must take place.
2. Design **begins after the software requirements have been analysed and modelled**. The requirements model is its input.
3. Design is the **final modelling step before construction**, where construction means code generation and testing.

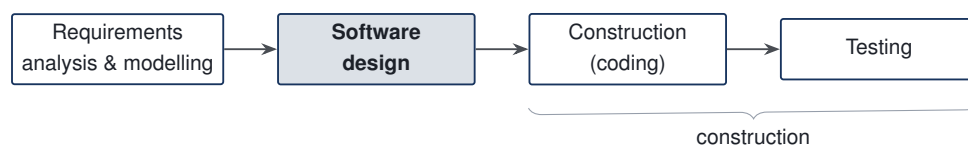


Figure 1.1. Design is the last modelling activity before construction.

1.2 Design Models and the Flow of Information

The requirements model contains four kinds of elements: scenario-based, class-based, flow-oriented and behavioural. Design uses these elements to produce **four essential design models**:

1. **Data/class design**
2. **Architectural design**
3. **Interface design**
4. **Component-level design**

The four models are traditionally drawn as a pyramid (Figure 1.2). Data/class design forms the base because every other design decision depends on the data the system manipulates. Architectural design rests on it, interface design sits above the architecture, and component-level design, the most detailed level, forms the apex.

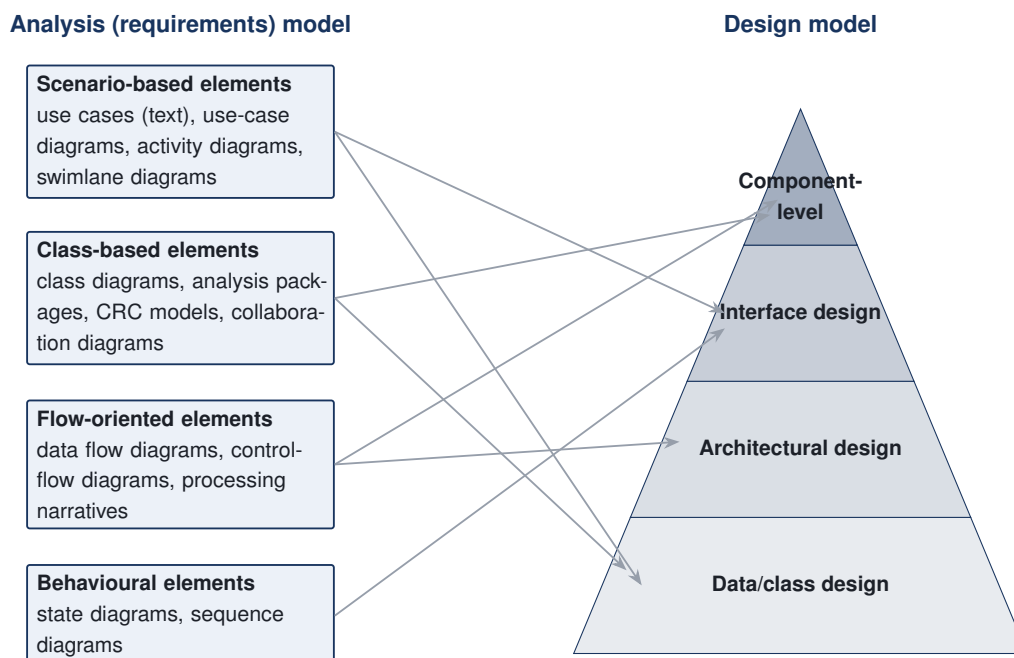


Figure 1.2. Translating the requirements model into the design model (adapted from the course material).

Table 1.1 pairs each requirements element with the design model to which it contributes most directly, following the correspondence presented in the course material.

IMPORTANT NOTE

The row-wise pairing in Table 1.1 is how the course material lays the two lists side by side; it is not a strict one-to-one mapping. In practice every design model draws on several analysis elements, as the multiple arrows in Figure 1.2 show. For example, component-level design uses class-based, flow and behavioural models together.

Table 1.1. Correspondence between requirements-model elements and design models.

Requirements element	What it describes	Design model	What it defines
Scenario-based	User interactions and system behaviour	Data/class design	Data structures and class relationships
Flow-oriented	Flow of data and control through the system	Architectural design	Overall structure: layers, modules and components
Class-based	The system's structure (classes, attributes, collaborations)	Interface design	User interfaces, APIs and interactions with external systems
Behavioural	Dynamic behaviour (states, sequences)	Component-level design	Internal implementation of individual components

1.2.1 The Four Design Models in Detail

1.2.1.1 Data/Class Design

Data/class design converts the analysis class models into *design class realisations* and defines the data structures required for implementation. CRC (Class–Responsibility–Collaborator) diagrams and class attributes are the main inputs. Some class design is carried out together with software architecture design, because the classes that hold key data often determine the architectural components.

1.2.1.2 Architectural Design

Architectural design defines the relationships between the major structural elements of the software. It uses architectural styles and design patterns to achieve the requirements, considers the constraints that affect how the architecture can be implemented, and is derived from the requirements model.

1.2.1.3 Interface Design

Interface design specifies how the software communicates with other systems and with the people who use it. It defines the flow of information (data and control) across the boundary and the associated behaviour. Usage scenarios and behavioural models are its principal inputs.

1.2.1.4 Component-Level Design

Component-level design converts the structural elements of the architecture into procedural descriptions of software components. Class-based, flow and behavioural models are used to define each component's internal algorithms and data.

1.3 Importance of Software Design

The course material lists six reasons why design matters. Each can be stated as a consequence of the blueprint analogy.

1. **Construction and maintainability.** Design directly determines how easily the software can be built and later changed.
2. **Quality.** Design is the place in software engineering where quality is established; it cannot be added after coding.
3. **Assessable representations.** Design produces models that can be reviewed and assessed for quality before any code exists.
4. **Foundation.** Design is the foundation for all subsequent activities: coding, testing, deployment and maintenance.
5. **Faithful translation.** Design accurately translates stakeholder requirements into a functioning system.
6. **Risk prevention.** Without design, the result is a system that is unstable, difficult to test and impossible to assess.

1.4 Software Quality Guidelines and Attributes

Designs are examined through technical reviews. The course material identifies three characteristics that a good design must have:

1. It must implement **all explicit requirements** stated in the requirements model and accommodate all **implicit requirements** expected by stakeholders.
2. It must be a **readable, understandable guide** for those who write the code and for those who test and later support it.
3. It must provide a **complete picture** of the software, addressing the data, functional and behavioural domains from an implementation perspective.

EXAM TIP

For a question on “characteristics of a good design”, the three points above form the core answer. Add the phrase “evaluated through technical reviews” and connect each characteristic to a consequence, e.g. an unreadable design leads to incorrect code because programmers misinterpret it.

1.5 Design Principles

The course material presents ten design principles. They are guidelines for the design *process* and for the design *product*. Table 1.2 summarises them and the subsections that follow explain each.

Table 1.2. The ten design principles.

No.	Principle	Essence
1	Avoid tunnel vision	Consider all requirements and alternatives, not one aspect only.
2	Traceable to the analysis model	Every design element has a reason in the requirements.
3	Do not reinvent the wheel	Reuse libraries, components and proven patterns.
4	Minimise intellectual distance	Structure the software the way the real-world problem is structured.
5	Uniformity and integration	Consistent structure, style and logic across all components.
6	Accommodate change	Modular, scalable structure that absorbs new requirements.
7	Graceful degradation	Bad data or errors must not crash the whole system.
8	Design is not coding; coding is not design	Design plans structure; coding implements it.
9	Assess quality during design	Evaluate efficiency, scalability and correctness while designing.
10	Review to minimise conceptual errors	Peer reviews catch logical flaws before they reach code.

1.5.1 Avoid Tunnel Vision

A designer who concentrates on a single aspect of the problem—say, performance—and ignores others such as security or usability produces an unbalanced design. The principle requires a broad perspective in which all requirements and alternative approaches are considered.

1.5.2 Traceable to the Analysis Model

The design must connect directly to the requirements and analysis models. Every feature in the design should have a clear justification in earlier analysis. Traceability allows reviewers to verify that nothing has been omitted and that nothing unnecessary has been added.

1.5.3 Do Not Reinvent the Wheel

Existing solutions, libraries and design patterns should be used in preference to building everything from scratch. Reuse saves time and yields proven, reliable implementations.

1.5.4 Minimise Intellectual Distance

The structure of the software should mirror, as closely as possible, the structure of the problem in the real world. A library system whose modules correspond to members, books and loans is easier to understand, modify and maintain than one organised around arbitrary technical divisions.

1.5.5 Uniformity and Integration

The design should display a consistent structure, style and logic throughout. Different teams should not work in disconnected ways that produce components which do not fit together.

1.5.6 Accommodate Change

Software should be designed so that new features or changed requirements can be incorporated easily. Modular and scalable approaches provide this flexibility.

1.5.7 Graceful Degradation

When something goes wrong—bad data, errors or unexpected operating conditions—the system should not fail completely. Error handling and fallback mechanisms allow it to continue with reduced functionality.

1.5.8 Design Is Not Coding; Coding Is Not Design

Design concentrates on planning and structure; coding is the actual implementation. Good design guides coding, but the two activities are separate. Even detailed procedural designs are written at a higher level of abstraction than source code.

1.5.9 Assess Quality During Design

Efficiency, scalability and correctness should be evaluated *while* the design is being created. Waiting until the end of development to discover quality problems makes them expensive to correct.

1.5.10 Review to Minimise Conceptual Errors

Regular peer reviews and feedback catch logical flaws early and prevent misunderstandings and incorrect assumptions from propagating into the design.

COMMON MISTAKE

Students often confuse principle 8 with an instruction to avoid coding during design. The principle means that the two activities have different goals and different levels of abstraction; a design may still include pseudocode for complex algorithms.

1.6 The Design Process

The course material depicts the design process as three iterative stages, each producing a document that is evaluated before the next stage begins (Figure 1.3).

The process has the following characteristics:

- **Iterative.** After each evaluation the designer may return to the generation step and improve the document.
- **Top-down.** Work proceeds from the system interface, through the architecture, to the detailed (component) design.

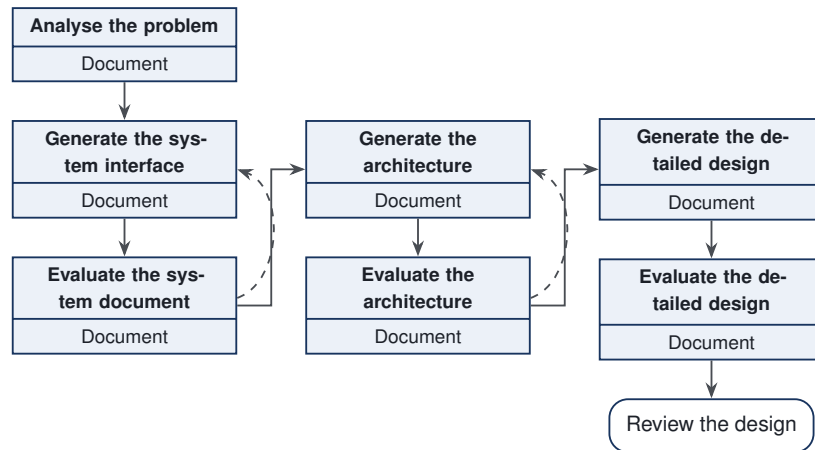


Figure 1.3. The design process: generate, document and evaluate at three levels; dashed arrows indicate iteration after evaluation.

- **Documented.** Every step yields a document that can be assessed, satisfying the importance-of-design point that design produces reviewable representations.
- **Reviewed.** The process ends with a formal review of the complete design.

1.7 Design Concepts: The Questions They Answer

Design concepts, treated in Chapters 2–4, help designers create better designs and apply more advanced design methods. The course material states that they help answer three questions:

1. How should software be divided into smaller components?
2. How can the details of functions and data structures be separated from the overall conceptual representation of the software?
3. What criteria define the quality of a software design?

Abstraction and refinement answer the second question; modularity, separation of concerns, architecture and partitioning answer the first; cohesion, coupling and the design principles answer the third.

1.8 Design versus Coding

QUICK RECAP

- Design is applied in every process model, follows requirements analysis and precedes construction.
- It produces four models: data/class, architectural, interface and component-level design.
- A good design implements explicit and implicit requirements, is readable, and gives a complete picture of data, function and behaviour.
- Ten principles guide design, from avoiding tunnel vision to reviewing for conceptual errors.

Table 1.3. Design compared with coding.

Aspect	Design	Coding
Purpose	Plan the structure, data and behaviour of the solution	Implement the plan in a programming language
Abstraction level	High to medium: models, diagrams, pseudocode	Low: executable statements
Input	Requirements (analysis) model	Design model
Output	Four design models (data, architecture, interface, component)	Source code
Quality assessment	Technical reviews of models	Unit testing, code review
Cost of correcting errors	Low	Higher, and rising in later phases

- The design process generates, documents and evaluates the interface, the architecture and the detailed design iteratively, ending in a design review.

Design Concepts

The course material presents a numbered sequence of fundamental design concepts: (1) abstraction, (2) architecture, (3) design patterns, (4) separation of concerns, (5) modularity, (6)–(7) cohesion and coupling in component design, (8) refinement together with architectural mapping, and (9) refactoring. This chapter treats abstraction, design patterns, separation of concerns, modularity, refinement and refactoring. Architecture and architectural mapping are developed in Chapter 4, and cohesion and coupling in Chapter 3, because each requires extended treatment.

2.1 Abstraction

2.1.1 Intuition

A driver operates a car through a steering wheel, pedals and a gear lever without knowing how fuel injection or the transmission works. The controls are an *abstraction*: they expose what the car does and hide how it does it. Software is full of such controls. A call to `sort(list)` tells the reader what happens without describing the algorithm.

DEFINITION – Abstraction

Abstraction is the process of hiding implementation details and showing only the essential features of a software system. It allows a problem to be considered at different levels, so that developers focus on *what* a system does rather than *how* it is implemented.

At the highest level of abstraction a solution is stated in the language of the problem domain. At lower levels it is described more and more procedurally, until at the lowest level it is stated in a form that can be implemented directly. Abstraction therefore **reduces complexity**: at any moment the designer deals with only the details relevant at the current level.

2.1.2 Types of Abstraction

The course material distinguishes three types of abstraction (Table 2.1).

2.1.3 Course Example: A Calculator in C++

The course material illustrates all three types with a single calculator program.

2.1.3.1 Data Abstraction

Listing 2.1. Data abstraction: the result attribute is hidden.

```
#include <iostream>
using namespace std;

class Calculator {
private:
```

Table 2.1. Types of abstraction.

Type	Definition	Example
Procedural abstraction	Breaking a process into well-defined procedures (functions or methods) that perform specific tasks without exposing their internal implementation.	A function <code>calculateInterest(principal, rate, time)</code> computes interest without revealing the formula it uses.
Data abstraction	Defining data structures so that implementation details are hidden and only the necessary functionality is exposed.	A <code>Car</code> class with attributes <code>speed</code> and <code>color</code> and a method <code>startEngine()</code> , without exposing engine mechanics.
Control abstraction	Hiding the details of control flow and execution logic so that developers focus on high-level tasks rather than low-level control mechanisms.	A for-each loop or iterator replaces manual management of loop counters and termination conditions.

```

1  int result;                                // hidden from direct access
2  public:
3      Calculator() : result(0) {}             // constructor initialises result
4      int getResult() const {
5          return result;                      // public accessor
6      }
7
8      void add(int a, int b);
9      void subtract(int a, int b);
10 };

```

The variable `result` is private; it cannot be read or modified from outside the class. A user obtains the value only through `getResult()`, which abstracts away the internal data structure. If the class later stored the result as a `long` or kept a history of results, callers would be unaffected.

2.1.3.2 Procedural Abstraction

Listing 2.2. Procedural abstraction: callers invoke `add()` and `subtract()` without seeing their logic.

```

1  void Calculator::add(int a, int b) {
2      result = a + b;                        // encapsulates the addition logic
3  }
4  void Calculator::subtract(int a, int b) {
5      result = a - b;                        // encapsulates the subtraction logic
6  }

```

The caller needs to know only the name and parameters of each method.

2.1.3.3 Control Abstraction

Listing 2.3. Control abstraction: a menu-driven driver hides looping and branching.

```

1  void runCalculator() {
2      Calculator calc;
3      int choice, a, b;

```

```

1  while (true) {
2      cout << "\n1. Add\n2. Subtract\n3. Get Result\n4. Exit\n";
3      cout << "Enter your choice: ";
4      cin >> choice;
5      switch (choice) {
6          case 1: cout << "Enter first number: "; cin >> a;
7                  cout << "Enter second number: "; cin >> b;
8                  calc.add(a, b);          break;
9          case 2: cout << "Enter first number: "; cin >> a;
10                 cout << "Enter second number: "; cin >> b;
11                 calc.subtract(a, b);     break;
12          case 3: cout << "Result: " << calc.getResult() << endl; break;
13          case 4: cout << "Exiting...\n"; return;
14          default: cout << "Invalid choice. Please try again.\n";
15      }
16  }
17  }
18  }
19  }
20  }

```

The function `runCalculator()` abstracts the control flow of the program. A caller simply invokes it and obtains a menu-driven interface, without dealing with the loop, the switch statement or the termination logic.

EXAM TIP

When asked to “explain abstraction with its types”, give the definition, then one line and one example per type. The `Calculator` class supplies all three examples in a single program, which is efficient in an answer script: `private result` (data), `add()` (procedural), `runCalculator()` (control).

COMMON MISTAKE

Abstraction and encapsulation are related but not identical. Abstraction is a design concept concerned with *what is exposed*; encapsulation (information hiding) is the mechanism that *prevents access* to what is not exposed, e.g. the `private` keyword.

2.2 Design Patterns

DEFINITION – Design Pattern

A design pattern is a proven solution to a recurring problem in software design. It provides a named structure that can be reused across different projects.

A pattern records the problem, the context in which it occurs and the essential structure of a solution. The course material states that patterns help a designer decide three things:

1. whether the pattern is **applicable** to the current problem;
2. whether it can be **reused**, thereby saving design effort; and
3. whether it can **guide the development** of a similar but functionally or structurally different pattern.

SUPPLEMENTARY EXPLANATION – Illustrative patterns

The course slide on design patterns carries no further text; the following widely used patterns are given as illustrations only.

- **Model–View–Controller (MVC):** separates data (model), presentation (view) and input handling (controller). The course material itself mentions MVC as a framework model (Section 4.2).
- **Singleton:** guarantees that a class has only one instance, e.g. a single configuration manager.
- **Observer:** notifies dependent objects automatically when an object's state changes, e.g. updating several dashboard widgets when a stock price changes.

2.3 Separation of Concerns

DEFINITION – Separation of Concerns

Separation of concerns is the design concept that a complex problem is easier to solve when it is broken into smaller pieces that can each be solved and optimised independently. A *concern* is a specific feature or behaviour specified as part of the requirements.

The justification given in the course material is a divide-and-conquer argument: solving one large problem is harder than solving many smaller problems. Separation of concerns is the reasoning that leads to modularity; modularity (Section 2.4) is its most common structural expression.

IMPORTANT NOTE

Separation of concerns is the *principle*; modularity, abstraction, functional independence and refinement are the *techniques* by which the principle is realised in a design.

2.4 Modularity

DEFINITION – Modularity

Modularity is the practice of breaking a system into smaller, manageable and independent components called modules, which work independently but can be combined to form a complete system.

Software architecture and design patterns both follow modularity principles. A monolithic program—one large block of code—cannot easily be understood by a single reader, because the number of control paths, variables and interactions is too large. Dividing it into modules makes each part intellectually manageable.

2.4.1 Course Example: A Modular Calculator in Python

Listing 2.4. Main module of a modular calculator.

```

1 def main():
2     while True:
3         print("\n1. Add")
4         print("2. Subtract")
5         print("3. Multiply")
6         print("4. Divide")
7         print("5. Exit")
8         choice = input("Enter your choice: ")
9         if choice == '5':
10            print("Exiting...")
11            break
12        num1 = float(input("Enter first number: "))
13        num2 = float(input("Enter second number: "))
14        if choice == '1':
15            result = add(num1, num2); print(f"Result: {result}")
16        elif choice == '2':
17            result = subtract(num1, num2); print(f"Result: {result}")
18        elif choice == '3':
19            result = multiply(num1, num2); print(f"Result: {result}")
20        elif choice == '4':
21            result = divide(num1, num2); print(f"Result: {result}")
22        else:
23            print("Invalid choice. Please try again.")

```

Listing 2.5. Operation modules of the calculator.

```

1 def add(a, b):
2     return a + b
3
4 def subtract(a, b):
5     return a - b
6
7 def multiply(a, b):
8     return a * b
9
10 def divide(a, b):
11     if b == 0:
12         return "Error: Division by zero"
13     return a / b

```

The `main()` function is the *main module*: the entry point and control centre. It contains no arithmetic; it delegates each calculation to a separate function and concentrates on user interaction. Each operation function is a standalone module. `divide()` additionally contains the logic that prevents division by zero, so that concern is handled in exactly one place.

The course material draws five lessons from this example:

- **Separation of concerns:** each operation lives in its own module with a single responsibility.
- **Reusability:** each module can be reused elsewhere in the program or in other projects.
- **Maintainability:** improving division error handling requires changing only `divide()`.

- **Scalability:** a new `power()` module can be added and integrated into `main()` without altering existing modules.
- **Testability:** `add()` can be unit-tested without running the whole program.

2.4.2 Key Benefits of Modularity

1. **Easier to understand:** a system broken into small parts is more manageable.
2. **Improved reusability:** modules can be reused in different projects.
3. **Simpler debugging and maintenance:** faults are fixed inside individual modules without affecting the whole system.
4. **Enhanced scalability:** new features are added as new modules.
5. **Parallel development:** different teams can work on different modules simultaneously.

2.4.3 Criteria for Evaluating Modularity

A design method is judged on how well it supports effective modularity. The course material lists five criteria (Table 2.2).

Table 2.2. Criteria for evaluating modularity.

Criterion	Meaning	Example
Modular decomposability	The system can be divided systematically into subproblems, making problem-solving easier.	A banking system divided into Account Management, Loan Processing and Transactions.
Modular composability	Existing modules can be reused (assembled) in new systems, reducing development time.	A payment-gateway module (e.g. a PayPal API) reused across many e-commerce platforms.
Modular understandability	Each module can be understood as a standalone unit without reference to other modules.	A User Authentication module (login, logout, password reset) understood independently of the Shopping Cart module.
Modular continuity	Small changes in requirements affect individual modules only, not the whole system.	A change in tax rules affects only the Tax module of an invoicing system.
Modular protection	An error inside one module does not propagate to other modules.	In a flight-booking system, a crash in Seat Selection does not affect the Payment module.

EXAM TIP – Mnemonic

DCUCP — “Designers Can Usually Create Programs”: Decomposability, Composability, Understandability, Continuity, Protection. Decomposability is about breaking *down*; composability is about building *up* from existing parts.

2.4.4 The Balancing Act: Undermodularity and Overmodularity

Modularity is not “the more modules, the better”. The course material describes two opposing risks:

- **Undermodularity:** too few modules. Each module is large and complex, approaching a monolithic program that is difficult to manage.
- **Overmodularity:** too many small modules. The effort and cost of integrating them—interfaces, calls and communication—grow.

The cost of developing an individual module falls as the number of modules increases, while the cost of integrating modules rises. Their sum has a minimum at an optimal number of modules M (Figure 2.1). The exact value of M cannot be predicted, but designers should aim for the region near it.

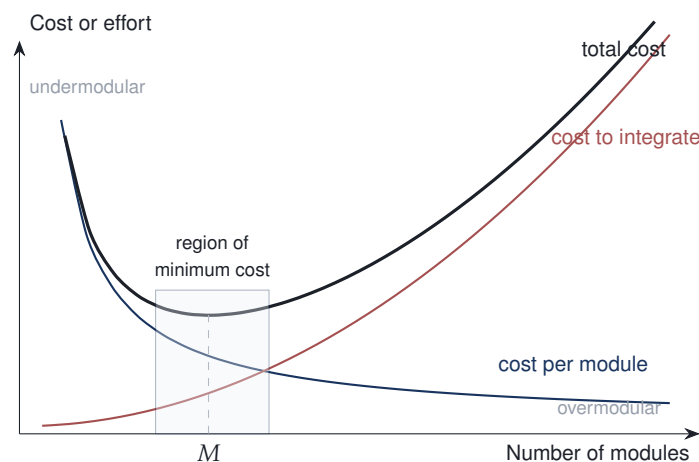


Figure 2.1. Modularity and software cost: an optimal number of modules M balances development and integration cost.

COMMON MISTAKE

Stating that “maximum modularity is best” is incorrect. The expected answer is that there is an optimal number of modules; too few leads to undermodularity and too many to overmodularity.

2.5 Refinement

DEFINITION – Refinement

Refinement (stepwise refinement) is a top-down design strategy in which a complex system or problem is gradually broken into smaller, more manageable components. Procedural detail is elaborated level by level until the design can be implemented in a programming language.

2.5.1 The Refinement Process

Refinement begins at an abstract level and adds detail step by step, moving from high-level concepts to detailed procedural steps. Each level adds specificity to a function or process. This ensures systematic decomposition instead of jumping into low-level coding. The result is a hierarchy in which abstract steps sit at the top and detailed implementation at the bottom; each step is decomposed until it can be coded directly.

IMPORTANT NOTE

Abstraction and refinement are *complementary*. Abstraction lets a designer state a procedure or data item while suppressing low-level detail; refinement reveals that detail progressively as design proceeds.

2.5.2 Course Examples

2.5.2.1 Sorting

Begin with a high-level algorithm (“sort the list”), refine it into specific steps such as divide, compare and swap, and finally implement it as an actual algorithm such as Quicksort or Merge Sort.

2.5.2.2 Login System

Figure 2.2 shows three levels of refinement for user authentication.

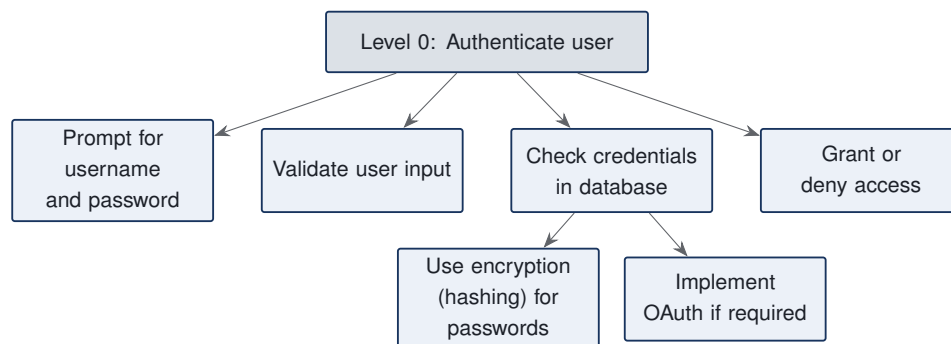


Figure 2.2. Stepwise refinement of a login system (course example).

2.5.2.3 File Processing System

- **High-level task:** process a file.
- **Refinement steps:** open the file; read the content; process the data; save the results.
- **Further breakdown:** “read the content” becomes read line by line and extract metadata; “process the data” becomes apply transformation, validate and filter.

2.5.3 Refinement and Partitioning

The course material relates refinement to requirements analysis: *refinement in design and coding corresponds to partitioning in requirements analysis*. In analysis, high-level requirements are partitioned into functional modules; in design and coding, high-level procedures are refined into detailed implementations.

2.6 Refactoring

DEFINITION – Refactoring

Refactoring is a reorganisation technique that improves the internal structure of a software design or its code without changing its external functionality or behaviour. Martin Fowler describes it as changing a software system so that its internal structure improves while its external behaviour is preserved.

2.6.1 Purpose

- Eliminate redundancy and unused design elements.
- Improve efficiency by optimising algorithms and data structures.
- Enhance readability, maintainability and scalability.

2.6.2 Identifying the Need for Refactoring

A designer examines the existing design for:

- poorly constructed or inappropriate design elements;
- low-cohesion components that perform unrelated functions; and
- unnecessary complexity.

2.6.3 Benefits

- Higher cohesion, because complex components are broken into smaller, well-defined modules.
- Software that is easier to integrate, test and maintain.
- Reduced technical debt, which makes future modifications smoother.

Worked Example 2.1 – Refactoring a low-cohesion component (course example, elaborated)

A component `ReportManager` contains three loosely related functions: it fetches sales records from the database, computes monthly statistics and e-mails a PDF to managers. A change to the e-mail server requires editing and re-testing a component that also contains the statistics logic.

Refactoring. Split it into three components—`SalesRepository` (data access), `SalesStatistics` (computation) and `ReportMailer` (delivery)—each with a single responsibility. The external behaviour (managers receive the same report) is unchanged.

Result. Each component is functionally cohesive (Chapter 3); the statistics can be unit-tested without a mail server; a change to e-mail delivery touches only `ReportMailer`.

SUPPLEMENTARY EXPLANATION – Technical debt

Technical debt is the accumulated future cost of rework caused by choosing expedient but poor design or code now—for example, duplicated logic, low-cohesion modules or missing tests. Like financial debt it accrues “interest”: every later change takes longer. Refactoring repays technical debt by improving structure while keeping behaviour constant.

COMMON MISTAKE – Refactoring is not debugging

Refactoring does not change behaviour, so it does not fix defects. Debugging changes behaviour to remove a fault. If a “refactoring” alters what the program outputs, it is not a refactoring.

QUICK RECAP

- Abstraction hides detail: procedural (functions), data (classes, hidden attributes) and control (loops, iterators, menu drivers).
- A design pattern is a proven, named solution to a recurring design problem.
- Separation of concerns: divide and conquer; a concern is a feature or behaviour.
- Modularity is evaluated by decomposability, composability, understandability, continuity and protection; the optimum lies between under- and overmodularity.
- Refinement is top-down elaboration; it complements abstraction.
- Refactoring improves internal structure without changing external behaviour and reduces technical debt.

Component-Level Design: Cohesion and Coupling

Component-level design defines the internal structure of each module identified by the architecture. The quality of that structure is judged by two complementary measures. *Cohesion* looks inside a module and asks how strongly its elements belong together. *Coupling* looks between modules and asks how strongly they depend on one another. Together they express the concept of functional independence, and the rule *high cohesion, low coupling* is the most frequently examined principle of modular design.

3.1 Cohesion

3.1.1 Intuition

A well-organised kitchen drawer holds only cutlery; a “junk drawer” holds batteries, receipts, string and a spoon. Both are drawers, but only the first can be understood, searched and reorganised easily. A module is highly cohesive when, like the cutlery drawer, everything in it serves one purpose.

DEFINITION – Cohesion

Cohesion is the degree to which the elements inside a module (functions, data and statements) are related to each other and belong together. A highly cohesive module performs a single, well-defined task with minimal dependency on other modules.

High cohesion means that the elements of a module are strongly related and work together towards a single task. **Low cohesion** means that the elements are loosely related or unrelated—randomly grouped functionality—which makes the module harder to understand, maintain and extend.

3.1.2 The Cohesion Scale

The course material ranks five types of cohesion from lowest (least desirable) to highest (most desirable), as shown in Figure 3.1.

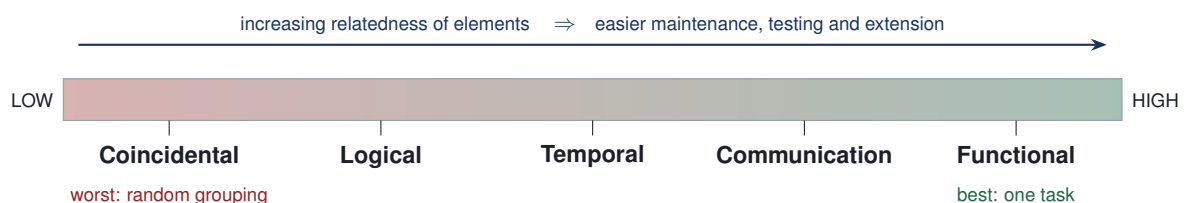


Figure 3.1. Cohesion scale used in the course material (lowest to highest).

SUPPLEMENTARY EXPLANATION – Procedural and sequential cohesion

Many textbooks place two further levels between temporal and functional cohesion: *procedural* cohesion (elements grouped because they follow a fixed order of execution) and *sequential* cohesion (the output of one element is the input of the next). The course material uses the five-level scale shown above; examination answers should follow the course scale and may mention the two additional levels only as an aside.

3.1.3 Types of Cohesion

3.1.3.1 Coincidental Cohesion (Lowest, Undesirable)

Unrelated functions are grouped together at random, usually because of poor design without planning.

```
def random_module():  
    print_report()    # Task 1: print a report  
    clear_screen()    # Task 2: clear the screen  
    play_music()      # Task 3: play music
```

The three tasks have no logical connection. The module is hard to maintain: a change to how music is played risks disturbing the report-printing logic that happens to live beside it.

3.1.3.2 Logical Cohesion

Functions that perform logically similar operations are grouped together, but the specific function executed is selected by an external control, such as a parameter or flag.

```
def output_handler(output_type, data):  
    if output_type == "screen":  
        print_to_screen(data)  
    elif output_type == "printer":  
        send_to_printer(data)  
    elif output_type == "file":  
        write_to_file(data)
```

All branches concern output, but the module's behaviour depends on an external condition. It is harder to test because every value of the flag must be exercised, and adding a new output type requires modifying the shared module.

3.1.3.3 Temporal Cohesion

Functions are grouped because they execute at the same time, not because they are functionally related.

```
def startup_module():  
    initialize_variables()    # Task 1  
    open_files()              # Task 2  
    clear_screen()            # Task 3
```

All three tasks occur at program start-up. If one of them changes, unrelated tasks in the same module may inadvertently be affected, reducing maintainability.

3.1.3.4 Communicational Cohesion

Functions that operate on the same data are grouped into a module. (The course material calls this *communication cohesion*; both names are used.)

```
def temperature_module():
    temp = get_temperature()    # Task 1: retrieve
    log_temperature(temp)       # Task 2: log to a file
    display_temperature(temp)    # Task 3: display on screen
```

The module is more cohesive than the previous types because every function works on the same datum, the temperature. The functions are nevertheless tied together only by shared data, which can make it slightly harder to modify individual parts independently.

3.1.3.5 Functional Cohesion (Highest, Most Desirable)

The module performs a single, well-defined task.

```
def calculate_average(numbers):
    total = sum(numbers)
    count = len(numbers)
    return total / count
```

Every statement contributes to computing one result. The module is easy to maintain, test and extend because it does one thing and does it well.

3.1.4 Comprehensive Cohesion Table

Table 3.1 consolidates the five types in the format required for revision. The “Recognition clue” column gives the question a student should ask when classifying an unfamiliar module.

Table 3.1. Types of cohesion: definition, example, relatedness, problems and recognition clue.

Type	Definition	Example	Relatedness	Problems	Recognition clue
Coincidental (lowest)	Unrelated functions grouped randomly	Print report, clear screen, play music	None	Very hard to understand, maintain or reuse; changes have unexpected side effects	“Why are these together?” — no answer
Logical	Logically similar functions; one is chosen by an external flag	Output handler selecting screen, printer or file	Same general category of activity	Behaviour depends on external control; flag parameters make testing and change harder	A mode/type parameter selects which branch runs
Temporal	Functions grouped because they execute at the same time	Start-up: initialise variables, open files, clear screen	Same point in time only	A change to one task may affect unrelated tasks	Names such as init, startup, shutdown

Type	Definition	Example	Relatedness	Problems	Recognition clue
Communicational	Functions operating on the same data	Get, log and display temperature	Share the same input or output data	Parts are tied by shared data and cannot easily be separated	All functions take or produce the same data item
Functional (highest)	Module performs one well-defined task	Calculate average of a list	Every element contributes to one task	None significant; the desired level	Purpose stated in one sentence with a single verb and object

3.1.5 Why Higher Cohesion Is Desirable

- **Easier to maintain:** the module focuses on one task, so its code is easier to understand and modify.
- **Easier to test:** a clear responsibility gives clear expected results.
- **Easier to extend:** modules are self-contained and focused, so new features carry less risk.

EXAM TIP – The one-sentence test

Try to describe the module in one sentence. If the sentence has a single verb and object (“computes the average”), the module is functionally cohesive. If it needs “and” joining unrelated activities, cohesion is lower; if it needs “at start-up”, it is temporal; if it needs “depending on the flag”, it is logical.

3.2 Coupling

3.2.1 Intuition

Two electrical appliances plugged into standard sockets are loosely coupled: either can be replaced without touching the other. Two appliances wired directly into each other’s circuit boards are tightly coupled: repairing one means dismantling both. Software modules should connect like appliances through sockets—through narrow, well-defined interfaces.

DEFINITION – Coupling

Coupling is the degree of interdependence between different modules or components of a software system. It measures how closely one module is connected to another.

Low coupling is desirable because it makes the system more modular and easier to maintain, test and extend. **High coupling** should be avoided because it increases complexity, reduces flexibility and makes the system harder to modify.

3.2.2 The Coupling Scale

The course material ranks six types of coupling from strongest (worst) to weakest (best), as in Figure 3.2.

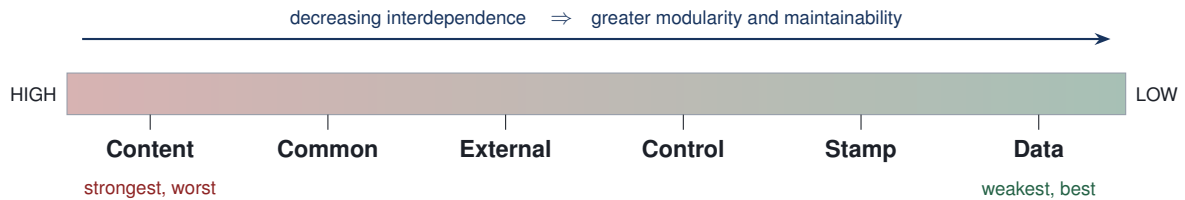


Figure 3.2. Coupling scale used in the course material (strongest to weakest).

3.2.3 Types of Coupling

3.2.3.1 Content Coupling (Strongest, Most Undesirable)

One module directly modifies, or depends on, the internal data or logic of another module.

```
# Module B
class ModuleB:
    def __init__(self):
        self.internal_data = 0

# Module A
def modify_module_b(module_b_instance):
    module_b_instance.internal_data = 42    # modifies B's internals
```

Module A reaches into Module B and changes a variable that B considers internal. This violates encapsulation and information hiding: if B changes its internal structure, A breaks. Debugging is difficult because B's internals are exposed to A.

3.2.3.2 Common Coupling

Several modules share the same global data.

```
global_config = {"theme": "dark", "language": "en"}    # global data

def change_theme(new_theme):                          # Module A
    global_config["theme"] = new_theme

def get_current_theme():                              # Module B
    return global_config["theme"]
```

When Module A changes the theme, Module B behaves differently although the two never interact directly. Such hidden dependencies make the system hard to debug, because a fault seen in one module may be caused by any other module that writes the global data.

3.2.3.3 External Coupling

Modules depend on external systems such as databases, files, devices or network services.

```
import requests

def authenticate_user(username, password):
    response = requests.post("https://api.example.com/auth",
                             data={"username": username,
                                   "password": password})

    return response.json()
```

The application is exposed to changes or failures in the third-party authentication API: if the API changes its interface or becomes unavailable, the system may fail. Flexibility and reliability are reduced.

3.2.3.4 Control Coupling

One module controls the behaviour of another by passing control flags or parameters.

```
def function_b(action):
    if action == "start":
        print("Starting process...")
    elif action == "stop":
        print("Stopping process...")

def function_a():
    function_b("start")    # A decides what B does
```

If `function_b` changes its internal logic or the meaning of the flag, `function_a` must also change. The caller needs knowledge of the callee's internal decisions. Note that the *callee* in control coupling usually exhibits *logical* cohesion.

3.2.3.5 Stamp Coupling

Modules pass complex data structures (objects or records) when only part of the data is required.

```
class Student:
    def __init__(self, id, name, age):
        self.id, self.name, self.age = id, name, age

def process_student(student):
    print(f"Processing student with ID: {student.id}")

def main():
    student = Student(1, "Alice", 20)
    process_student(student)    # entire object passed; only id used
```

`process_student` needs only the identifier but receives the whole object, creating an unnecessary dependency on the structure of `Student`. If that class changes, `process_student` may need updating.

3.2.3.6 Data Coupling (Weakest, Most Desirable)

Modules share only the necessary data, as elementary parameters in function calls.

```
def greet_user(name, age):
    print(f"Hello, {name}! You are {age} years old.")

def main():
    greet_user("Alice", 20)    # only the required values are passed
```

Each module is independent and shares the minimum data required, which makes the system easier to maintain, test and extend.

3.2.4 Comprehensive Coupling Table

Table 3.2. Types of coupling: definition, example, dependency, problems and recognition clue.

Type	Definition	Example	Dependency	Problems	Recognition clue
Content (strongest)	One module modifies internal data or logic of another	A sets B.internal_data = 42	On the other module's internals	Violates information hiding; B's internal change breaks A	Direct access to another module's private members
Common	Modules share global data	Many modules read and write global_config	On shared global variables	Hidden side effects; hard debugging	global variables, shared configuration dictionaries
External	Modules depend on external systems (database, file, network service, device)	Authentication via third-party API	On an externally imposed format or protocol	Vulnerable to external change or outage	HTTP calls, file formats, device protocols
Control	One module passes a flag that controls another's behaviour	function_b("start")	On the callee's internal decision logic	Changes to flag meaning ripple to callers	A mode/action parameter chooses the callee's path
Stamp	A whole data structure is passed when only part is needed	Entire Student passed; only id used	On the structure of the record	Unnecessary dependency; record changes ripple	Object or record parameter mostly unused
Data (weakest)	Only required elementary data passed	greet_user(name, age)	On the values passed only	Minimal; desired level	Small list of simple parameters, all used

EXAM TIP – Mnemonic for coupling order

From worst to best: Can Computers Ever Control Software Data? — Content, Common, External, Control, Stamp, Data. For cohesion from worst to best: Cats Like Tuna Canned Fresh — Coincidental, Logical, Temporal, Communicational, Functional.

3.3 High Cohesion and Low Coupling

DEFINITION – Functional independence

A module is functionally independent when it has a single, well-defined purpose (high cohesion) and a minimal, simple interface to other modules (low coupling). The design goal “high cohesion, low coupling” is the practical statement of functional independence.

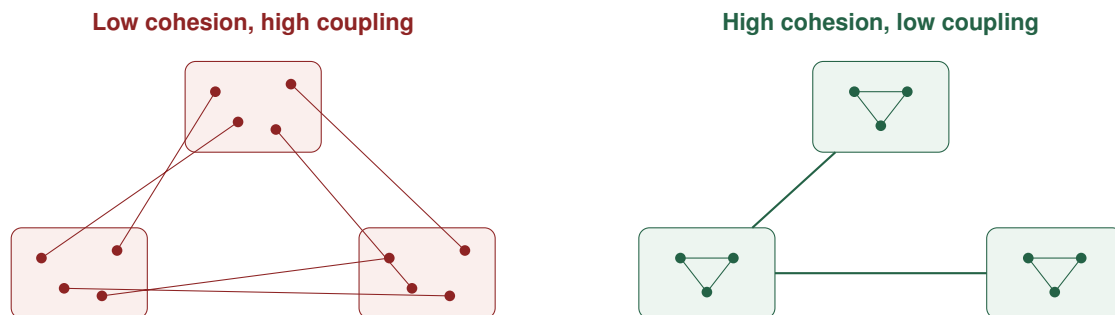


Figure 3.3. Left: elements of unrelated concerns scattered across modules with many cross-dependencies. Right: each module groups related elements and communicates through few, narrow connections.

3.3.1 Why the Principle Matters

The two properties reinforce each other. When related elements are placed together (high cohesion), fewer connections are needed to other modules (low coupling). When modules communicate through few, simple parameters (low coupling), each can be understood in isolation, which makes it natural to keep only related elements inside it (high cohesion). The consequences are concrete:

1. **Localised change.** A requirement change affects one module; this is *modular continuity* (Section 2.4.3).
2. **Contained errors.** A fault in one module is unlikely to propagate; this is *modular protection*.
3. **Independent testing.** Each module can be unit-tested with simple stubs and drivers because its interface is small.
4. **Reuse.** A functionally cohesive, data-coupled module can be lifted into another system; this is *modular composability*.
5. **Parallel development.** Teams can work on different modules with little coordination.
6. **Readability.** Each module can be understood on its own; this is *modular understandability*.

Worked Example 3.1 – Classifying cohesion and coupling

Classify each fragment.

1. `def end_of_day(): close_db(); backup_logs(); send_summary_email()`
2. `def validate_email(address): ... return True/False`
3. `shipping_cost(order)` uses only `order.weight` from an `Order` object with twenty fields.
4. `render(report, fmt)` with `fmt` in `{"pdf", "csv", "html"}` and a separate branch for each.
5. Two modules both read and write a global variable `current_user`.

Solution.

1. *Temporal cohesion*: the three tasks are grouped only because they run at the end of the day.
2. *Functional cohesion*: a single, well-defined task.
3. *Stamp coupling*: an entire record is passed although one field is needed. Improve by passing `order.weight` (data coupling).
4. Inside `render`: *logical cohesion*. Between the caller and `render`: *control coupling*, since `fmt` selects the behaviour.
5. *Common coupling* through shared global data.

COMMON MISTAKE

- Reversing the scales: *functional* is the *highest* cohesion; *data* is the *lowest* (best) coupling; *content* is the strongest (worst) coupling.
- Writing “high coupling is desirable because modules cooperate”. Cooperation through narrow interfaces is still *low* coupling.
- Confusing communicational cohesion (functions inside *one* module sharing data) with common coupling (*different* modules sharing *global* data).

QUICK RECAP

- Cohesion (inside a module), lowest to highest: coincidental, logical, temporal, communicational, functional.
- Coupling (between modules), strongest to weakest: content, common, external, control, stamp, data.
- The design goal is *high cohesion and low coupling*, i.e. functional independence, which yields localised change, contained errors, easy testing and reuse.

Architectural Design

Architecture is the second design concept in the course sequence and the subject of the second level of the design pyramid. This chapter defines software architecture, presents the five architectural models, develops the structural concepts of control hierarchy and partitioning, and then teaches the two examination procedures by which a data flow diagram (DFD) is mapped into a program structure: *transform mapping* and *transaction mapping*.

4.1 Software Architecture

DEFINITION – Software Architecture

Software architecture is the overall structure of the software: its components and the manner in which those components interact. It provides the framework within which detailed design activities are carried out.

Architectural patterns help solve common design problems at this level. The course material identifies three key properties that an architectural design must specify:

1. **Structural properties:** the system components (modules, objects, filters) and the ways in which they are packaged and interact.
2. **Extra-functional properties:** how the architecture achieves requirements for performance, security, reliability, adaptability and similar characteristics.
3. **Families of related systems:** the architecture should draw on repeatable patterns that are common to families of similar systems.

4.2 Architectural Models

An architectural design can be represented by one or more of five kinds of model (Table 4.1).

EXAM TIP – Mnemonic

Some Frameworks Do Process Functions: Structural, Framework, Dynamic, Process, Functional.

4.2.1 Course Example: Library Management System

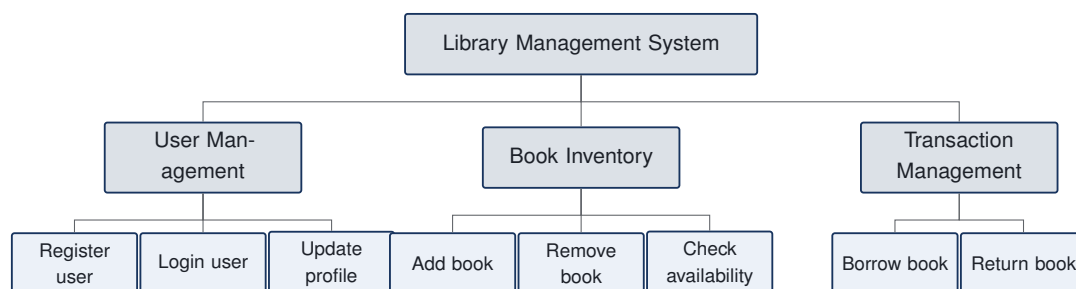
The course material applies each model to a single system, a Library Management System (LMS).

4.2.1.1 Structural View

The LMS is divided into three modules with specific responsibilities: *User Management* handles registration, login and profile updates; *Book Inventory* manages books, their details and availability; and *Transaction Management* handles borrowing and returning (Figure 4.1).

Table 4.1. Architectural models.

Model	Definition	Example
Structural model	Represents the architecture as an organised collection of program components (modules, objects, filters) and their interactions.	Layered web application: presentation layer → business-logic layer → data-access layer.
Framework model	Raises the level of abstraction by identifying repeatable architectural design frameworks found in similar applications.	MVC framework in web development: the model manages data, the view handles the UI and the controller processes input.
Dynamic model	Addresses the behavioural aspects of the architecture, showing how the structure or configuration changes in response to external events.	A real-time traffic-monitoring system that adjusts signals dynamically from live traffic data.
Process model	Represents the business or technical processes that the system must accommodate, focusing on how tasks are executed.	Order processing: order placement → payment → inventory check → shipping.
Functional model	Represents the functional hierarchy of the system, showing how functions interact and depend on one another.	Banking: “Manage Accounts” decomposed into “Create Account”, “Deposit Money” and “Withdraw Money”.

**Figure 4.1.** Structural model of the Library Management System.

4.2.1.2 Framework View

A web-application framework such as Django (Python) or Spring Boot (Java) could be used to build the LMS. The framework supplies pre-built components for common tasks: *authentication* (user login and registration), *database access* through object-relational mapping (managing book inventory and user data), *URL routing* (handling borrow and return requests) and *templates* (HTML views).

4.2.1.3 Dynamic View

The dynamic model is expressed through behaviour over time, for example a sequence of messages for borrowing a book (Figure 4.2) or the equivalent data flow.

The course material gives the corresponding data flow for the same scenario: the *User* sends a borrow request to the *System*; the *System* checks availability with *Book Inventory*; *Book Inven-*

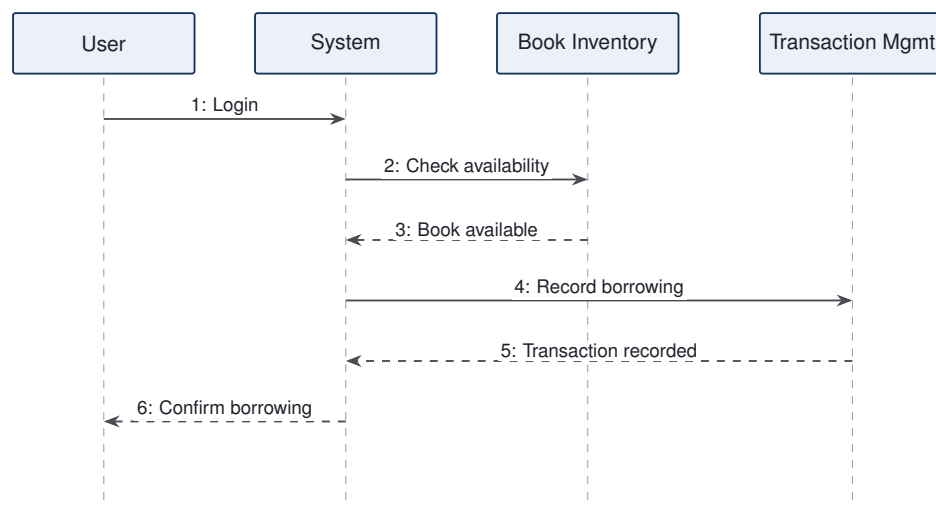


Figure 4.2. Dynamic model: sequence diagram for borrowing a book (dashed arrows are replies).

tory's status update flows to *Transaction Management*; and *Transaction Management* returns a borrowing confirmation to the *User*.

4.2.1.4 Functional View

The functional model lists what the system does without concern for how: *Register User*, *Login User*, *Search Books* (by title, author or genre), *Borrow Book* (if available) and *Return Book*.

4.3 Control Hierarchy

DEFINITION – Control Hierarchy

Control hierarchy, also called program structure, is the organisation of program components (modules) that shows how control flows from one module to another: higher-level components manage and delegate tasks to lower-level components.

Control hierarchy is usually drawn as a tree-like *structure chart*. It does not show the sequence of processing, the order of decisions or the repetition of operations; it shows only which module invokes which. The course material defines six measures and relationships (Table 4.2).

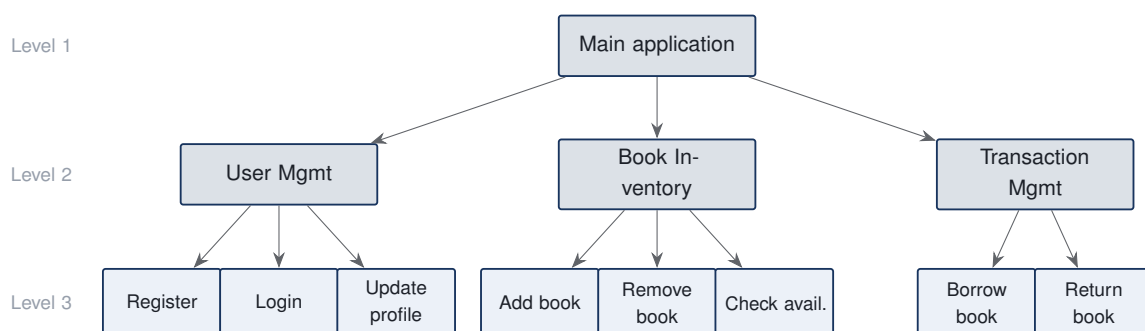


Figure 4.3. Control hierarchy of the Library Management System (course example).

Table 4.2. Measures and relationships in a control hierarchy.

Term	Meaning
Depth	The number of levels of control in the hierarchy.
Width	The overall span of control: the number of modules at a level; the width of the structure is the largest such number.
Fan-out	The number of modules directly controlled by a given module.
Fan-in	The number of modules that directly control a given module.
Superordinate module	A module that controls another module.
Subordinate module	A module that is controlled by another module.

In the course example, the *Main Application* orchestrates the system. When a user borrows a book, the Main Application calls the Transaction Management module, which in turn interacts with the Book Inventory module to check availability.

Worked Example 4.1 – Measuring the LMS control hierarchy

Using Figure 4.3:

- **Depth** = number of levels = 3 (Main; three subsystem modules; eight worker modules).
- **Width** = largest number of modules on any level = 8 (level 3).
- **Fan-out**: Main = 3; User Mgmt = 3; Book Inventory = 3; Transaction Mgmt = 2; every worker module = 0.
- **Fan-in**: every module other than Main has fan-in 1; Main has fan-in 0.
- Main is *superordinate* to User Mgmt, Book Inventory and Transaction Mgmt; each of these is *subordinate* to Main and superordinate to its own workers.

Worked Example 4.2 – Fan-in with a shared utility module

A structure chart has root A controlling B, C and D. B controls E and F; C controls F and G; D controls G. Find depth, width, and fan-in and fan-out of every module.

Levels. Level 1: {A}; level 2: {B, C, D}; level 3: {E, F, G}. Hence depth = 3 and width = $\max(1, 3, 3) = 3$.

Module	A	B	C	D	E	F	G
Fan-out	3	2	2	1	0	0	0
Fan-in	0	1	1	1	1	2	2

Check. The sum of all fan-outs equals the sum of all fan-ins, because each is a count of the same arrows: $3 + 2 + 2 + 1 = 8$ and $1 + 1 + 1 + 1 + 2 + 2 = 8$.

Interpretation. F and G have fan-in 2; they are reused utilities, which is favourable provided they are functionally cohesive.

SUPPLEMENTARY EXPLANATION – Design heuristics for fan-in and fan-out

A widely used heuristic is to keep fan-out moderate (very high fan-out suggests that a module makes too many decisions and should be factored with an intermediate control layer) and to strive for fan-in as depth increases (reusable lower-level modules). Some texts count depth as the number of levels *below* the root; this book follows the course definition (number of control levels), so a single root module alone has depth 1. State the convention used in an answer.

COMMON MISTAKE

Fan-in is *incoming* control (who calls me); fan-out is *outgoing* control (whom I call). Reversing them is the most common error in structure-chart questions.

4.4 Structural Partitioning

If the architectural style of a system is hierarchical, the program structure can be partitioned both horizontally and vertically.

4.4.1 Horizontal Partitioning

DEFINITION – Horizontal Partitioning

Horizontal partitioning splits the software into separate branches of the hierarchy for each major function. In the simplest form there are three partitions: *input* (data collection and preprocessing), *process* (core computation and logic) and *output* (presentation of results). Equivalently, the system is divided into layers or tiers by functionality, such as presentation, business logic and data storage.

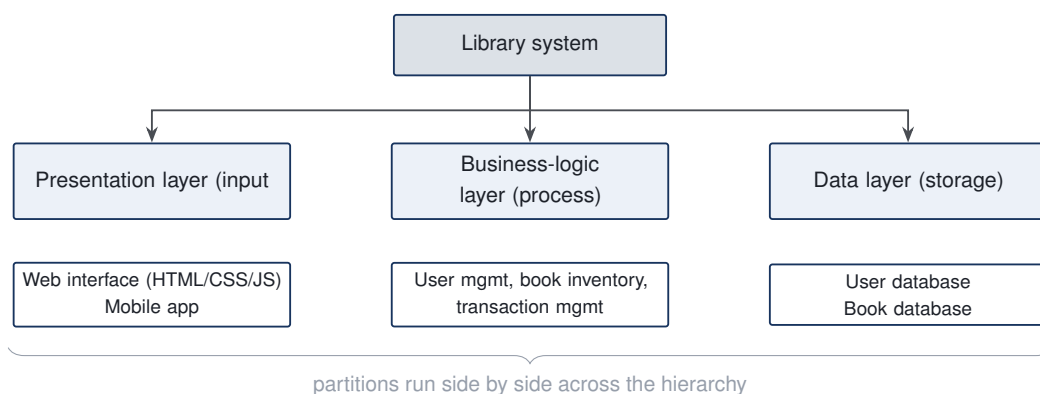


Figure 4.4. Horizontal partitioning of the Library Management System into layers (course example).

The *presentation layer* handles user input and displays results; the *business-logic layer* holds the core logic (for example, checking that a book is available before it is borrowed); the *data layer* stores and retrieves user profiles and inventory. The UI can be changed without affecting business logic or storage.

Merits: easier to test and debug; easier to maintain and modify; fewer side effects, because errors do not spread easily; extensible, because new functions are easy to add. **Demerits:** more data must be passed across module interfaces; control of program flow becomes more complex.

4.4.2 Vertical Partitioning (Factoring)

DEFINITION – Vertical Partitioning

Vertical partitioning, also called factoring, organises the program structure top-down so that control and decision-making reside in the upper-level modules, while the lower-level modules are *workers* that perform input, computation and output. The system is thereby divided into independent functional units or subsystems, each responsible for a specific part of the functionality.

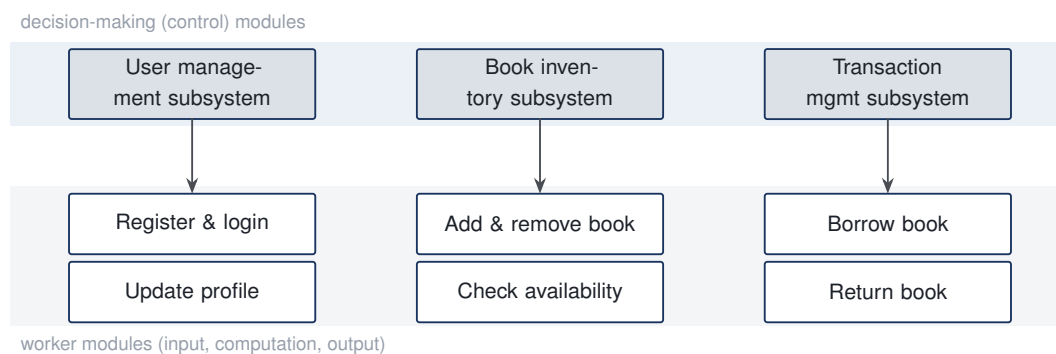


Figure 4.5. Vertical partitioning: independent subsystems, with control at the top and workers at the bottom (course example).

Effects of change: a change in a *control* module affects all its subordinates; a change in a *worker* module has minimal impact on other modules. Because most requirement changes affect input, computation or output, a vertically partitioned (factored) structure is easier to maintain. The subsystems communicate through well-defined interfaces but are otherwise decoupled, which supports scalability and modularity.

IMPORTANT NOTE

The course material describes each form of partitioning in two ways. Horizontal partitioning is presented both as *input–process–output branches* and as *layers or tiers*; vertical partitioning is presented both as *factoring (control on top, workers below)* and as *independent functional subsystems*. In an examination, give the defining characteristic first (branches for major functions versus top-down factoring of control), then the course’s LMS example.

Table 4.3. Horizontal versus vertical partitioning.

Aspect	Horizontal partitioning	Vertical partitioning (factoring)
Basis of division	Major functions or layers (input, process, output; presentation, logic, data)	Level of control (decision-makers on top, workers at bottom); functional subsystems
LMS example	Presentation, business-logic and data layers	User management, book inventory and transaction subsystems
Main merit	Easy testing, maintenance, extension; fewer side effects	Changes in workers have minimal impact; scalability
Main demerit	More data passed between modules; complex flow control	A change in a control module affects all subordinates

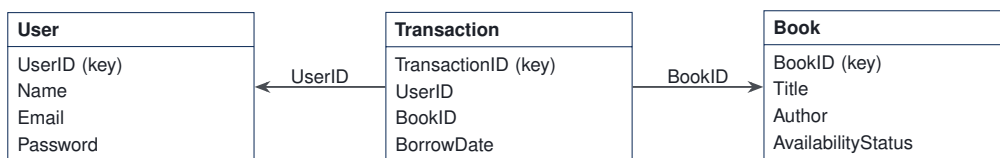
4.5 Logical Data Representation

DEFINITION – Logical Data Representation

Logical data representation defines how data elements relate to each other: how data is structured and organised within the system in terms of logical relationships between entities, independently of physical storage.

It specifies four things: **data organisation** (how data is structured), **methods of access** (how data is retrieved), **associations** (how data items are connected) and **processing alternatives** (different ways of handling the data).

In the LMS, the **User** entity has UserID, Name, Email and Password; the **Book** entity has BookID, Title, Author and AvailabilityStatus; and the **Transaction** entity has TransactionID, UserID, BookID and BorrowDate. A transaction links a user to a book, recording that the user has borrowed it. This logical view is the same whether the data is physically kept in a relational database, a NoSQL store or flat files, which keeps the data well organised and consistent.

**Figure 4.6.** Logical data representation for the LMS: a transaction associates a user with a book.

4.6 Architectural Mapping Using Data Flow

Requirements models for many systems include data flow diagrams. *Architectural mapping* is the method that converts a DFD into a program structure (call-and-return architecture). The course material states a six-step process:

1. **Identify the type of information flow:** transform flow or transaction flow.
2. **Define flow boundaries:** where data enters and leaves the system's processing core.

3. **Map the DFD into the program structure:** convert the DFD into an organised software architecture.
4. **Define the control hierarchy:** establish the decision-making structure.
5. **Refine the design using design heuristics:** optimise for functional independence—high cohesion and minimal coupling.
6. **Elaborate the architectural description:** improve and document the architecture.

Two kinds of information flow govern the detailed procedure: *transform flow*, handled by transform mapping, and *transaction flow*, handled by transaction mapping.

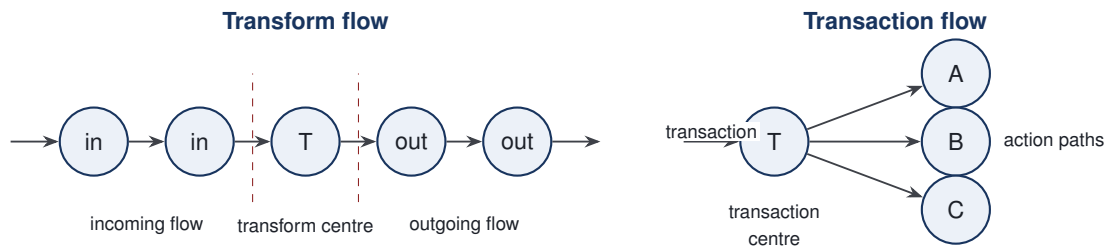


Figure 4.7. Transform flow is linear through a transform centre; transaction flow fans out from a transaction centre into alternative action paths.

4.7 Transform Mapping

4.7.1 Concept

DEFINITION – Transform Flow

Transform flow is information flow in which data enters the system along incoming paths, is converted from an external form to an internal representation, is processed at a *transform centre*, and leaves the system along outgoing paths after conversion back to an external form. The overall flow is linear and sequential.

DEFINITION – Transform Mapping

Transform mapping is the set of design steps that converts a DFD exhibiting transform flow into a program structure with separate input, transform (processing) and output branches under a main controller.

The characteristics of transform flow listed in the course material are: a *linear* flow of data processing; conversion of input data into an internal representation; processing at a transform centre, which holds the core logic; and conversion of the output back to an external representation.

4.7.2 Steps in Transform Mapping

1. **Identify the information flow:** confirm that the DFD exhibits transform flow.
2. **Define the flow boundaries:** locate the incoming boundary (where external data has become internal) and the outgoing boundary (where internal results begin to be converted to external form).
3. **Identify the transform centre:** the bubbles between the two boundaries.

4. **Perform first-level factoring:** create a top-down structure in which the *top-level* component makes decisions, *middle-level* components manage control (an input controller, a transform controller and an output controller), and *low-level* components perform input, computation and output.
5. **Perform second-level factoring:** map each individual DFD bubble into a software module beneath the appropriate controller.
6. **Refine the architecture:** improve cohesion, reduce coupling and enhance maintainability.

4.7.3 Course Example: Payroll System

4.7.3.1 Step 1: The DFD and Its Flow Type

The payroll DFD (Figure 4.8) receives working hours from an *Employee*, fetches employee details from a data store, and passes the data linearly through five processes. There is one input stream and one path through the system, so the flow is a **transform flow**.

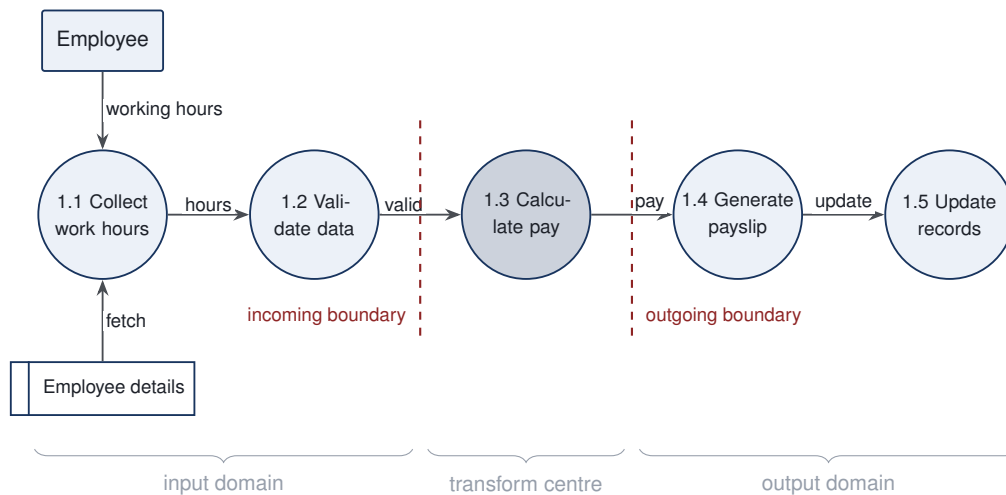


Figure 4.8. Payroll DFD with flow boundaries (course example redrawn).

4.7.3.2 Steps 2–3: Boundaries and Transform Centre

Before *Validate Data* completes, the data is still in its external (raw) form; after validation it is internal, trusted data. The **incoming boundary** is therefore drawn after *Validate Data*. *Generate Payslip* begins converting results into an external document, so the **outgoing boundary** is drawn before it. Hence:

- **Input domain:** Collect Work Hours, Validate Data.
- **Transform centre:** Calculate Pay.
- **Output domain:** Generate Payslips, Update Records.

4.7.3.3 Step 4: First-Level Factoring

A main controller, *Payroll System*, is created at the top. Beneath it are three controllers, one per domain (Figure 4.9).

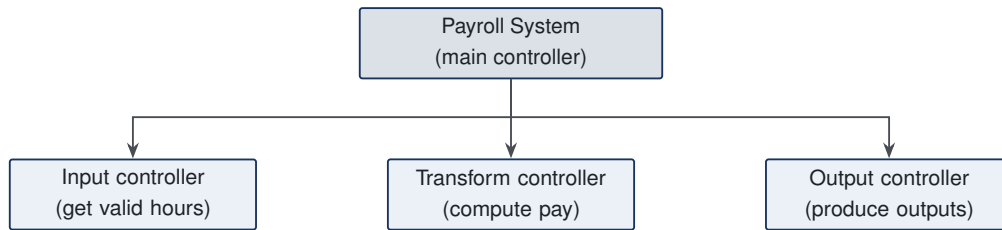


Figure 4.9. First-level factoring of the payroll system.

4.7.3.4 Step 5: Second-Level Factoring

Each bubble becomes a module beneath the controller of its domain (Figure 4.10). This is the structure given in the course material: input modules *Collect Work Hours()* and *Validate Data()*, transformation-centre module *Calculate Pay()*, and output modules *Generate Payslips()* and *Update Records()*.

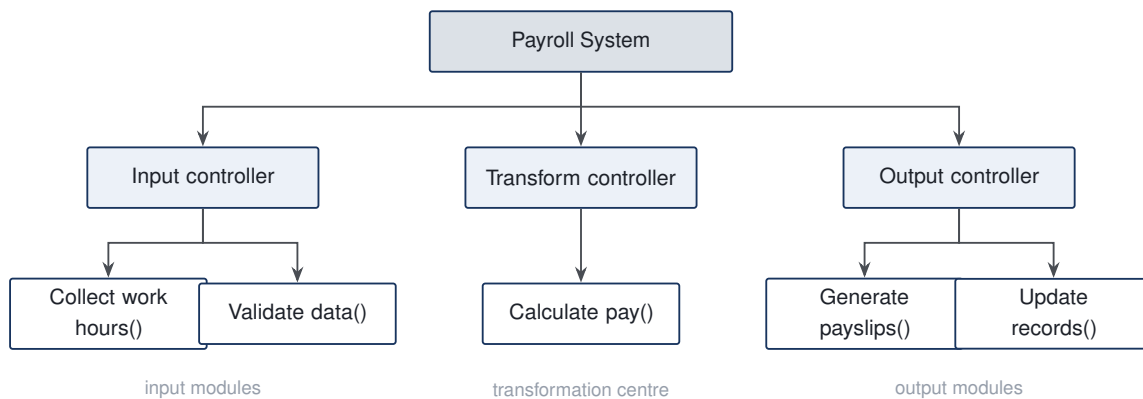


Figure 4.10. Program structure for the payroll system after second-level factoring.

4.7.3.5 Step 6: Refinement

The structure is now examined against the heuristics of Chapter 3:

- Each worker module performs one task (functional cohesion) and receives only the data it needs, e.g. *Calculate Pay* receives validated hours and rate (data coupling).
- If *Collect Work Hours* and *Validate Data* are always used together and are small, they may be merged into a single “Get Valid Hours” module to reduce the number of interfaces; if *Calculate Pay* is complex, it may be factored further into *Compute Gross Pay*, *Compute Deductions* and *Compute Net Pay*.
- Controllers make decisions; workers do not call controllers. This keeps control coupling pointing downwards only.

EXAM TIP – Transform mapping checklist

1. Draw or copy the DFD; state that the flow is transform (linear, single stream).
2. Mark the incoming and outgoing boundaries with dashed lines and name the three domains.
3. Draw first-level factoring: main controller over input, transform and output controllers.

4. Draw second-level factoring: every bubble becomes a module under its controller.
5. Refine: comment on cohesion, coupling, merging or further factoring.

Marks are commonly lost for omitting the boundaries or for drawing the DFD bubbles themselves as the final architecture without controllers.

4.8 Transaction Mapping

4.8.1 Concept

DEFINITION – Transaction Flow

Transaction flow is information flow in which a single data item, the *transaction*, triggers one of several alternative flows of processing. A *transaction centre* evaluates the transaction and routes it to the appropriate action path.

DEFINITION – Transaction Mapping

Transaction mapping is the set of design steps that converts a DFD exhibiting transaction flow into a program structure consisting of an incoming (reception) branch and a dispatch branch in which a transaction dispatcher controls one subordinate module per action path.

According to the course material, in transaction flow the system's behaviour is triggered by an external transaction (a user input or an event); the system branches into different execution paths depending on the type of transaction; and a transaction centre controls decision-making and routes requests accordingly.

4.8.2 Steps in Transaction Mapping

1. **Identify the transaction centre:** the process that controls branching, e.g. a request handler or menu process.
2. **Define the input transactions:** the external triggers that activate different processing paths.
3. **Establish the processing paths:** each transaction type has its own execution flow (action path).
4. **Map the DFD to the program structure:** create a dispatcher module and assign one module to each transaction type.
5. **Refine the software architecture:** optimise for independence, modularity and scalability; factor each action path further where needed.

4.8.3 Course Example: ATM System

4.8.3.1 The DFD

A *User* supplies a menu choice to the *Menu Selection Process*. Depending on the selection, control and data flow to *Withdraw*, *Deposit* or *Balance Enquiry* (Figure 4.11). One input triggers one of several mutually exclusive paths, so the flow is a **transaction flow**, and the Menu Selection Process is the **transaction centre**.

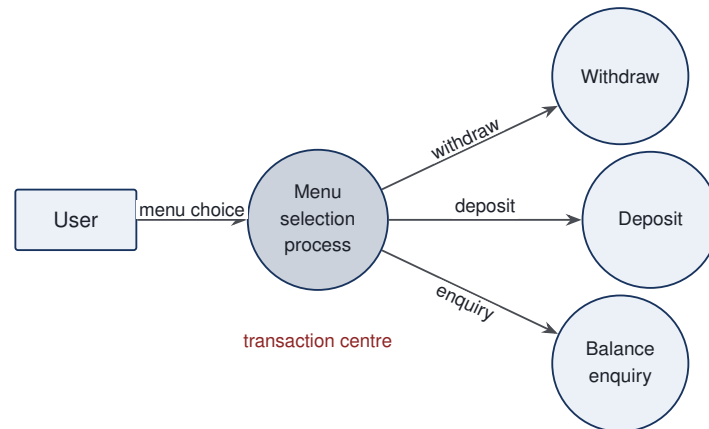


Figure 4.11. ATM DFD showing transaction flow (course example redrawn).

4.8.3.2 Mapping

The transaction centre becomes a *transaction dispatcher*; each action path becomes a subordinate module. The resulting structure in the course material is shown in Figure 4.12, together with a reception branch that obtains the menu selection.

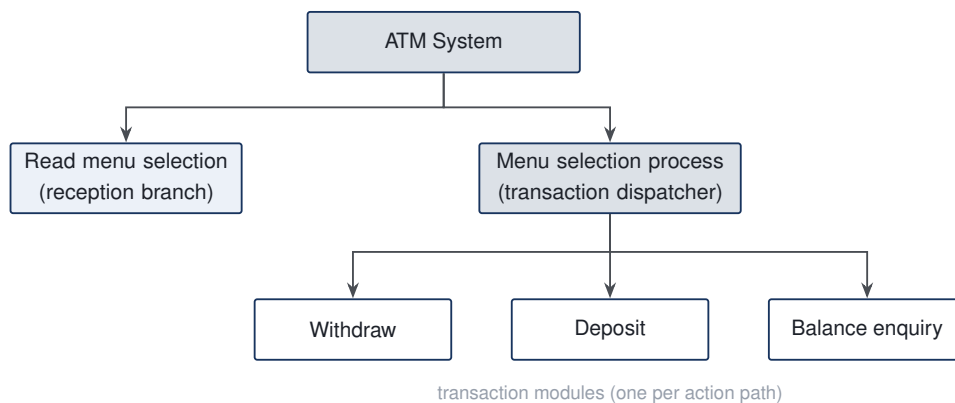


Figure 4.12. Program structure for the ATM system after transaction mapping.

4.8.3.3 Refinement

Each transaction module is factored further into its own action path. For example, *Withdraw* controls *Get Amount*, *Check Balance*, *Dispense Cash* and *Update Account*; *Deposit* controls *Accept Cash*, *Verify Amount* and *Update Account*. The shared module *Update Account* acquires a fan-in of 2, which is good reuse provided it is functionally cohesive. Adding a new transaction, such as *Transfer Funds*, requires only a new subordinate of the dispatcher, demonstrating scalability.

IMPORTANT NOTE

An action path within a transaction-flow system may itself exhibit transform flow. For example, *Withdraw* reads an amount (input), checks and computes the new balance (transform) and dispenses cash and a receipt (output). Such a path is mapped with transform mapping beneath the dispatcher.

EXAM TIP – Transaction mapping checklist

1. State that the flow is transaction flow: one input, several alternative paths.
2. Name the transaction centre and the input transaction.
3. Draw the top module, the reception branch and the dispatcher.
4. Place one module per transaction type under the dispatcher.
5. Refine each action path; comment on fan-in of shared modules and ease of adding transactions.

4.8.4 Transform Mapping versus Transaction Mapping**Table 4.4.** Transform mapping compared with transaction mapping.

Aspect	Transform mapping	Transaction mapping
Applicable flow	Transform flow: linear, sequential	Transaction flow: branching on transaction type
Key element	Transform centre (core processing)	Transaction centre (decision and routing)
Boundaries	Incoming and outgoing flow boundaries	Reception path and action paths
Top structure	Main controller over input, transform and output controllers	Main controller over reception branch and dispatcher
Factoring	First-level then second-level factoring	One module per transaction, then factoring of each path
Course example	Payroll system	ATM system
Adding a feature	Usually a new module in one domain	A new module under the dispatcher

Worked Example 4.3 – Choosing the mapping

For each system, identify the flow type and the centre.

1. An exam-result processor reads marks, validates them, computes grades and GPA, and prints mark sheets.
2. A library kiosk offers *Issue*, *Return*, *Renew* and *Search*.

Solution. (1) Transform flow. Input domain: read and validate marks; transform centre: compute grades and GPA; output domain: format and print mark sheets. Use transform mapping. (2) Transaction flow. Transaction centre: the kiosk menu/command process; action paths: *Issue*, *Return*, *Renew* and *Search*. Use transaction mapping, with four modules under a dispatcher.

QUICK RECAP

- Architecture defines components and their interactions; its properties are structural, extra-functional and family-based.
- Five architectural models: structural, framework, dynamic, process, functional.

- Control hierarchy measures: depth, width, fan-out, fan-in; superordinate and subordinate modules.
- Horizontal partitioning: branches/layers for major functions. Vertical partitioning: factoring with control on top and workers below.
- Architectural mapping has six steps. Transform mapping: boundaries, transform centre, first- and second-level factoring. Transaction mapping: transaction centre, dispatcher, one module per path.

User Interface Design

The user interface (UI) is the part of the software that people see, hear and touch. However sound the architecture and components, a system whose interface confuses or frustrates its users will be judged a failure. This chapter presents the three golden rules of interface design, the four models that shape an interface, the classification of users, the iterative UI design process, the recurring design issues, the characteristics of a good interface and the sixteen interface design principles listed in the course material.

DEFINITION – User Interface Design

User interface design is the activity of creating an effective communication medium between a human and a computer: identifying interface objects and actions, and producing screen layouts and interaction mechanisms that allow users to perform their tasks efficiently, intuitively and with satisfaction.

5.1 The Golden Rules of Interface Design

Three golden rules form the basis of interface design principles:

1. Place the user in control.
2. Reduce the user's memory load.
3. Make the interface consistent.

5.1.1 Rule 1: Place the User in Control

The interface should respond to the user's needs so that users feel in charge of the system rather than controlled by it. Table 5.1 lists the seven design guidelines that implement this rule.

5.1.2 Rule 2: Reduce the User's Memory Load

The more a user must remember, the more error-prone the interaction. A well-designed interface does not tax the user's short-term memory (Table 5.2).

5.1.3 Rule 3: Make the Interface Consistent

Consistency lets users transfer what they have learnt in one part of the system to every other part (Table 5.3).

EXAM TIP

A 10-mark question on the golden rules expects the three rules as headings with at least three or four guidelines under each, each supported by an example. The examples in Tables 5.1–5.3 are the ones used in the course material.

Table 5.1. Guidelines for placing the user in control.

Guideline	Explanation and example
Allow user control over the system	Design interfaces that react to user needs, so users feel in charge rather than controlled.
Define interaction modes flexibly	Do not force users into unnecessary or undesired actions; allow easy entry to and exit from modes (e.g. turning spell-check mode on and off in a word processor).
Provide flexible interaction options	Support multiple interaction methods—keyboard, mouse, touch, voice—to suit different preferences.
Enable interruptible and undoable actions	Allow users to pause a sequence, switch tasks and undo actions without losing progress.
Streamline interaction for advanced users	Offer customisation such as macros so skilled users can simplify repetitive tasks.
Hide technical details	Shield casual users from system internals (e.g. operating-system commands) and keep interaction task-oriented.
Enable direct manipulation	Let users act directly on on-screen objects (e.g. resizing or moving items), which gives a sense of control.

Table 5.2. Guidelines for reducing the user's memory load.

Guideline	Explanation and example
Minimise reliance on short-term memory	Use visual cues so users recognise past actions instead of recalling them.
Set meaningful defaults	Provide sensible defaults for the average user, allow personalisation, and offer a reset option.
Design intuitive shortcuts	Use mnemonics tied to actions that are easy to remember (e.g. Alt+P for print).
Use real-world metaphors	Base layouts on familiar concepts (e.g. a chequebook metaphor for bill payment).
Progressive disclosure of information	Present a high-level overview first and reveal detail as the user shows interest (e.g. hierarchical menus).

5.2 Interface Design Models

Four different models come into play when a user interface is analysed and designed (Figure 5.1).

The **goal** is to align the implementation model with the user's mental model, so that users can use the system effectively and comfortably. The **key principle** is “know the user, know the tasks”: designers must not substitute their own perspective for that of the user.

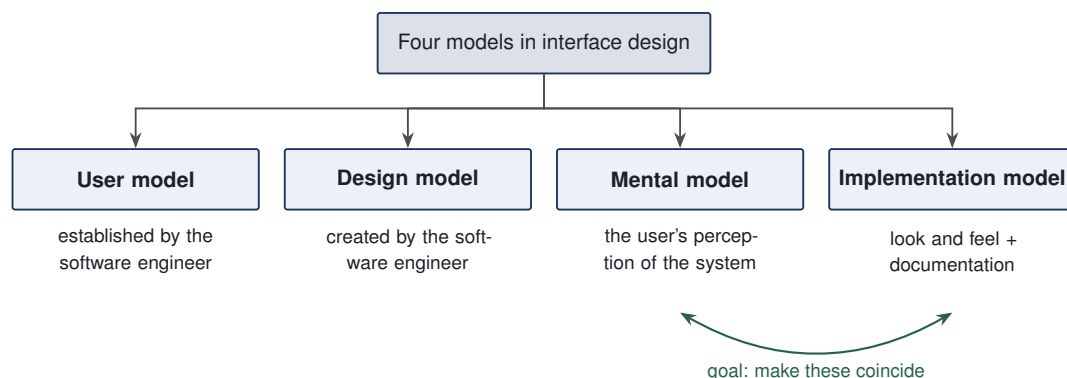
5.2.1 Categories of Users

The user model classifies users as follows:

- **Novices:** no *syntactic* knowledge of the system (how to operate the interface) and little *semantic* knowledge of the application or computer usage in general.

Table 5.3. Guidelines for making the interface consistent.

Guideline	Explanation and example
Organise visual information consistently	Apply uniform design rules across all screens for clarity and coherence.
Limit and standardise input mechanisms	Use a consistent set of input methods throughout the application.
Standardise navigation	Define and implement consistent ways of moving between tasks.
Provide meaningful context	Use indicators—titles, icons, colours—so users understand their current task and location.
Maintain consistency across applications	All products in a family should follow the same design principles.
Respect established user expectations	Do not change widely accepted interaction patterns (e.g. Alt+S for save) unless there is a compelling reason, since change confuses users.

**Figure 5.1.** The four models of interface design.**Table 5.4.** The four interface design models.

Model	Description
User model	Profile of the end users: demographics, abilities, goals. Established by the software engineer.
Design model	Created by software engineers to align with the user model and guide the design process.
Mental model (system perception)	The user's internal understanding of how the system works, influenced by experience and familiarity.
Implementation model	The system's outward appearance (look and feel) combined with supporting documentation (manuals, help files).

- **Knowledgeable, intermittent users:** reasonable semantic knowledge of the application but relatively low recall of the syntactic details needed to use the interface.
- **Knowledgeable, frequent users:** good semantic and syntactic knowledge; “power users” who look for shortcuts and abbreviated modes of interaction.

Worked Example 5.1 – Designing for all three user categories

A hospital billing application is used by trainee clerks, by doctors who log in once a week, and by senior billing staff all day. How should the interface serve each group?

Solution. Trainee clerks (novices) need guided workflows, meaningful defaults, visible menus and context-sensitive help. Doctors (knowledgeable, intermittent) understand billing but forget interface details, so they benefit from consistent layout, recognisable icons and labelled menus that do not rely on recall. Senior staff (knowledgeable, frequent) need keyboard shortcuts, macros for repetitive entries and the ability to skip confirmation screens. Providing all three through progressive disclosure satisfies the golden rules for every category.

5.3 The User Interface Design Process

The UI design process is iterative and is represented by a **spiral model** with four framework activities (Figure 5.2).

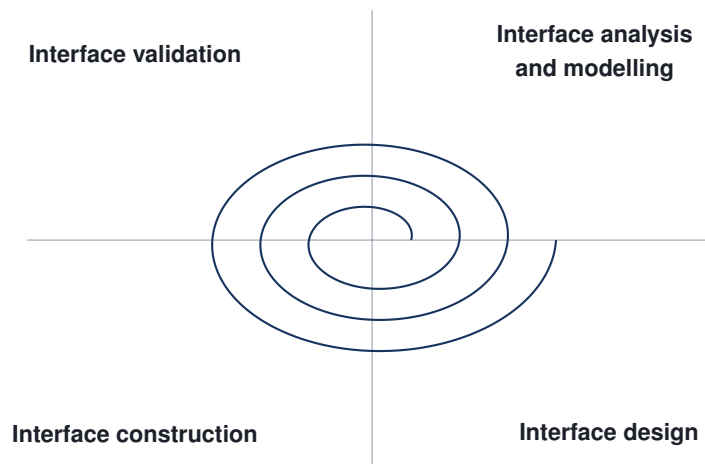


Figure 5.2. The UI design process as a spiral of four activities; each pass elaborates the interface further.

1. **Interface analysis and modelling.** Understand user profiles, skill levels and environmental factors; define user categories and elicit requirements for each; perform *task analysis* to identify and describe user tasks; analyse the physical work environment (location, lighting, noise, human factors).
2. **Interface design.** Define interface objects, actions and screen representations that meet usability goals, so that tasks can be performed efficiently and intuitively.
3. **Interface construction.** Begin with a prototype to evaluate usage scenarios; use UI toolkits to refine and complete the interface.
4. **Interface validation.** Confirm that the interface supports all user tasks and their variations; assess ease of use, ease of learning and user acceptance.

Two properties are emphasised. **Prototyping** is essential for evaluating and refining the interface during construction. The process is **iterative**: several passes through the spiral elaborate task details, design information and operational features incrementally, so every detail need not be specified in the first iteration.

5.4 Interface Design Issues

Four common design issues almost always surface—system response time, help facilities, error handling and command labelling—together with application accessibility and internationalisation.

5.4.1 System Response Time

Response time is measured from the point at which the user performs an action (pressing Enter, clicking a button) until the system responds with the desired output or action. It has two important characteristics:

- **Length**: long delays cause frustration.
- **Variability**: the deviation from the average response time. Consistent response times are preferable to highly variable ones, even if slightly longer.

The goal is to minimise variability and so establish a predictable rhythm of interaction.

COMMON MISTAKE

Many students write only about the *length* of response time. Variability is equally important: a response that takes 1 s on some occasions and 6 s on others is more disturbing than one that consistently takes 2 s.

5.4.2 Help Facilities

Help guides users when they encounter difficulties. The design considerations are:

- **Availability**: is help provided for all functions or only a subset?
- **Access**: through a help menu, a function key or a help command.
- **Representation**: a separate window, a one-line suggestion or hypertext.
- **Structure**: flat, layered or hypertext-based organisation.
- **Return**: an easy return to normal interaction.

5.4.3 Error Handling

Error messages should be clear, constructive and non-judgemental. An effective error message:

1. uses language the user can understand;
2. gives constructive, actionable advice for recovering from the error;
3. indicates any negative consequences (e.g. possible data corruption);
4. is accompanied by a visual or auditory cue (colour, beep); and
5. never blames the user.

Poorly designed messages increase frustration and confusion. Compare “Error 0x80070057” with “The date must be in DD/MM/YYYY format, for example 23/09/2026. Your other entries have been kept.”

5.4.4 Menu and Command Labelling

Commands and menu labels should be consistent and clear across applications. Key considerations:

1. **Correspondence:** menu options match the underlying commands.
2. **Form:** control sequences, function keys or typed words.
3. **Learnability:** commands are easy to learn and remember; reminders are available for forgotten commands.
4. **Customisation:** users can customise or abbreviate commands.
5. **Clarity:** labels and submenus are self-explanatory and consistent.

5.4.5 Application Accessibility

Interfaces must accommodate users with physical challenges—visual, hearing or mobility impairments. Designers follow accessibility guidelines (e.g. W3C guidelines and assistive-technology standards). Ethical, legal and business imperatives all drive accessibility requirements.

5.4.6 Internationalisation

Software is created as “globalised” software with a generic core of functionality that can be adapted to different locales and languages. *Localisation* features customise it for a specific market. Challenges include screen layouts that vary between regions and different alphabets and symbols that need specialised labelling and spacing. Standards such as **Unicode** address the multilingual challenge by providing a single character encoding for virtually all written languages.

DEFINITION – Internationalisation and Localisation

Internationalisation is designing software with a locale-independent core so that it can be adapted to many languages and regions without redesign. *Localisation* is the adaptation of internationalised software to a particular locale: translated text, date, number and currency formats, and culturally appropriate layout.

5.5 Characteristics of a Good User Interface

The course material lists three characteristics:

- **Visually apparent and forgiving:** users feel in control, with clear options and easy recovery from mistakes.
- **Hides system internals:** work is saved automatically and actions can be undone without users worrying about system mechanics.
- **Minimises user input:** functionality is maximised while the information users must supply is minimised.

5.6 User Interface Design Principles

The course material lists sixteen principles, drawn from WebApp interface design practice (Table 5.5).

Table 5.5. The sixteen user interface design principles.

No.	Principle	Meaning and example
1	Anticipation	Predict the user's next need and provide it; e.g. after a user views printer information, offer the driver download pre-emptively.
2	Communication	Communicate the status of any activity (text message, animation) and the user's location within the WebApp.
3	Consistency	Keep navigation controls, menus, icons and aesthetics uniform; never reuse a symbol for a conflicting purpose.
4	Controlled autonomy	Let users move freely, but enforce navigation conventions and security (e.g. password-protected areas).
5	Efficiency	Optimise the user's work efficiency, not the developer's convenience or the machine's efficiency.
6	Flexibility	Support both task-oriented and exploratory users; provide undo and clear paths back.
7	Focus	Keep the interface focused on the user's primary task; avoid distracting, loosely related content.
8	Fitts's law	Minimise the time to make a selection by placing related options close together and making frequent targets large.
9	Human interface objects	Reuse established interface objects (buttons, menus) from libraries to enhance usability.
10	Latency reduction	Use multitasking to hide delays, acknowledge delays with feedback (progress bars) and occupy the user during long operations.
11	Learnability	Design for quick learning and minimal relearning through simple, intuitive organisation.
12	Metaphors	Use appropriate metaphors (e.g. a chequebook for bill payment) to make the interface easy to learn.
13	Maintain work-product integrity	Autosave user data and provide recovery for interrupted sessions.
14	Readability	Make information readable for all age groups: clear fonts, adequate sizes, high-contrast colours.
15	Track state	Store interaction state (e.g. through cookies) so users can resume later.
16	Visible navigation	Create the illusion that users stay in one place, bringing content and functions to them rather than making them navigate.

SUPPLEMENTARY EXPLANATION – Fitts's law

Fitts's law models the time T to move a pointer to a target of width W at distance D :

$$T = a + b \log_2 \left(\frac{D}{W} + 1 \right),$$

where a and b are empirically determined constants for a device and user. The logarithmic term, the *index of difficulty*, grows as the target becomes farther or smaller. Hence large, nearby targets are selected faster, which is why the course principle advises placing related options close together. The course material states the principle qualitatively; the formula is given here for understanding.

Worked Example 5.2 – Applying Fitts's law qualitatively

On a checkout page the *Pay now* button is a small link at the top of the page, while the cursor usually rests near the order total at the bottom. Which principles are violated and what should change?

Solution. Fitts's law is violated: the target is small (W small) and far (D large), so selection time is high. Focus and efficiency are also affected. Place a large *Pay now* button directly beneath the order total, grouped with *Apply coupon* and *Change address*. As a numerical illustration, with $D = 800$ px and $W = 40$ px the index of difficulty is $\log_2(21) \approx 4.39$ bits; with $D = 100$ px and $W = 200$ px it is $\log_2(1.5) \approx 0.58$ bits, a large reduction.

EXAM TIP – Mnemonic for the sixteen principles

Group them: *communicate* (anticipation, communication, latency reduction, track state); *constrain* (consistency, controlled autonomy, focus, visible navigation); *ease* (efficiency, flexibility, Fitts's law, learnability, metaphors, readability); *protect* (human interface objects, work-product integrity). Four groups of four are easier to recall than a list of sixteen.

COMMON MISTAKE

Do not confuse *accessibility* (use by people with disabilities) with *internationalisation* (use across languages and regions), or *consistency* (a UI principle) with *uniformity and integration* (a general design principle, Section 1.5).

QUICK RECAP

- Golden rules: place the user in control; reduce memory load; make the interface consistent.
- Four models: user, design, mental, implementation; align the implementation model with the mental model.
- User categories: novices; knowledgeable intermittent; knowledgeable frequent.
- Process (spiral): analysis and modelling, design, construction, validation; iterative, prototype-driven.
- Issues: response time (length and variability), help, error handling, menu/command labelling, accessibility, internationalisation (Unicode).
- Sixteen principles from anticipation to visible navigation.

Software Testing Fundamentals

Design determines how software *should* behave; testing establishes whether it *does*. This chapter opens Module 5 by defining software testing, stating its objectives, placing it within the testing strategy, describing the testing strategies listed in the course material and classifying the types of testing.

6.1 What Is Software Testing?

DEFINITION – Software Testing

Software testing is the process of evaluating a software application to ensure that it meets the specified requirements and is free of defects. It involves executing a program or system with the intent of finding errors, verifying its functionality and ensuring that it performs as expected.

The phrase *with the intent of finding errors* is essential. A test that finds no error is not thereby a success; a good test case is one with a high probability of finding an as-yet-undiscovered error. Testing can show the presence of defects but cannot prove their absence.

SUPPLEMENTARY EXPLANATION – Error, fault and failure

An *error* (mistake) is a human action that produces an incorrect result; it leads to a *fault* (defect, bug) in the code; executing the fault may cause a *failure*, an observable deviation from expected behaviour. Testing observes failures in order to locate faults; debugging removes them.

6.2 Objectives of Testing

The course material lists five objectives:

1. **Detect bugs:** find defects before users do.
2. **Validate requirements:** confirm that the software does what the requirements specify.
3. **Improve quality:** raise reliability, performance and usability.
4. **Ensure user satisfaction:** confirm that the product meets user expectations.
5. **Comply with industry standards and regulations:** e.g. security, safety or accessibility regulations applicable to the domain.

6.3 The Testing Strategy

Testing proceeds from the small to the large. The course material depicts the strategy as concentric levels that mirror the development activities (Figure 6.1).

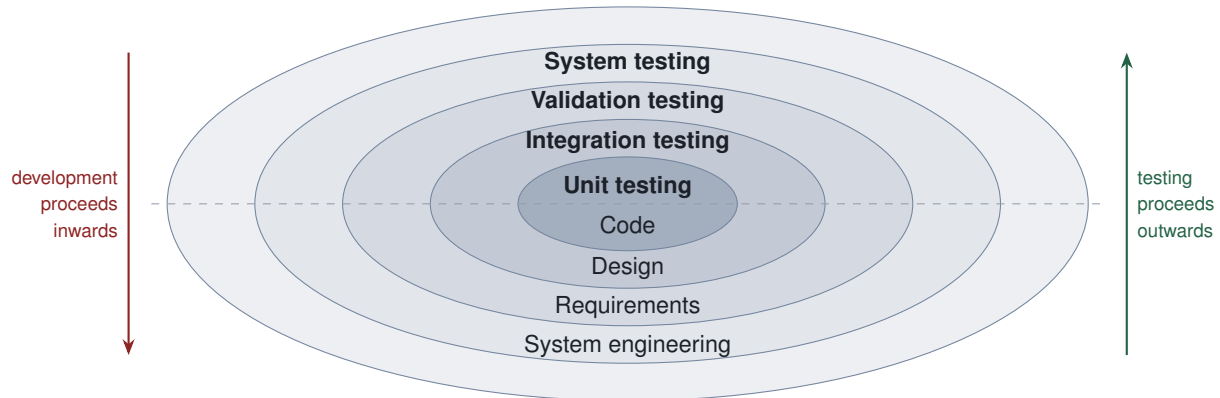


Figure 6.1. Testing strategy: each testing level (upper half) verifies the corresponding development level (lower half).

Development spirals inward from system engineering through requirements and design to code. Testing spirals outward: **unit testing** verifies individual components as coded; **integration testing** verifies the assembled components against the design; **validation testing** confirms the software against the requirements; and **system testing** verifies the software together with other system elements (hardware, people, databases).

6.4 Testing Strategies Listed in the Course Material

Table 6.1. Testing strategies and their purpose.

Strategy	Description
Requirement-based testing	Verifies that the software meets the specified requirements. Test cases are derived from the documented requirements and the software is evaluated against them.
Functional testing	Validates the functional aspects of the software by testing individual functions or features and verifying that each performs as intended.
Integration testing	Tests the interaction and integration between components or modules, ensuring they work together and exchange data correctly.
System testing	Validates the entire integrated system, including all components and subsystems, against the overall system requirements.
Performance testing	Evaluates performance under specific conditions such as load, stress or scalability; identifies bottlenecks, measures response times and confirms that expected workloads can be handled.
Regression testing	Retests previously tested functionality to ensure that recent changes or fixes have not introduced new defects or unintended side effects.
User acceptance testing (UAT)	Performed by end users or their representatives to verify that the software meets their expectations and is ready for deployment in the real environment.
Security testing	Identifies vulnerabilities and verifies resistance to threats: authentication, authorisation, data encryption and protection against common attacks.

Strategy	Description
Usability testing	Assesses user-friendliness, ease of use and overall user experience, including interface design, navigation and interaction.
Maintenance testing	Performed after updates or modifications to ensure that the changes have not introduced new defects or adversely affected existing functionality.

IMPORTANT NOTE

Regression testing and maintenance testing are closely related. Regression testing is the *technique* of re-running earlier tests after any change; maintenance testing is testing in the *context* of changes to operational software, and it normally includes regression testing together with tests of the modification itself.

6.5 Types of Testing

The course material classifies testing along three independent dimensions (Figure 6.2).

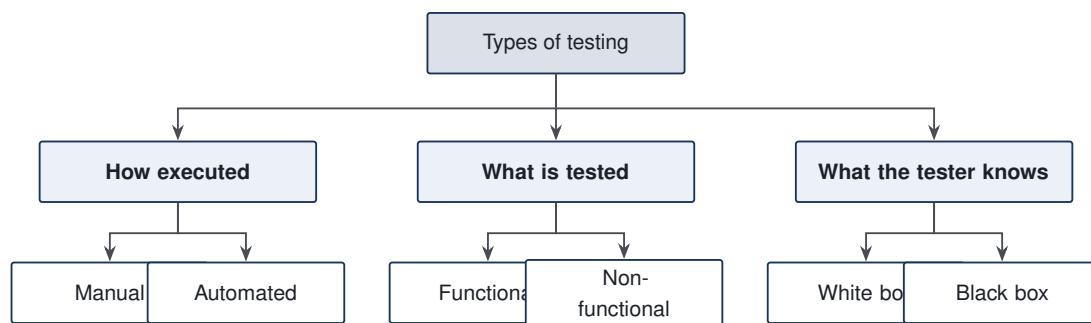


Figure 6.2. Three dimensions for classifying testing.

- **Manual testing:** test cases are executed by a person without automation.
- **Automated testing:** scripts and tools (e.g. Selenium, JUnit) execute the tests.
- **Functional testing:** verifies specific functions of the application (e.g. unit testing, integration testing).
- **Non-functional testing:** tests quality attributes such as performance, security and usability.
- **White-box testing:** the internal code structure is tested (Chapter 7).
- **Black-box testing:** the tester evaluates functionality without looking at the internal code (Chapter 10).

Because the dimensions are independent, a single test activity has a position on each: an automated, functional, black-box regression suite is common in practice.

6.5.1 Static and Dynamic Techniques

The course material also presents a taxonomy that separates *static* techniques, which examine work products without executing them, from *dynamic* techniques, which execute the software (Figure 6.3).

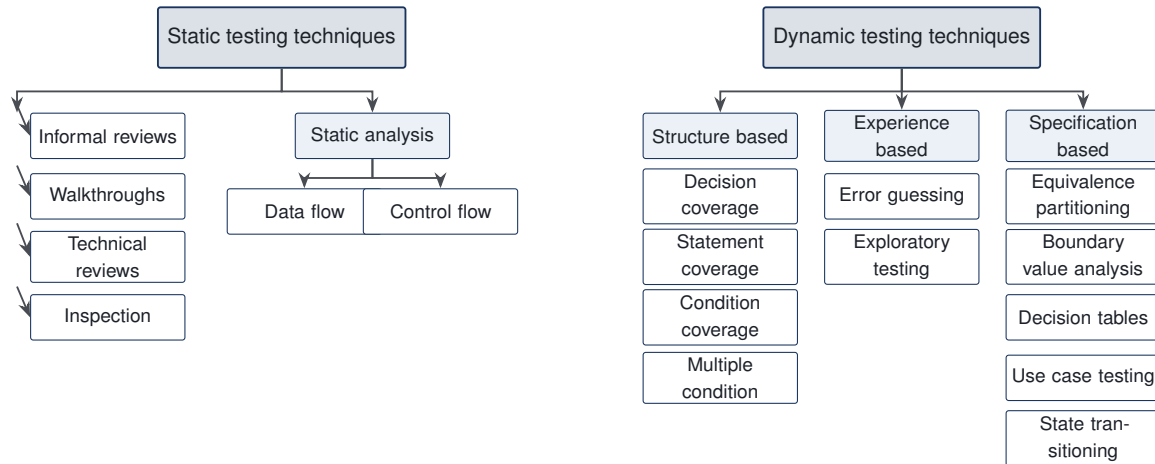


Figure 6.3. Static and dynamic testing techniques (course taxonomy redrawn).

Structure-based dynamic techniques are white-box techniques; specification-based techniques are black-box techniques; experience-based techniques rely on the tester's knowledge and intuition.

6.5.2 White-, Black- and Grey-Box Technique Families

A further course figure groups well-known techniques under three families. *White-box*: control flow testing, branch testing, basis path testing, data flow testing and loop testing. *Black-box*: equivalence partitioning, boundary value analysis, fuzzing, cause-effect graphing, orthogonal array testing, all-pairs testing and state transition testing. *Grey-box* (partial knowledge of internals): orthogonal array testing, matrix testing, regression testing and pattern testing.

Table 6.2. Manual versus automated testing.

Aspect	Manual testing	Automated testing
Execution	By a human tester	By scripts and tools (Selenium, JUnit)
Best suited to	Exploratory, usability and ad-hoc tests; tests run rarely	Regression, load and repetitive tests; tests run often
Initial cost	Low	High (script development)
Cost per run	High (human time)	Low
Reliability	Prone to human error and fatigue	Consistent and repeatable
Maintenance	Test documents only	Scripts must be updated as the software changes

QUICK RECAP

- Testing executes software with the intent of finding errors and verifying requirements.

Table 6.3. Functional versus non-functional testing.

Aspect	Functional testing	Non-functional testing
Question answered	<i>What</i> does the system do?	<i>How well</i> does it do it?
Basis	Functional requirements, use cases	Quality attributes (performance, security, usability)
Examples	Unit, integration, system, UAT	Performance, load, stress, security, usability
Typical result	Pass/fail for a feature	Measured values against targets (e.g. 2 s response)

- Objectives: detect bugs, validate requirements, improve quality, ensure satisfaction, comply with standards.
- Strategy: unit → integration → validation → system, mirroring code, design, requirements and system engineering.
- Ten strategies from requirement-based to maintenance testing.
- Types: manual/automated, functional/non-functional, white-box/black-box; static versus dynamic techniques.

White-Box Testing

White-box testing designs tests from the program's internal structure. This chapter defines the approach, its objectives and the defects it reveals, develops the three code-coverage measures used in the course material—statement, branch and path coverage—with their formulas, and explains control-structure (condition) testing. Basis path testing and cyclomatic complexity, the most important white-box technique for examinations, follow in Chapter 8.

7.1 Concept

7.1.1 Intuition

An electrician who inspects a building's wiring behind the walls can check every circuit, junction and switch. A visitor who only flips switches can confirm that lights come on but cannot see a loose connection that has not yet failed. White-box testing is the electrician's inspection; black-box testing is the visitor's.

DEFINITION – White-Box Testing

White-box testing is a software testing technique that examines the internal structure, logic and code of an application. Tests are designed from knowledge of the internal implementation, and testers have access to the source code, algorithms and architecture. It is also known as *clear-box*, *open-box* or *glass-box* testing.

7.1.2 Objectives

- To ensure that all internal operations function as expected according to the design and specifications.
- To identify hidden errors, security vulnerabilities and inefficiencies in the code.

7.1.3 Types of Defects Identified

- Logical errors in the code.
- Broken or poorly structured paths.
- Syntax errors or typographical errors in the code.
- Unreachable or redundant code.

SUPPLEMENTARY EXPLANATION – Why white-box testing is needed in addition to black-box testing

Logic errors and incorrect assumptions are inversely proportional to the probability that a path is executed: rarely executed paths are where errors hide. Typographical errors are random, so one may lie on any path. Black-box tests derived from the specification may never reach such paths; white-box tests are designed to reach them deliberately.

7.2 Code Coverage

DEFINITION – Code Coverage

Code coverage is a measure of how much of an application's code is executed and validated by its test suite. Statement, branch and path coverage indicate how much of the application's logic is actually exercised.

7.2.1 Statement Coverage

Statement coverage aims to execute as much of the source code as possible: every executable statement at least once.

FORMULA 7.1 – Statement coverage

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} \times 100\%$$

7.2.2 Branch Coverage

In programming, a *branch* is an outcome of a decision. An `if` statement has two branches, *true* and *false*. Branch coverage checks that each branch is executed at least once, so an `if` statement needs two test conditions: one that validates the true branch and one that validates the false branch.

FORMULA 7.2 – Branch coverage

$$\text{Branch coverage} = \frac{\text{Number of executed branches}}{\text{Total number of branches}} \times 100\%$$

7.2.3 Path Coverage

Path coverage examines all the paths in the program, ensuring that every path is explored at least once. It is a thorough strategy, more effective than branch coverage, and useful for complicated applications. Because loops can produce an unbounded number of paths, practical path testing uses the *basis set* of independent paths (Chapter 8).

FORMULA 7.3 – Path coverage

$$\text{Path coverage} = \frac{\text{Number of paths executed}}{\text{Total number of (feasible) paths}} \times 100\%$$

IMPORTANT NOTE – Subsumption

100% path coverage implies 100% branch coverage, which implies 100% statement coverage. The converse implications do not hold. This ordering is why the course material describes path coverage as the strongest of the three.

Worked Example 7.1 – Computing statement, branch and path coverage

```
def classify(m):
    result = "Fail"           # S1
    if m >= 40:                # S2 (decision D1)
        result = "Pass"       # S3
    if m >= 75:                # S4 (decision D2)
        result = "Distinction" # S5
    return result             # S6
```

There are 6 executable statements (S1–S6) and 2 decisions with 2 outcomes each, i.e. 4 branches: D1-T, D1-F, D2-T, D2-F.

(a) **Test set {m = 80}**. Executes S1, S2, S3, S4, S5, S6.

$$\begin{aligned}\text{Statement coverage} &= \frac{6}{6} \times 100 = 100\%, \\ \text{Branch coverage} &= \frac{2}{4} \times 100 = 50\% \quad (\text{D1-T, D2-T}).\end{aligned}$$

(b) **Test set {m = 30}**. Executes S1, S2, S4, S6.

$$\begin{aligned}\text{Statement coverage} &= \frac{4}{6} \times 100 = 66.67\%, \\ \text{Branch coverage} &= \frac{2}{4} \times 100 = 50\% \quad (\text{D1-F, D2-F}).\end{aligned}$$

(c) **Test set {m = 80, m = 30}**. All 6 statements and all 4 branches: 100% statement and 100% branch coverage.

(d) **Paths**. Combinations of the two decisions give four paths: TT, TF, FT, FF. The path FT ($m < 40$ and $m \geq 75$) is *infeasible*. The three feasible paths need $m = 80$ (TT), $m = 50$ (TF) and $m = 30$ (FF). Test set (c) covers TT and FF, so its path coverage is $\frac{2}{3} \times 100 = 66.67\%$; adding $m = 50$ gives 100%.

Conclusion. A single test achieved full statement coverage but only half the branches, and two tests achieved full branch coverage but not full path coverage—a direct illustration of the subsumption order.

COMMON MISTAKE

- Counting non-executable lines (function headers, `else`, closing braces) as statements. State clearly which lines are counted.
- Counting branches as the number of decisions. Each two-way decision contributes *two* branches.
- Assuming that 100% statement coverage means all branches are tested. An `if` without an `else` can have every statement executed while its false branch never runs.

7.3 Control-Structure Testing: Errors in Conditions

Basis path testing is simple and effective but not sufficient by itself. Control-structure testing broadens coverage by concentrating on the *conditions* inside decisions. The course material lists five categories of common errors in conditions (Table 7.1).

Table 7.1. Common errors in conditions.

Error type	Description	Example (intended → written)
Boolean operator error	Incorrect, missing or extra Boolean operator	<code>a > 0 OR b > 0</code> → <code>a > 0 AND b > 0</code>
Boolean variable error	Wrong variable used in a condition	<code>if (isAdmin)</code> → <code>if (isUser)</code>
Parenthesis error	Misplaced or missing parentheses changing evaluation order	<code>(a OR b) AND c</code> → <code>a OR b AND c</code>
Relational operator error	Wrong relational operator	<code>age > 18</code> → <code>age >= 18</code>
Arithmetic expression error	Error in an expression used within a condition	<code>total/count > 50</code> with <code>count = 0</code> (division by zero)

SUPPLEMENTARY EXPLANATION – Designing tests for relational expressions

For a relational expression $E_1 \text{ op } E_2$, three tests—making E_1 greater than, equal to and slightly less than E_2 —detect most relational-operator errors. For `age > 18`, the tests `age = 19`, `age = 18` and `age = 17` distinguish `>` from `>=`, `<`, `<=`, `==` and `!=`: only the correct operator yields T, F, F. This is the white-box counterpart of boundary value analysis (Chapter 10).

Worked Example 7.2 – Detecting a relational-operator error

Specification: a customer is eligible for a senior discount if *age is at least 60*. Code: `if (age > 60) discount = 0.1;`

age	Expected	Actual (>)	Verdict
61	discount	discount	pass
60	discount	no discount	fail
59	no discount	no discount	pass

Only the test on the boundary (`age = 60`) reveals the relational-operator error; a test with a typical value such as 70 would never expose it.

QUICK RECAP

- White-box (clear, open, glass box) testing uses knowledge of code, algorithms and architecture.
- It finds logical errors, broken paths, typographical errors and unreachable code.

- Statement, branch and path coverage, each = $(\text{executed}/\text{total}) \times 100\%$; path $\Rightarrow$ branch $\Rightarrow$ statement.
- Condition errors: Boolean operator, Boolean variable, parenthesis, relational operator, arithmetic expression.

Basis Path Testing and Cyclomatic Complexity

Basis path testing is the white-box technique most frequently examined numerically. Given a program, the student is expected to draw its control flow graph, compute its cyclomatic complexity by more than one method, list a basis set of independent paths and design one test case per path. This chapter develops every step from first principles and then works through eight problems of increasing difficulty; four are taken from the course material and four are additional practice problems.

8.1 Basis Path Testing

DEFINITION – Basis Path Testing

Basis path testing is a structured white-box testing method that identifies a set of basis paths (independent paths) in a program's control flow graph and designs test cases so that every independent path is executed at least once. Each path corresponds to a unique sequence of statements and decision outcomes.

The objectives stated in the course material are:

- to ensure that every independent path in the program is executed at least once during testing; and
- to achieve thorough path coverage, so that all logical branches and conditions are tested.

IMPORTANT NOTE

Executing a basis set guarantees that every statement is executed at least once and every branch is taken in each direction at least once. The course material describes this as “100% path coverage” in the sense of covering the basis paths; strictly, a program with loops has more paths than the basis set, but the basis set is the practical, finite target.

8.2 Control Flow Graphs

DEFINITION – Control Flow Graph

A control flow graph (CFG), or flow graph, is a graphical representation of a program's logic in which *nodes* represent statements or blocks of code, *edges* represent the flow of control between them, and *decision points* (if, else, switch, loops) create branches in the graph.

8.2.1 Terminology

- **Node:** a circle representing one or more procedural statements. A sequence of statements with no branching may be collapsed into one node.
- **Edge (link):** an arrow representing transfer of control. Every edge must terminate at a node.
- **Predicate (decision) node:** a node containing a condition; it has two or more outgoing edges.
- **Region:** an area bounded by edges and nodes. The area outside the graph counts as one region.
- **Independent path:** a path through the program that introduces at least one new set of processing statements or a new condition, i.e. it traverses at least one edge not traversed by any previously listed path.

8.2.2 Flow-Graph Notation for Structured Constructs

Figure 8.1 gives the standard subgraphs. Any structured program's CFG is built by composing them.

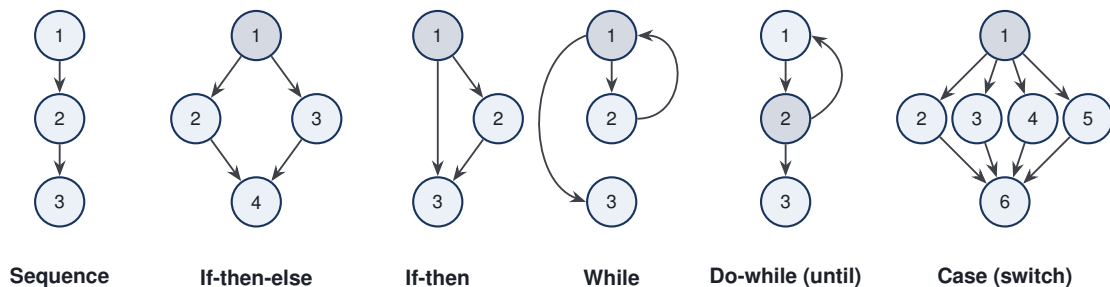


Figure 8.1. Flow-graph notation for structured constructs; shaded nodes are predicate nodes.

8.2.3 Rules for Drawing a CFG

1. Number the executable statements of the program.
2. Give each decision (`if`, loop test, `switch`) its own node.
3. Merge purely sequential statements into one node if desired; this does not change the cyclomatic complexity, because it removes one node and one edge together.
4. Draw a join node where branches reconverge (an `ENDIF` or the statement after the construct).
5. For a loop, draw the back edge from the end of the loop body to the loop test, and an exit edge from the loop test to the statement after the loop.
6. A compound condition such as `a AND b` should be split into one predicate node per simple condition (Worked Example 8.6).

8.3 Cyclomatic Complexity

DEFINITION – Cyclomatic Complexity

Cyclomatic complexity, denoted M or $V(G)$, is a software metric that gives the number of linearly independent paths in a program's control flow graph. It is therefore the number of test cases required to execute every independent (basis) path at least once, and an upper bound on the number of tests needed to cover every edge.

8.3.1 Three Methods of Computation

FORMULA 8.1 – Cyclomatic complexity from edges and nodes

$$M = V(G) = E - N + 2P$$

E = number of edges; N = number of nodes; P = number of connected components (for a single program or function, $P = 1$, giving $M = E - N + 2$).

FORMULA 8.2 – Cyclomatic complexity from decision nodes

$$M = V(G) = D + 1$$

D = number of decision (predicate) nodes, each a two-way decision.
A multi-way decision with k outgoing edges contributes $k - 1$ to D .

FORMULA 8.3 – Cyclomatic complexity from regions

$$M = V(G) = R$$

R = number of regions of the planar flow graph,
including the one outer (unbounded) region.

COMMON MISTAKE – Two different uses of the letter P

In $M = E - N + 2P$, P is the number of *connected components*. One course slide writes “Method 2: $P = 2$, $V(G) = P + 1 = 3$ ”, where P means the number of *predicate* nodes. To avoid confusion, this book writes D for predicate (decision) nodes. In an answer, always define the symbol you use.

8.3.2 Meaning of M

- M is the size of a *basis set*: the number of independent paths such that every path through the program can be formed from them.
- M is the **minimum number of basis-path test cases** needed so that every independent path, and therefore every statement and every branch, is exercised.
- M also measures logical complexity. As a supplementary guideline, modules with M above about 10 are commonly considered hard to test and candidates for refactoring.

8.3.3 The Basis Path Testing Procedure

The course material lists the following steps.

1. **Draw the control flow graph**: represent the logic as nodes and edges and identify decision points and their branches.
2. **Calculate the cyclomatic complexity**: $M = E - N + 2$, or count decision points D and use $M = D + 1$.
3. **Identify independent paths**: determine the minimum set of paths that covers all edges and nodes; each path must include at least one edge not covered by the previous paths.
4. **Design test cases**: create inputs that execute each independent path, so that all conditions and branches are tested.
5. **Execute the test cases**: run them and compare the program's behaviour with the expected outcomes.

EXAM TIP – Answer layout for numerical questions

Write the solution in this order: (1) numbered code; (2) CFG with numbered nodes and predicate nodes marked; (3) list of edges with the count E ; (4) N and P ; (5) $M = E - N + 2P$ with substitution; (6) verification by $D + 1$ and, if asked, by regions; (7) table of independent paths; (8) table of test cases with inputs and expected outputs. Every step earns marks.

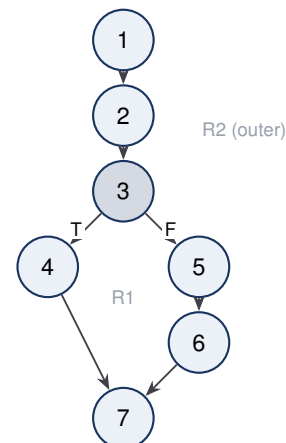
8.4 Worked Examples

Worked Example 8.1 – Larger of two numbers (course example)

```

1 Read A
2 Read B
3 IF (A > B) THEN
4     Print "A is greater"
5 ELSE
6     Print "B is greater"
7 ENDIF

```



Step 1 — CFG. Each numbered line is a node; node 3 is the only predicate node (shaded).

Step 2 — Edges. $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $3 \rightarrow 5$, $5 \rightarrow 6$, $4 \rightarrow 7$, $6 \rightarrow 7$; hence $E = 7$. Nodes: $N = 7$. Components: $P = 1$.

Step 3 — Complexity.

$$M = E - N + 2P = 7 - 7 + 2(1) = 2.$$

Verification. $D = 1$ (node 3), so $M = D + 1 = 1 + 1 = 2$. Regions: R1 (bounded by 3–4–7–6–5–3) and the outer region R2, so $R = 2$. All three methods agree.

Step 4 — Independent paths.

Path	Sequence	New edges
P1	1–2–3–4–7	all edges of the path
P2	1–2–3–5–6–7	3 → 5, 5 → 6, 6 → 7

Step 5 — Test cases.

TC	Input	Path	Expected output
TC1	A = 10, B = 5	P1	A is greater
TC2	A = 3, B = 8	P2	B is greater

Coverage. Two tests execute all 7 nodes and all 7 edges: 100% statement and branch coverage.

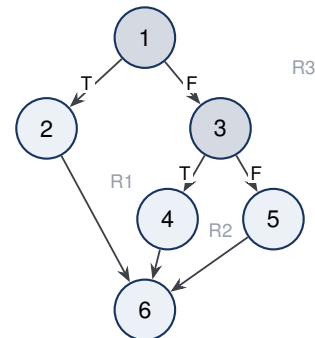
Observation. For $A = B$ (e.g. 4 and 4) the program takes P2 and prints “B is greater”, which is incorrect for equal values. Basis path testing covers the paths, but finding this defect needs a boundary test (Chapter 10); the techniques are complementary.

Worked Example 8.2 – Nested if-else (course example)

```

1  if (a < b)
2      F1();
3  else if (a < c)
4      F2();
5  else
6      F3();
7  /* end */

```



Node numbering: 1 = $a < b$; 2 = $F1()$; 3 = $a < c$; 4 = $F2()$; 5 = $F3()$; 6 = end (join).

Edges. $1 \rightarrow 2$, $1 \rightarrow 3$, $3 \rightarrow 4$, $3 \rightarrow 5$, $2 \rightarrow 6$, $4 \rightarrow 6$, $5 \rightarrow 6$; $E = 7$, $N = 6$, $P = 1$.

$$M = E - N + 2P = 7 - 6 + 2 = 3.$$

Verification. $D = 2$ (nodes 1 and 3): $M = 2 + 1 = 3$. Regions: $R1$ (1–2–6–4–3–1), $R2$ (3–4–6–5–3) and the outer region $R3$: $R = 3$.

Path	Sequence	a	b	c	Expected
P1	1–2–6	1	5	any (e.g. 0)	$F1()$ executed
P2	1–3–4–6	5	2	9	$F2()$ executed
P3	1–3–5–6	5	2	3	$F3()$ executed

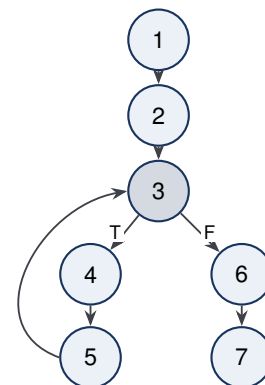
Three test cases, equal to M , cover every node and edge.

Worked Example 8.3 – Summation loop (course example)

```

1  int sum = 0; int i;
2  i = 1;
3  while (i <= 5) {           /* for-loop
4      test */
5      sum = sum + i;
6      i++;
7  }
8  printf("The sum is: %d\n", sum);
9  return 0;

```



The course code is a for loop; it is shown here in its equivalent while form, which makes the initialisation, test and increment explicit. Nodes: 1 = declarations and $sum=0$; 2 = $i=1$; 3 = $i \leq 5$; 4 = $sum=sum+i$; 5 = $i++$; 6 = $printf$; 7 = return.

Edges. $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $4 \rightarrow 5$, $5 \rightarrow 3$, $3 \rightarrow 6$, $6 \rightarrow 7$; $E = 7$, $N = 7$.

$$M = 7 - 7 + 2 = 2, \quad D + 1 = 1 + 1 = 2.$$

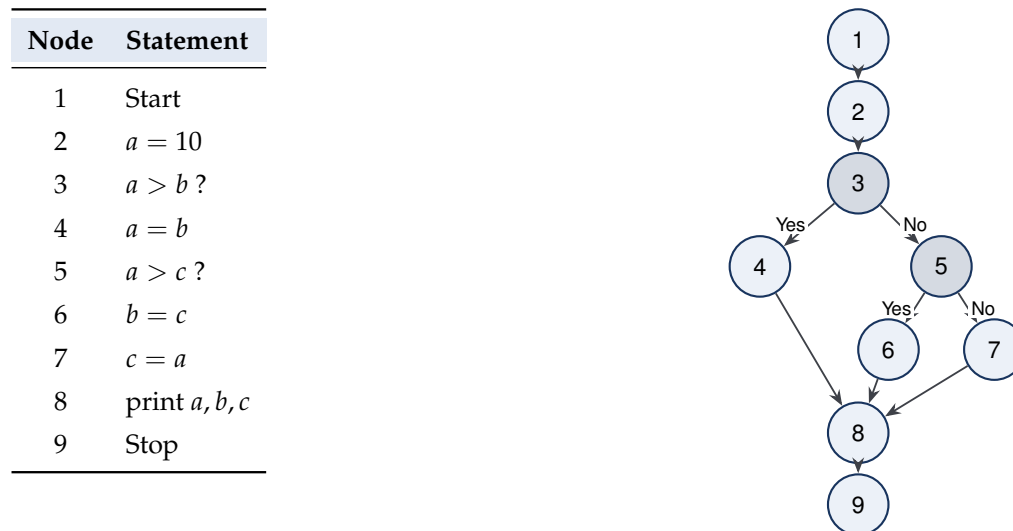
Independent paths. P1: 1–2–3–6–7 (loop not entered). P2: 1–2–3–4–5–3–6–7 (loop entered).

Test cases and feasibility. Because the loop bounds are constants, the condition $i \leq 5$ is true on first evaluation, so P1 is *infeasible* in this program: no input can make it happen. P2

is executed by the single run of the program, which iterates five times ($1 + 2 + 3 + 4 + 5$) and prints The sum is: 15. That one execution also traverses the exit edge $3 \rightarrow 6$, so all 7 edges are covered. When a basis path is infeasible, it is recorded as infeasible and the remaining paths are tested; if the bound were an input n , P1 would be tested with $n = 0$ (expected output The sum is: 0).

Worked Example 8.4 – Flowchart with two decisions (course example)

The course flowchart: Start; $a = 10$; if $a > b$ then $a = b$; otherwise, if $a > c$ then $b = c$, else $c = a$; then print a, b, c ; Stop. Values of b and c are inputs.



Method 1. Edges: $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $3 \rightarrow 5$, $5 \rightarrow 6$, $5 \rightarrow 7$, $4 \rightarrow 8$, $6 \rightarrow 8$, $7 \rightarrow 8$, $8 \rightarrow 9$, so $E = 10$; $N = 9$; $P = 1$.

$$V(G) = E - N + 2P = 10 - 9 + 2(1) = 3.$$

Method 2. Predicate nodes 3 and 5: $D = 2$, so $V(G) = D + 1 = 3$. (The slide writes this as $P + 1$ with P denoting predicates.)

Method 3. Regions: $3-4-8-6-5-3$, $5-6-8-7-5$ and the outer region: $R = 3$.

Path	Sequence	b	c	Expected print (a, b, c)
P1	1-2-3-4-8-9	5	3	5, 5, 3
P2	1-2-3-5-6-8-9	20	4	10, 4, 4
P3	1-2-3-5-7-8-9	20	30	10, 20, 10

Check of P1: $a = 10 > 5$ is true, so $a \leftarrow b = 5$; printed values 5, 5, 3. P2: $10 > 20$ false; $10 > 4$ true, so $b \leftarrow c = 4$. P3: $10 > 20$ false; $10 > 30$ false, so $c \leftarrow a = 10$.

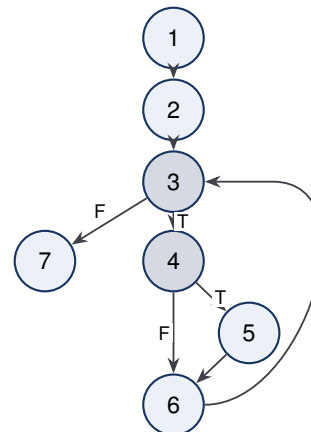
Worked Example 8.5 – Loop containing a decision

```

count = 0
i = 0
while i < n:
    if a[i] > 0:
        count = count + 1
    i = i + 1
print(count)

```

The function counts the positive elements of an array *a* of length *n*. Each line is a node.



Edges. $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 4, 3 \rightarrow 7, 4 \rightarrow 5, 4 \rightarrow 6, 5 \rightarrow 6, 6 \rightarrow 3$: $E = 8$; $N = 7$.

$$M = 8 - 7 + 2 = 3, \quad D + 1 = 2 + 1 = 3 \text{ (nodes 3 and 4).}$$

Check by merging. Nodes 1 and 2 are sequential; merging them gives $N = 6$ and $E = 7$, and $M = 7 - 6 + 2 = 3$, unchanged.

Path	Sequence	Input	Expected output
P1	1-2-3-7	$n = 0, a = []$	0
P2	1-2-3-4-6-3-7	$n = 1, a = [-2]$	0
P3	1-2-3-4-5-6-3-7	$n = 1, a = [5]$	1

P2 introduces edges $3 \rightarrow 4, 4 \rightarrow 6, 6 \rightarrow 3$; P3 introduces $4 \rightarrow 5$ and $5 \rightarrow 6$.

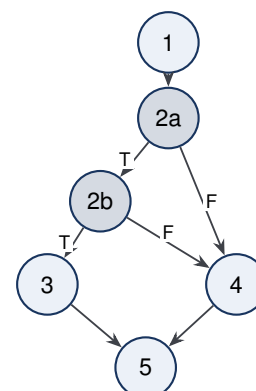
Worked Example 8.6 – Compound condition

```

read x, y
if (x > 0 AND y > 0)
    z = x + y
else
    z = 0
print z

```

The compound condition is split into two predicate nodes: 2a ($x > 0$) and 2b ($y > 0$). Nodes: 1, 2a, 2b, 3 ($z = x + y$), 4 ($z = 0$), 5 (print).



Edges. $1 \rightarrow 2a, 2a \rightarrow 2b, 2a \rightarrow 4, 2b \rightarrow 3, 2b \rightarrow 4, 3 \rightarrow 5, 4 \rightarrow 5$: $E = 7$; $N = 6$.

$$M = 7 - 6 + 2 = 3, \quad D + 1 = 2 + 1 = 3.$$

Path	Sequence	x	y	Expected z
P1	1-2a-4-5	-1	7	0
P2	1-2a-2b-4-5	2	-3	0
P3	1-2a-2b-3-5	2	3	5

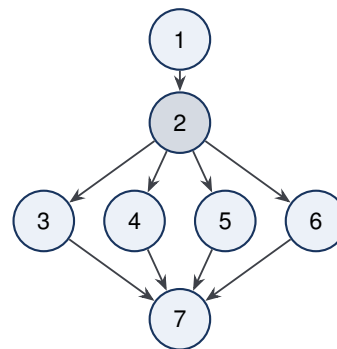
Why split? If the compound condition were drawn as a single node, M would be 2 and only one false case would be tested; an error such as writing OR for AND could then go undetected, since $x = -1, y = 7$ and $x = 2, y = -3$ behave differently under OR. Splitting forces both simple conditions to be exercised as false.

Worked Example 8.7 – Multi-way decision (switch)

```

read day;
switch (day) {
  case 1: print("Mon"); break;
  case 2: print("Tue"); break;
  case 3: print("Wed"); break;
  default: print("Invalid");
}
/* end */

```



Nodes: 1 = read; 2 = switch; 3, 4, 5 = cases; 6 = default; 7 = end.

Edges. $1 \rightarrow 2$, four edges from node 2, four edges into node 7: $E = 1 + 4 + 4 = 9$; $N = 7$.

$$M = 9 - 7 + 2 = 4.$$

Verification by decisions. Node 2 has $k = 4$ outgoing edges, so it contributes $k - 1 = 3$ binary decisions: $M = 3 + 1 = 4$. Counting the switch as a single decision ($D = 1, M = 2$) would be wrong.

Test cases. $\text{day} = 1 \rightarrow \text{Mon}$; $\text{day} = 2 \rightarrow \text{Tue}$; $\text{day} = 3 \rightarrow \text{Wed}$; $\text{day} = 9 \rightarrow \text{Invalid}$.

Worked Example 8.8 – Loop with nested decision and early exit

```

1 total = 0; i = 1
2 while i <= n:
3     if i % 2 == 0:
4         total = total + 2*i
5     else:
6         total = total + i
7     if total > 100:
8         break
9     i = i + 1
10 print (total)

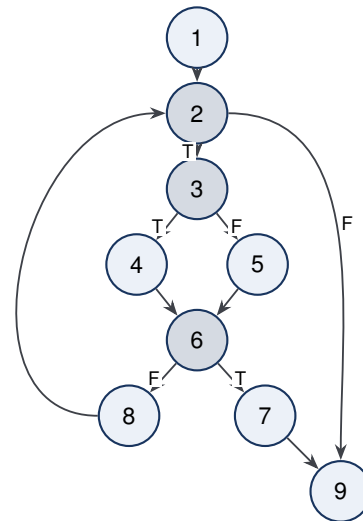
```

Nodes: 1 = initialise; 2 = $i \leq n$; 3 = $i \% 2 == 0$; 4 = even update; 5 = odd update; 6 = $\text{total} > 100$; 7 = break; 8 = $i = i + 1$; 9 = print. (Node numbers differ from listing line numbers because the else line is not a node.)

Edges. $1 \rightarrow 2, 2 \rightarrow 3, 2 \rightarrow 9, 3 \rightarrow 4, 3 \rightarrow 5, 4 \rightarrow 6, 5 \rightarrow 6, 6 \rightarrow 7, 6 \rightarrow 8, 7 \rightarrow 9, 8 \rightarrow 2$: $E = 11$; $N = 9$.

$$M = 11 - 9 + 2 = 4, \quad D + 1 = 3 + 1 = 4 \text{ (nodes 2, 3, 6).}$$

Trace of totals. i : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 give running totals 1, 5, 8, 16, 21, 33, 40, 56, 65, 85, 96, 120. The first total exceeding 100 is 120, at $i = 12$.



Path	Sequence	New edges	n	Output
P1	1-2-9	$1 \rightarrow 2, 2 \rightarrow 9$	0	0
P2	1-2-3-5-6-8-2-9	$2 \rightarrow 3, 3 \rightarrow 5, 5 \rightarrow 6, 6 \rightarrow 8, 8 \rightarrow 2$	1	1
P3	1-2-3-5-6-8-2-3-4-6-8-2-9	$3 \rightarrow 4, 4 \rightarrow 6$	2	5
P4	...-3-4-6-7-9 (loop until $i = 12$)	$6 \rightarrow 7, 7 \rightarrow 9$	12	120

Remarks. Paths P3 and P4 cannot be made shorter, because i starts at 1 (odd): the even branch can be reached only on a second iteration, and the break only after the total exceeds 100. Any $n \geq 12$ produces the output 120. All 11 edges are covered by the four tests.

8.5 Summary of Results**COMMON MISTAKE**

- Forgetting the back edge of a loop, which reduces E by one and M by one.
- Counting a node with a join (several incoming edges) as a decision. Only *outgoing* branching makes a predicate node.
- Writing $M = E - N + 2P$ and then substituting P = number of predicates.
- Listing more independent paths than M , or listing a path that adds no new edge.

Table 8.1. Summary of the worked cyclomatic-complexity examples.

Example	Program	E	N	D	$E - N + 2$	$D + 1$	R
8.1	Larger of A, B	7	7	1	2	2	2
8.2	Nested if-else	7	6	2	3	3	3
8.3	Summation loop	7	7	1	2	2	2
8.4	Flowchart (a, b, c)	10	9	2	3	3	3
8.5	Count positives	8	7	2	3	3	3
8.6	Compound condition	7	6	2	3	3	3
8.7	Switch (4 ways)	9	7	3*	4	4	4
8.8	Loop, break	11	9	3	4	4	4

* A 4-way switch counts as $4 - 1 = 3$ binary decisions.

- Omitting expected outputs from test cases; a test case without an expected result cannot pass or fail.

QUICK RECAP

- CFG: nodes are statements or blocks; edges are control transfers; predicate nodes branch.
- $M = E - N + 2P = D + 1 = R$; for one program $P = 1$.
- M = number of independent paths = minimum number of basis-path test cases.
- Procedure: CFG $\rightarrow M \rightarrow$ independent paths $\rightarrow$ test cases $\rightarrow$ execution.
- Split compound conditions; count a k -way switch as $k - 1$ decisions; mark infeasible paths explicitly.

Graph Matrices and Connection Matrices

A flow graph is an effective aid in path testing, but as the graph grows, tracing paths by hand becomes difficult and error-prone: a link may be missed or covered twice. A software tool can manage this, and the *graph matrix* is the data structure on which such a tool is built. This chapter explains graph matrices, connection matrices and the matrix method for computing cyclomatic complexity, with every intermediate calculation shown.

9.1 Graph Matrices

DEFINITION – Graph Matrix

A graph matrix is a square matrix whose number of rows and columns equals the number of nodes in the flow graph. Each row and each column identifies a particular node, and each entry records a direct connection (link) from the row node to the column node.

The course material states four conventions:

1. Each cell represents a direct connection or link from one node to another.
2. Direction matters: a connection from node *a* to node *b* does *not* imply a connection from *b* to *a*. The matrix is therefore generally not symmetric.
3. Digits are used to name nodes and letters to name edges (links).
4. An empty cell means that there is no direct link.

IMPORTANT NOTE – Reading a graph matrix

The entry in row *i*, column *j* is the link *from* node *i* *to* node *j*. A row lists the *outgoing* links of a node; a column lists its *incoming* links. The number of entries in a row is the out-degree of the node; a row with two or more entries is a decision node.

9.2 Connection Matrices

DEFINITION – Connection Matrix

A connection matrix is a graph matrix in which each cell entry is a *link weight*. In its simplest form the weight is 1 when a connection exists between two nodes and 0 otherwise; zeros are usually left blank to reduce clutter.

Adding link weights turns the graph matrix into a powerful testing tool, because the weight carries information about control flow. A connection matrix is used to compute the cyclomatic complexity of the flow graph.

SUPPLEMENTARY EXPLANATION – Other link weights

Weights other than 1/0 can be used to answer other questions: the probability that a link will be executed, the processing time spent traversing a link, the memory required, or the resources consumed. Path-level values are then obtained by combining the link weights along the path.

9.3 Computing Cyclomatic Complexity from a Connection Matrix

The course material gives the following procedure.

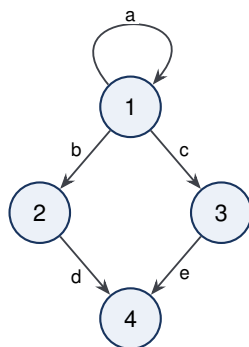
FORMULA 9.1 – Cyclomatic complexity from a connection matrix

$$M = \sum_{\text{rows } i \text{ with } c_i > 0} (c_i - 1) + 1$$

c_i = number of 1s (connections) in row i . Rows with $c_i = 0$ (exit nodes) are ignored.

1. For each row, count the number of 1s and write the count at the end of the row.
2. Subtract 1 from each count; ignore rows whose count is 0.
3. Add the results of Step 2 for all rows.
4. Add 1 to the sum from Step 3.
5. The final value is the cyclomatic complexity of the flow graph.

Why it works. A row with c_i outgoing links represents $c_i - 1$ binary decisions: a simple sequence node ($c_i = 1$) contributes 0, a two-way decision contributes 1, and a k -way decision contributes $k - 1$. The procedure therefore computes $D + 1$ directly from the matrix.

Worked Example 9.1 – Graph matrix with a self-loop (course example)

Links. a: $1 \rightarrow 1$ (self-loop); b: $1 \rightarrow 2$; c: $1 \rightarrow 3$; d: $2 \rightarrow 4$; e: $3 \rightarrow 4$.

Graph matrix.

	1	2	3	4
1	a	b	c	
2				d
3				e
4				

Connection matrix and row counts.

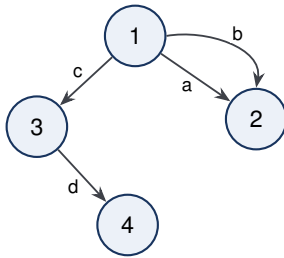
	1	2	3	4	1s in row	count - 1
1	1	1	1		3	$3 - 1 = 2$
2				1	1	$1 - 1 = 0$
3				1	1	$1 - 1 = 0$
4					0	ignore
Sum						2

$$M = 2 + 0 + 0 + 1 = 3.$$

Verification. $E = 5$ (a, b, c, d, e), $N = 4$: $M = E - N + 2 = 5 - 4 + 2 = 3$. ✓

Paths (course listing). 1-1 forms a cycle and is avoided when listing simple paths; the paths from entry to exit are 1-2-4 (links b, d) and 1-3-4 (links c, e). The third independent path is one that traverses the self-loop a before leaving node 1, e.g. 1-1-2-4 (links a, b, d), which adds the new edge a. This gives $M = 3$ independent paths.

Worked Example 9.2 – Parallel links (course example)



Task (course question). Represent the graph as a graph matrix, draw the connection matrix, find the cyclomatic complexity and list all paths.

Graph matrix. Two parallel links a and b run from 1 to 2, so the cell is written $a + b$.

	1	2	3	4
1		a+b	c	
2				
3				d
4				

Connection matrix (course solution). Each cell records whether a connection exists between the pair of nodes:

	1	2	3	4	1s in row	count - 1
1		1	1		2	$2 - 1 = 1$
2					0	ignore
3				1	1	$1 - 1 = 0$
4					0	ignore
Sum						1

$$M = 1 + 0 + 1 = 2.$$

List of paths. 1-2 and 1-3-4. The two paths agree with $M = 2$.

Note on the convention. The course solution treats the parallel links $a + b$ as one connection between nodes 1 and 2. If each link is weighted separately, the cell holds 2, row 1 counts 3 links, and the procedure gives $M = (3 - 1) + 0 + 1 = 3$: the extra independent path is the one that uses link b instead of a. Follow the course convention unless a question specifies link-by-link weights, and state the assumption in the answer.

Worked Example 9.3 – Matrix method for the count-positives program

The CFG of Worked Example 8.5 has links a: $1 \rightarrow 2$, b: $2 \rightarrow 3$, c: $3 \rightarrow 4$, d: $3 \rightarrow 7$, e: $4 \rightarrow 5$, f: $4 \rightarrow 6$, g: $5 \rightarrow 6$, h: $6 \rightarrow 3$.

	1	2	3	4	5	6	7	1s	count – 1
1		1						1	0
2			1					1	0
3				1			1	2	1
4					1	1		2	1
5						1		1	0
6			1					1	0
7								0	ignore
Sum									2

$$M = 2 + 1 = 3,$$

in agreement with $E - N + 2 = 8 - 7 + 2 = 3$ from Chapter 8. Note the back edge h appears in row 6, column 3, *below* the diagonal: entries below the diagonal indicate backward (loop) links when nodes are numbered in execution order.

Worked Example 9.4 – Matrix method for a nine-node graph

For the CFG of Worked Example 8.8 (links $1 \rightarrow 2$, $2 \rightarrow 3$, $2 \rightarrow 9$, $3 \rightarrow 4$, $3 \rightarrow 5$, $4 \rightarrow 6$, $5 \rightarrow 6$, $6 \rightarrow 7$, $6 \rightarrow 8$, $7 \rightarrow 9$, $8 \rightarrow 2$):

	1	2	3	4	5	6	7	8	9	1s	count – 1
1		1								1	0
2			1						1	2	1
3				1	1					2	1
4						1				1	0
5						1				1	0
6							1	1		2	1
7									1	1	0
8		1								1	0
9										0	ignore
Sum											3

$$M = 3 + 1 = 4 \quad (\text{check: } E - N + 2 = 11 - 9 + 2 = 4).$$

The total number of 1s in the matrix, 11, equals E ; the number of rows, 9, equals N . Both are useful self-checks.

9.4 Producing Paths from a Graph Matrix

The course material notes that the graph matrix is used to produce the set of all possible paths between nodes. Starting at the entry row, one follows each non-empty cell to the row of its column node, continuing until an exit row (a row with no entries) is reached. For Worked Example 9.1: row 1 leads to 2 (b) and 3 (c); row 2 leads to 4 (d); row 3 leads to 4 (e); row 4 is empty. The paths are therefore 1–2–4 and 1–3–4, while the entry a in cell (1,1) signals a cycle.

SUPPLEMENTARY EXPLANATION – Matrix multiplication and two-link paths

If the graph matrix A holds link names, the product A^2 lists all paths of exactly two links: entry (i, j) of A^2 is $\sum_k A_{ik}A_{kj}$, written as concatenations of link names. For Worked Example 9.1,

$$(A^2)_{1,4} = A_{1,2}A_{2,4} + A_{1,3}A_{3,4} = bd + ce,$$

which are exactly the two entry-to-exit paths. Similarly $(A^2)_{1,2} = ab$ and $(A^2)_{1,3} = ac$ show paths that pass through the self-loop. This algebra is the basis of path-tracing tools.

EXAM TIP

In a matrix question, draw both matrices with node numbers on both axes, write the row counts in a separate column exactly as in the course layout, show “count – 1” for every row and write “ignore” for empty rows. Then cross-check with $E - N + 2P$. Examiners award the procedure marks even if a single arithmetic slip occurs.

COMMON MISTAKE

- Filling cell (j, i) instead of (i, j) : rows are *from* nodes, columns are *to* nodes.
- Subtracting 1 from an empty row, which gives -1 and an incorrect total; empty rows are ignored.
- Forgetting the final “+1”.
- Making the matrix symmetric, as if the graph were undirected.

QUICK RECAP

- Graph matrix: $N \times N$; cell (i, j) names the link from i to j ; direction matters.
- Connection matrix: link weights, simplest form 1 for a connection.
- $M = \sum(\text{row count} - 1) + 1$ over non-empty rows; equals $D + 1$ and $E - N + 2$.
- Graph matrices support automated path tracing; parallel links follow the course convention.

Black-Box Testing

Black-box testing designs tests from the specification alone. It complements white-box testing: where basis path testing ensures that code is exercised, black-box techniques ensure that the required behaviour is checked, including behaviour at boundaries and in combinations of conditions that the code might not handle. This chapter treats the four techniques in the course material—equivalence class partitioning, boundary value analysis, decision-table testing and state transition testing—each with complete worked examples.

10.1 Concept

DEFINITION – Black-Box Testing

Black-box testing checks the functionality of a system without knowledge of its internal structure or design, by comparing the outputs produced for given inputs with the expected outputs. It is also known as *behavioural testing*, and it can be performed by a tester with no knowledge of the internal structure.

Black-box testing is used for the following purposes (the course material's categories):

- to test modules independently;
- to test the functional validity of the software;
- to look for interface errors;
- to test system behaviour and check performance; and
- to test the maximum load or stress on the system.

Table 10.1. White-box versus black-box testing.

Aspect	White-box testing	Black-box testing
Other names	Clear, open, glass box	Behavioural testing
Basis for tests	Internal code, logic, structure	Requirements and specification
Tester knowledge	Programming and code access required	No knowledge of internals needed
Typical level	Unit (component) testing	Integration, system, acceptance testing
Techniques	Basis path, coverage, control-structure testing, graph matrices	ECP, BVA, decision tables, state transitions
Finds	Logic errors, broken paths, unreachable code	Missing or incorrect functions, interface, behaviour and performance errors
Weakness	Cannot detect missing functionality	Cannot reveal untested internal paths

10.2 Equivalence Class Partitioning

DEFINITION – Equivalence Class Partitioning

Equivalence class partitioning (ECP) divides the input data of an application into groups, called equivalence classes, such that all members of a class are expected to behave in the same way. One representative value from each class is tested, on the assumption that if it works correctly, all other values in the class will too.

The technique reduces the number of test cases while ensuring comprehensive coverage. Three terms are used:

- **Equivalence class:** a set of inputs expected to behave the same way.
- **Valid equivalence class:** inputs that are valid according to the requirements; the system should produce correct outputs for them.
- **Invalid equivalence class:** inputs that are invalid; they represent unexpected inputs or error conditions that the system should handle gracefully.

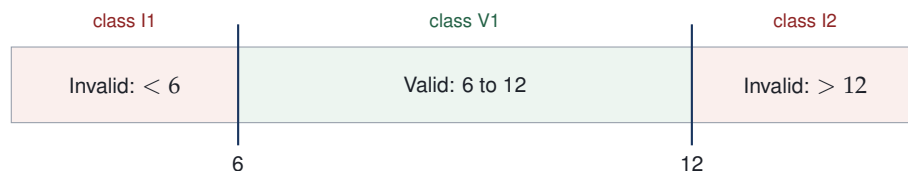


Figure 10.1. Equivalence classes for a password length that must be 6 to 12 characters.

Worked Example 10.1 – Password length, 6–12 characters (course example)

Problem. A login system accepts a password only if its length is between 6 and 12 characters inclusive. Any password shorter than 6 or longer than 12 characters is invalid. Identify the valid and invalid equivalence classes and create test cases.

Step 1 — Understand the input domain. The input is the password length L . Valid: $6 \leq L \leq 12$. Invalid: $L < 6$ or $L > 12$.

Step 2 — Identify the equivalence classes.

- V1 (valid): $6 \leq L \leq 12$.
- I1 (invalid, too short): $L < 6$.
- I2 (invalid, too long): $L > 12$.

Step 3 — Select representative values. V1: "abcdef" ($L = 6$); I1: "abc" ($L = 3$); I2: "abcdefghijklmnop" ($L = 16$).

Step 4 — Write the test cases.

TC	Input (password)	Expected output	Class
TC1	"abcdef" (length 6)	Valid	V1
TC2	"abcdefghijkl" (length 12)	Valid	V1
TC3	"abc" (length 3)	Invalid (too short)	I1
TC4	"abcdefghijklmnop" (length 16)	Invalid (too long)	I2

Minimum. Three test cases, one per class, satisfy ECP; the course solution adds TC2 as a second valid value. Values such as length 9 are equally good representatives of V1.

SUPPLEMENTARY EXPLANATION – Guidelines for deriving equivalence classes

1. An input *range* gives one valid and two invalid classes (below and above the range).
2. A specific *value* gives one valid and two invalid classes.
3. A *member of a set* gives one valid and one invalid class.
4. A *Boolean* input gives one valid and one invalid class.

Classes for other properties of the input—type (non-numeric text in a numeric field), emptiness (no input) and format—should also be considered.

Worked Example 10.2 – Two input fields

An insurance form accepts *age* 18–60 and *policy type* from {Gold, Silver}. Derive the classes and a minimal test set.

Classes. Age: V1 $18 \leq \text{age} \leq 60$; I1 $\text{age} < 18$; I2 $\text{age} > 60$. Policy: V2 $\in \{\text{Gold}, \text{Silver}\}$; I3 any other value.

Test set. Valid classes can be combined in one test; each invalid class is tested alone, with all other inputs valid, so that the cause of rejection is unambiguous.

TC	Age	Policy	Classes covered	Expected
TC1	35	Gold	V1, V2	Accepted
TC2	12	Gold	I1	Rejected: age
TC3	70	Silver	I2	Rejected: age
TC4	35	Platinum	I3	Rejected: policy

10.3 Boundary Value Analysis

DEFINITION – Boundary Value Analysis

Boundary value analysis (BVA) tests the boundaries of input domains, selecting values just below, exactly on and just above each boundary, because errors frequently occur at the edges of equivalence classes.

Programmers often write $<$ where $\leq$ was intended, or index from 1 where indexing starts at 0. Such off-by-one errors leave typical values unaffected and are revealed only at boundaries. For an input range $[6, 12]$, the boundaries are 6 (minimum valid) and 12 (maximum valid); testing just inside, on and just outside them identifies these faults.

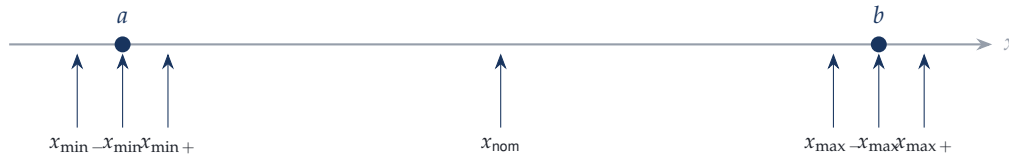


Figure 10.2. Boundary values for a range $[a, b]$: just below, on and just above each boundary, plus a nominal value.

Worked Example 10.3 – Password length by BVA (course example)

Step 1 — Identify the boundaries. Lower boundary: 6 characters (minimum valid length). Upper boundary: 12 characters (maximum valid length).

Step 2 — Select three values per boundary.

Boundary	Just below	On	Just above
Lower (6)	5 (invalid)	6 (valid)	7 (valid)
Upper (12)	11 (valid)	12 (valid)	13 (invalid)

Step 3 — Design the test cases.

TC	Test input	Classification	Expected result	Reason
TC1	"abcde" (length 5)	Invalid	Rejected (too short)	Just below lower boundary
TC2	"abcdef" (length 6)	Valid	Accepted	Exactly on lower boundary
TC3	"abcdefg" (length 7)	Valid	Accepted	Just above lower boundary
TC4	"abcdefghijk" (length 11)	Valid	Accepted	Just below upper boundary
TC5	"abcdefghijkl" (length 12)	Valid	Accepted	Exactly on upper boundary
TC6	"abcdefghijklm" (length 13)	Invalid	Rejected (too long)	Just above upper boundary

Fault detection. If the code were written `len > 6 && len < 12`, TC2 and TC5 would fail; if written `len >= 5`, TC1 would fail. A mid-range value such as 9 would pass in every case and detect nothing.

SUPPLEMENTARY EXPLANATION – Counting boundary-value test cases

In *single-fault* BVA, each of n input variables is set in turn to $x_{\min}$, $x_{\min+}$, $x_{\max-}$, $x_{\max}$ while the others take nominal values, and one all-nominal test is added. This gives $4n + 1$ test cases. Robustness testing adds $x_{\min-}$ and $x_{\max+}$ for each variable, giving $6n + 1$. The course password example, with three values per boundary, is a robustness-style selection for one variable (six tests, without the nominal value).

FORMULA 10.1 – Number of boundary-value test cases (supplementary)

$$T_{\text{BVA}} = 4n + 1, \quad T_{\text{robust}} = 6n + 1$$

n = number of independent input variables.

Worked Example 10.4 – Age field 18–60 by ECP and BVA

ECP. Classes: $age < 18$ (invalid), $18 \leq age \leq 60$ (valid), $age > 60$ (invalid). Representatives: 10, 35, 75.

BVA. Lower boundary 18: 17, 18, 19. Upper boundary 60: 59, 60, 61.

Combined minimal suite. {17, 18, 19, 35, 59, 60, 61}. The nominal value 35 covers the valid class; 17 and 61 also represent the two invalid classes, so separate values 10 and 75 are unnecessary. Seven tests cover both techniques.

Counting check. For $n = 1$, $4n + 1 = 5$ (18, 19, 35, 59, 60) and $6n + 1 = 7$ (adding 17 and 61). ✓

Table 10.2. Equivalence partitioning versus boundary value analysis.

Aspect	Equivalence partitioning	Boundary value analysis
Focus	Classes of inputs that behave alike	Edges of those classes
Values chosen	One representative per class (any member)	Values just below, on and just above each boundary
Password example	3 tests: lengths 3, 6 (or 9), 16	6 tests: lengths 5, 6, 7, 11, 12, 13
Detects	Wrong handling of a whole class	Off-by-one and relational-operator errors
Relationship	Applied first to find the classes	Applied second, at the class boundaries

10.4 Decision-Table-Based Testing

DEFINITION – Decision Table Testing

Decision table testing is a black-box technique for systems with complex business logic or multiple input conditions. A decision table lists all combinations of input conditions and the corresponding outputs (actions); each combination is a rule, and each rule yields a test case.

A decision table has four parts:

- **Conditions:** input conditions or criteria that affect the system's behaviour, e.g. "Is the user logged in?" or "Is the cart total above \$100?".
- **Actions:** outcomes or behaviours of the system, e.g. "Apply a 10% discount" or "Redirect to login page".

- **Rules:** each rule is a unique combination of conditions with its corresponding action(s); a complete table contains all possible rules.
- **Goal:** to identify and test systematically all combinations of inputs and their expected outputs.

FORMULA 10.2 – Number of rules in a complete decision table

$$\text{Rules} = 2^n$$

n = number of Boolean (true/false) conditions. For conditions with $v_1, v_2, \dots$ possible values, $\text{Rules} = v_1 \times v_2 \times \dots$.

Worked Example 10.5 – Login decision table (course example)

Conditions. C1: username correct (T/F); C2: password correct (T/F). **Actions.** E: show error message; H: show home page. Two Boolean conditions give $2^2 = 4$ rules.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

Test cases derived from the rules.

TC	Username	Password	Rule	Expected
TC1	invalid	invalid	R1	Error message
TC2	valid	invalid	R2	Error message
TC3	invalid	valid (of another user)	R3	Error message
TC4	valid	valid	R4	Home page

Worked Example 10.6 – E-commerce discount (conditions from the course material)

Specification. A logged-in user whose cart total exceeds \$100 receives a 10% discount. A logged-in user with a smaller total pays full price. A user who is not logged in is redirected to the login page at checkout, whatever the total.

Step 1 — Conditions. C1: user logged in? C2: cart total > \$100?

Step 2 — Actions. A1: apply 10% discount; A2: charge full price; A3: redirect to login page.

Step 3 — Rules ($2^2 = 4$):

	R1	R2	R3	R4
C1: Logged in?	T	T	F	F
C2: Total > \$100?	T	F	T	F
A1: Apply 10% discount	X			
A2: Charge full price		X		
A3: Redirect to login			X	X

Step 4 — Test cases. R1: logged in, total \$150 → pays \$135. R2: logged in, total \$80 → pays \$80. R3: not logged in, \$150 → login page. R4: not logged in, \$80 → login page.

Collapsing. R3 and R4 have the same action and differ only in C2, so they may be merged into one rule with C2 = “–” (don’t care), reducing the test count to 3. Combining with BVA, the total \$100.00 (not > 100) and \$100.01 should be added to test the boundary of C2.

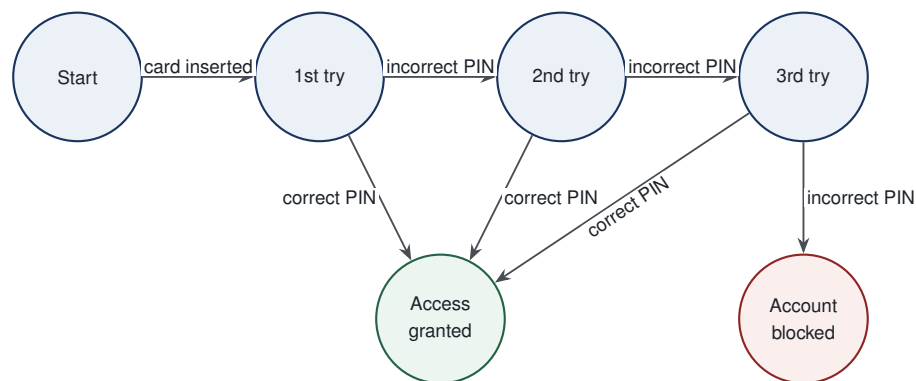
10.5 State Transition Testing

DEFINITION – State Transition Testing

State transition testing is a black-box technique for systems whose behaviour depends on their current state and on the inputs they receive, i.e. on the sequence of previous inputs. Its goal is to verify that the system moves between states correctly according to the defined rules.

The elements of the model are:

- **State:** a condition in which the system resides at a given time, e.g. an ATM in *Idle*, *Card Inserted*, *PIN Entered* or *Transaction Complete*.
- **Transition:** movement from one state to another in response to an input, e.g. inserting a card moves the ATM from *Idle* to *Card Inserted*.
- **Event/input:** the action or trigger that causes a transition, e.g. entering a PIN or cancelling.
- **Output/action:** the system’s behaviour on the transition, e.g. displaying a menu after a valid PIN.

Worked Example 10.7 – PIN entry with three attempts (course diagram)**Figure 10.3.** State transition diagram for PIN entry (course example redrawn).

Current state	Event	Next state	Output
Start	Card inserted	1st try	Ask for PIN
1st try	Correct PIN	Access granted	Show menu
1st try	Incorrect PIN	2nd try	Ask again
2nd try	Correct PIN	Access granted	Show menu
2nd try	Incorrect PIN	3rd try	Ask again (last attempt)
3rd try	Correct PIN	Access granted	Show menu
3rd try	Incorrect PIN	Account blocked	Retain card, block account

Test scenarios. (1) correct PIN first time; (2) one wrong then correct; (3) two wrong then correct; (4) three wrong → blocked. Scenario (4) is the most important negative test; together the four sequences cover all seven transitions.

Worked Example 10.8 – Login system (course example)

Problem. States: *Idle* (waiting for login), *Username Entered*, *Logged In* and *Error* (failed attempt); the course statement also lists *Password Entered*, which is transient and is merged into the transition from *Username Entered*. Events: enter a valid username; enter a valid password; enter an invalid password; retry; logout.

Step 1 — State transition diagram.

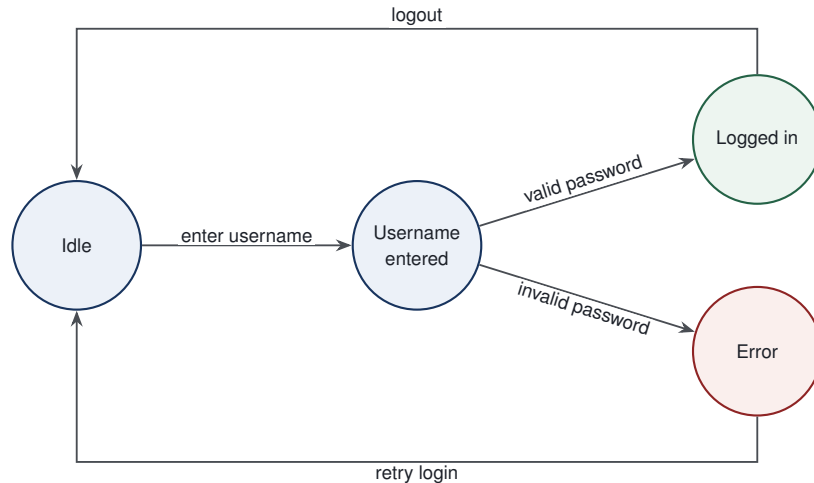


Figure 10.4. State transition diagram for the login system.

Step 2 — State transition table.

Current state	Event/input	Next state	Output/action
Idle	Enter username	Username entered	Prompt for password
Username entered	Enter valid password	Logged in	Display dashboard
Username entered	Enter invalid password	Error	Display error message
Error	Retry login	Idle	Restart login process
Logged in	Logout	Idle	Return to idle state

Step 3 — Test scenarios.

TC	Initial state	Input/event	Expected state	Expected output
TC1	Idle	Enter username	Username entered	Prompt for password
TC2	Username entered	Valid password	Logged in	Display dashboard
TC3	Username entered	Invalid password	Error	Display error message
TC4	Error	Retry login	Idle	Restart login process
TC5	Logged in	Logout	Idle	Return to idle state

Invalid transitions (supplementary). Events that are not allowed in a state should also be tested, e.g. *Logout* while in *Idle*, or *valid password* while in *Idle*; the expected result is that the state does not change.

COMMON MISTAKE

- In BVA, testing only the boundaries themselves (6 and 12) and omitting the values just outside (5 and 13).
- In ECP, placing two invalid classes in the same test case, so that the cause of rejection is unknown.

Table 10.3. Decision-table testing versus state transition testing.

Aspect	Decision table	State transition
Suited to	Combinations of conditions (business rules)	Behaviour depending on history (sequence of inputs)
Memory of past inputs	None: output depends on current conditions only	Essential: output depends on current state
Model	Table of conditions, actions, rules	State diagram and transition table
Test case	One per rule	One per transition or event sequence
Course example	Username/password combinations	PIN attempts, login states

- In decision tables, listing fewer than 2^n rules without justifying merged “don’t care” entries.
- In state transition testing, drawing states without labelling the events on the arrows.

QUICK RECAP

- Black-box (behavioural) testing uses the specification only.
- ECP: one representative per valid and invalid class; password: V1 6–12, I1 < 6, I2 > 12.
- BVA: just below, on, just above each boundary; password: 5, 6, 7, 11, 12, 13.
- Decision tables: conditions, actions, 2^n rules, one test per rule.
- State transition: states, events, transitions, outputs; diagram → table → scenarios.

Regression Testing and Mutation Testing

Software changes continually: defects are fixed, features added and code refactored. Regression testing asks whether a change has broken something that used to work. Mutation testing asks a different question: whether the test suite itself is good enough to notice a fault. This chapter treats both.

11.1 Regression Testing

DEFINITION – Regression Testing

Regression testing is software testing performed to ensure that recent changes—bug fixes, new features or code modifications—do not adversely affect the existing functionality of the application. It re-runs previously executed test cases to verify that the system still behaves as expected after the changes.

11.1.1 Why Regression Testing Is Needed

Modules are coupled. A change in one module can alter shared data, interfaces or timing and so break a feature that was never touched. A developer fixing the tax calculation may, for example, change a rounding helper that the invoice-printing module also uses. Such *unintended side effects* are called *regressions*: previously working behaviour has regressed to a failing state.

11.1.2 Key Characteristics

- **Purpose:** to detect unintended side effects caused by code changes.
- **Scope:** can be applied at the unit, integration and system levels.
- **Automation:** often automated, because regression tests are repeated frequently.

11.1.3 When Regression Testing Is Performed

- after bug fixes;
- after new features or functionality are added;
- after refactoring or optimising code; and
- before releasing a new version of the software.

11.1.4 Steps in Regression Testing

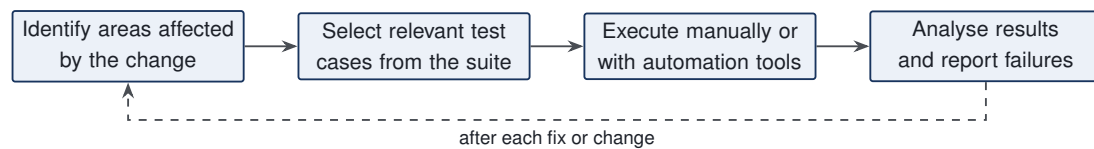


Figure 11.1. The regression testing cycle.

1. Identify the areas of the application affected by the changes (impact analysis).
2. Select relevant test cases from the existing test suite.
3. Execute the test cases manually or using automation tools.
4. Analyse the results and report any failures.

11.1.5 Selecting Test Cases

Re-running the entire suite (*retest all*) is safest but can be too slow. In practice the regression suite is built from three groups of tests: a representative sample that exercises all major functions (a “smoke” subset); tests that focus on functions likely to be affected by the change; and tests that focus on the components that were changed. The course material’s step “select relevant test cases” corresponds to this selection.

Worked Example 11.1 – Selecting a regression suite

An online store changes its *DiscountCalculator* to support percentage coupons. The existing suite has 400 tests: 60 for discounts, 90 for checkout and payment, 50 for invoice generation (which prints discounts), 200 for catalogue browsing and search.

Impact analysis. *DiscountCalculator* is called by checkout and by invoice generation; catalogue and search do not use it.

Selection. All 60 discount tests (changed component); 90 checkout and 50 invoice tests (affected functions); a sample of, say, 20 catalogue and search tests (representative smoke tests). Total $60 + 90 + 50 + 20 = 220$ tests instead of 400, plus new tests for percentage coupons, which are then added to the suite for future regression runs.

11.1.6 Advantages and Challenges

Advantages: ensures the stability of the application after changes; helps maintain quality over time; reduces the risk of introducing new defects.

Challenges: can be time-consuming if done manually; requires an up-to-date test suite; automation tools and scripts need frequent updating to keep pace with changes.

11.1.7 Regression Testing versus Retesting

Table 11.1. Regression testing versus retesting.

Aspect	Regression testing	Retesting (confirmation testing)
Purpose	Confirm that changes have not broken <i>unchanged</i> functionality	Confirm that a <i>specific reported defect</i> is fixed
Test cases	Previously passed tests	The test cases that previously failed
Scope	Wide: affected and related areas	Narrow: the defect only
Defect verification	Not aimed at a known defect	Aimed at a known defect
Automation	Highly suitable	Often manual, since each defect differs
Order	Usually after retesting succeeds	First, immediately after the fix

COMMON MISTAKE

Retesting and regression testing are not synonyms. After a fix, the failed test is *retested* to confirm the fix, and the surrounding suite is then run as a *regression* test to confirm that the fix did not break anything else.

11.2 Mutation Testing

11.2.1 Intuition

A fire alarm is tested by producing a little smoke on purpose: if the alarm stays silent, the alarm is at fault, not the building. Mutation testing does the same to a test suite. It plants small, deliberate faults in the code and checks whether the tests raise the alarm.

DEFINITION – Mutation Testing

Mutation testing is a white-box technique used to evaluate the quality of test cases. Small, deliberate changes (mutations) are made to the source code, and the test suite is run to check whether it detects them. If the tests fail on a mutated program, they are effective; if they pass, the tests may need improvement.

11.2.2 Terminology

- **Mutant:** a copy of the program containing one small change.
- **Mutation operator:** a rule that produces mutants, e.g. replace one arithmetic operator by another.
- **Killed mutant:** a mutant for which at least one test produces a result different from the original program, so the test suite fails.
- **Surviving mutant:** a mutant for which all tests still pass, indicating a potential weakness in the test suite.
- **Equivalent mutant** (supplementary): a mutant that is syntactically different but behaves identically to the original for every input; no test can kill it, so it is excluded from the score.

11.2.3 How Mutation Testing Works

1. **Mutant creation:** introduce small changes such as changing operators (+ to -), modifying conditions (`if (x > 5)` to `if (x >= 5)`), or removing or altering statements.
2. **Test execution:** run the test suite on each mutated program.
3. **Result analysis:** if the tests fail, the mutant is *killed*; if they pass, the mutant *survives*.

Table 11.2. Common mutation operators.

Operator	Original	Mutant
Arithmetic operator replacement	<code>a + b</code>	<code>a - b, a * b</code>
Relational operator replacement	<code>x > 5</code>	<code>x >= 5, x < 5</code>
Logical operator replacement	<code>a && b</code>	<code>a b</code>
Constant replacement	<code>x > 5</code>	<code>x > 6</code>
Statement deletion	<code>count++;</code>	(deleted)
Negation of condition	<code>if (valid)</code>	<code>if (!valid)</code>

11.2.4 Mutation Score

The key metric is the *mutation score*, the percentage of mutants killed by the test suite; a higher score indicates better test quality.

FORMULA 11.1 – Mutation score

$$\text{Mutation score} = \frac{\text{Killed mutants}}{\text{Total mutants} - \text{Equivalent mutants}} \times 100\%$$

The denominator is the number of relevant (non-equivalent) mutants. If no equivalent mutants are identified, it is simply the total number of mutants.

Worked Example 11.2 – Basic mutation score

A test suite is run against 120 mutants and kills 90. No equivalent mutants were found.

$$\text{Score} = \frac{90}{120} \times 100 = 75\%.$$

Thirty mutants survive; each survivor points to a missing or weak test.

Worked Example 11.3 – Score with equivalent mutants

Of 50 mutants, 35 are killed, 10 survive and 5 are shown to be equivalent. Improved tests later kill 7 more of the survivors. Find the score before and after.

$$\text{Before: } \frac{35}{50 - 5} \times 100 = \frac{35}{45} \times 100 = 77.78\%,$$

$$\text{After: } \frac{35 + 7}{45} \times 100 = \frac{42}{45} \times 100 = 93.33\%.$$

Check. $35 + 10 + 5 = 50$ mutants in total. After improvement, $10 - 7 = 3$ mutants still survive: $42 + 3 = 45$ relevant mutants. ✓

Worked Example 11.4 – Relational-operator mutants and boundary tests

Original: `if (x > 5) return "High"; else return "Low";` Mutants: M1 `x >= 5`; M2 `x < 5`; M3 `x > 6`. Test suite $T = \{x = 10, x = 2\}$.

Input	Original	M1 (≥ 5)	M2 (< 5)	M3 (> 6)
$x = 10$	High	High	Low	High
$x = 2$	Low	Low	High	Low
Status	–	survives	killed	survives

$$\text{Score} = \frac{1}{3} \times 100 = 33.33\%.$$

Improving the suite. Add $x = 5$: original gives Low, M1 gives High, so M1 is killed. Add $x = 6$: original gives High, M3 gives Low, so M3 is killed. With $T = \{10, 2, 5, 6\}$ all three mutants are killed: score $= \frac{3}{3} \times 100 = 100\%$. The mutants were killed by *boundary values*, showing why BVA and mutation analysis reinforce each other.

Worked Example 11.5 – Arithmetic-operator mutants

Original: `return a + b;` Mutants: M1 `a - b`; M2 `a * b`; M3 `a / b`. Test: $(a, b) = (2, 2)$, expected 4.

M1 gives 0 (killed); M2 gives 4 (survives, because $2 + 2 = 2 \times 2$); M3 gives 1 (killed). Score $= \frac{2}{3} \times 100 = 66.67\%$.

Adding the test $(3, 1)$, expected 4: M2 gives 3 and is killed. Score becomes 100%. The lesson is that test data should avoid values for which different operators coincide.

SUPPLEMENTARY EXPLANATION – Recognising an equivalent mutant

In `for (i = 0; i < n; i++)`, replacing `i < n` with `i != n` produces the same behaviour whenever $n \geq 0$, because `i` increases by 1 from 0 and reaches `n` exactly. If `n` can never be negative, the mutant is equivalent and is removed from the denominator.

11.2.5 Advantages and Challenges

Advantages: identifies weak or incomplete test cases; encourages thorough testing by making tests sensitive to code changes; provides a quantitative measure of test effectiveness.

Challenges: computationally expensive because of the large number of mutants; significant effort is needed to analyse results and improve tests; may produce false positives if mutations are unrealistic (and equivalent mutants must be identified by hand).

COMMON MISTAKE

- A *killed* mutant is good news for the test suite; a *surviving* mutant is bad news. Students frequently reverse this.
- Mutation testing evaluates the *tests*, not the program. A 100% score does not prove the program correct.
- Include equivalent mutants in the denominator only if the question gives no information about them.

QUICK RECAP

- Regression testing re-runs earlier tests after changes; performed after fixes, new features, refactoring and before release; often automated.
- Steps: identify affected areas, select tests, execute, analyse.
- Retesting confirms a specific fix; regression testing guards everything else.
- Mutation testing: create mutants, run tests, classify killed or surviving.
- $\text{Score} = \text{killed} / (\text{total} - \text{equivalent}) \times 100\%$.

Object-Oriented Testing

Object-oriented (OO) programs are built from classes whose objects hold state, hide their data and collaborate by sending messages. These features change what must be tested and how.

DEFINITION – Object-Oriented Testing

Object-oriented testing is a testing approach tailored to object-oriented software. Because OO programming emphasises encapsulation, inheritance and polymorphism, traditional techniques may not suffice; OO testing verifies the correctness of objects, classes and their interactions.

12.1 Why OO Software Needs a Different Approach

In procedural software the unit of testing is a function, whose output depends only on its inputs. In OO software the natural unit is a *class*: an operation's result depends on the object's current state, operations may be inherited or overridden, and the code executed for a call may be chosen only at run time. Testing an operation in isolation from its class is therefore of limited value.

12.2 Levels and Kinds of OO Testing

Table 12.1. Kinds of object-oriented testing.

Kind	What is tested	Focus
Class testing	Individual classes in isolation	Methods, attributes and their behaviour; state changes
Integration testing	Interactions between classes and objects	Objects collaborate correctly (message passing)
Inheritance testing	Derived classes	They inherit and override methods correctly
Polymorphism testing	Dynamic binding	Overridden methods behave as expected when called through a base reference
Scenario-based testing	Real-world use cases	Multiple objects interacting to achieve a user goal

12.2.1 Class Testing

Listing 12.1. A class whose behaviour depends on state.

```
class Account {
    private double balance = 0;
    private boolean frozen = false;
```

```

1 public void deposit(double amt) {
2     if (frozen || amt <= 0) throw new IllegalStateException();
3     balance += amt;
4 }
5 public void withdraw(double amt) {
6     if (frozen || amt > balance) throw new IllegalStateException();
7     balance -= amt;
8 }
9
10 public void freeze() { frozen = true; }
11 public double getBalance() { return balance; }
12 }

```

Class testing exercises *sequences* of operations, because the result of `withdraw` depends on what happened before. Useful sequences are: `deposit(100); withdraw(40); getBalance()` → 60; `withdraw(10)` on a new account → exception (insufficient balance); `deposit(50); freeze(); withdraw(10)` → exception (frozen). The attribute `balance` is private (encapsulation), so the test observes it only through `getBalance()`.

12.2.2 Inheritance Testing

Listing 12.2. A derived class that overrides an inherited method.

```

1 class SavingsAccount extends Account {
2     private static final double MIN = 500;
3     @Override
4     public void withdraw(double amt) {
5         if (getBalance() - amt < MIN) throw new IllegalStateException();
6         super.withdraw(amt);
7     }
8 }

```

Tests that passed for `Account` must be re-run in the context of `SavingsAccount`, because an inherited or overridden method operates on a new class with new rules. A `SavingsAccount` with balance 1000 must allow `withdraw(500)` but reject `withdraw(501)`. Conversely, a change in the base class `Account` can affect every derived class, requiring extensive retesting.

12.2.3 Polymorphism Testing

Listing 12.3. Dynamic binding through a base-class reference.

```

1 abstract class Shape { abstract double area(); }
2 class Circle extends Shape { double r; Circle(double r){this.r=r;}
3     double area(){ return Math.PI*r*r; } }
4 class Rect extends Shape { double w,h; Rect(double w,double h){this.w=w;
5     this.h=h;}
6     double area(){ return w*h; } }
7
8 double total(List<Shape> shapes) {
9     double sum = 0;
10    for (Shape s : shapes) sum += s.area();    // bound at run time
11    return sum;
12 }

```

The call `s.area()` may execute `Circle.area` or `Rect.area`; which one is decided at run time. Polymorphism testing supplies a test for *each* possible binding: `total([Rect(2,3)]) → 6`; `total([Circle(1)]) →  $\pi \approx 3.1416$` ; and a mixed list `[Rect(2,3), Circle(1)] →  $6 + \pi \approx 9.1416$` . If a new subclass is added, a new test binding must be added.

12.2.4 Scenario-Based Testing

Scenario-based testing follows a real use case across several objects. For an online library: a `Member` searches the `Catalogue`, reserves a `Book`, a `Loan` object is created, and a `Notification` is sent. The test checks the final state of all objects together: book unavailable, loan recorded, member's loan count incremented, notification queued.

12.3 Challenges in OO Testing

- **Encapsulation:** limited visibility of private methods and attributes makes testing harder; state must be observed through public operations.
- **Inheritance:** changes in base classes can affect derived classes, requiring extensive retesting.
- **Polymorphism:** dynamic method binding makes it hard to predict which method will execute at run time.
- **State dependencies:** objects maintain internal state, which makes it difficult to isolate tests.

12.4 Techniques for OO Testing

- **Random testing:** generates random inputs (and random operation sequences) to test object behaviour.
- **Fault-based testing:** introduces faults to identify weaknesses in the design; mutation testing (Chapter 11) is a fault-based technique.
- **Scenario-based testing:** simulates real-world scenarios to test object interactions.
- **Use-case testing:** tests based on user-defined use cases to ensure that the system meets its requirements.

12.5 Advantages and Challenges

Advantages: aligns with OOP principles such as modularity and reuse; facilitates testing complex systems of interacting objects; encourages early detection of design flaws.

Challenges: testers need a deep understanding of OO concepts; test design is complex because of inheritance and polymorphism; objects are hard to isolate for unit testing because of dependencies.

QUICK RECAP

- OO testing verifies classes, objects and their interactions.
- Kinds: class, integration, inheritance, polymorphism, scenario-based.
- Challenges: encapsulation, inheritance, polymorphism, state dependencies.

Table 12.2. Class testing versus integration testing in OO systems.

Aspect	Class testing	Integration testing
Unit under test	One class in isolation	A group of collaborating classes
Focus	Methods, attributes, state sequences	Message passing and collaboration
Test design	Operation sequences on one object	Use cases, scenarios, collaboration threads
Dependencies	Replaced by stubs or mock objects	Real collaborating objects
Example	Account: deposit then withdraw	Member-Loan-Book borrowing

- Techniques: random, fault-based, scenario-based, use-case testing.

Testing Web-Based Systems

DEFINITION – Web-Based System

A web-based system is an application that runs in a web browser and interacts with users over the internet.

Testing such systems is crucial because they must function correctly across many devices, browsers and network conditions while maintaining performance, security and usability. Testing a web-based system therefore involves many types of testing to ensure functionality, reliability and user experience.

13.1 Why Web Testing Is Distinctive

- **Unknown client environment:** the same page is rendered by different browsers, versions, operating systems and screen sizes.
- **Networked, multi-tier architecture:** browser, web server, application server, APIs and database must all cooperate.
- **Open access:** any user anywhere may connect, so security, load and accessibility are critical.
- **Frequent releases:** continuous updates make regression testing essential.

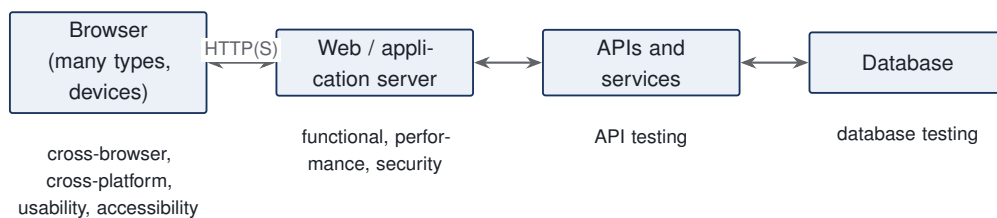


Figure 13.1. Tiers of a web-based system and the testing types that target each.

13.2 Types of Web Testing

The course material lists twelve types (Table 13.1).

Table 13.1. Types of testing for web-based systems.

Type	What it verifies	Realistic example
Functional testing	Features work as specified: forms, links, workflows	Registration form rejects an e-mail without “@”; checkout creates an order
Cross-browser testing	Consistent behaviour and rendering in different browsers and versions	Date picker works in Chrome, Firefox, Safari and Edge

Type	What it verifies	Realistic example
Cross-platform testing	Correct behaviour on different operating systems and devices	Site works on Windows, macOS, Android and iOS, on phone, tablet and desktop
Performance testing	Response time, load and stress behaviour, including network conditions	5,000 concurrent users during a sale; page load on a slow 3G connection
Security testing	Resistance to attacks; authentication, authorisation, encryption	SQL injection in the search box; session hijacking; HTTPS on all pages
Usability testing	Ease of use, navigation and user experience	New users complete a purchase without help within three minutes
Compatibility testing	Operation with different software, hardware and configurations	Works with browser extensions, different screen resolutions and zoom levels
Database testing	Data integrity, correct storage and retrieval, transactions	An order and its payment are saved atomically; no orphaned records after failure
API testing	Correct requests, responses, status codes and error handling of services	GET /orders/42 returns 200 with correct JSON; invalid token returns 401
Accessibility testing	Usability by people with disabilities, per accessibility guidelines	Screen reader reads form labels; all functions usable by keyboard; sufficient contrast
Localisation and internationalisation testing	Adaptation to languages and regions; locale-independent core	Tamil and Hindi text display correctly (Unicode); dates in DD/MM/YYYY; currency in rupees (INR)
Regression testing	Existing features still work after changes	After a new payment gateway is added, login, cart and search still pass

IMPORTANT NOTE

The course slide lists “Localization and Internalization Testing”; “internalization” is a typographical slip for *internationalisation*.

Worked Example 13.1 – Planning a browser–platform test matrix

A ticket-booking site must support Chrome, Firefox, Safari and Edge, on Windows, macOS and Android. How many configurations exist, and how can the effort be reduced?

Configurations. 4 browsers $\times$ 3 platforms = 12 combinations. Some are unrealistic (Safari is not generally available on Windows or Android, and Edge on Android is rare), so the feasible set is smaller: Chrome, Firefox and Edge on Windows (3); all four on macOS (4); Chrome and Firefox on Android (2), giving 9 configurations.

Reduction. Run the full functional suite on the most-used configurations (e.g. Chrome/Windows and Chrome/Android), and a smaller smoke suite on the rest; automate with a tool such as Selenium so that regression runs cover all nine.

Table 13.2. Cross-browser versus cross-platform testing.

Aspect	Cross-browser testing	Cross-platform testing
Varies	Browser and browser version	Operating system and device
Typical defects	Rendering differences, unsupported CSS/JavaScript features	Layout on small screens, touch input, OS-specific fonts or file handling
Example	Layout correct in Chrome, broken in Safari	Works on desktop, but buttons too small on Android phones

Table 13.3. Accessibility, localisation and internationalisation testing.

Aspect	Accessibility	Localisation	Internationalisation
Question	Can people with disabilities use it?	Is it correct for one specific locale?	Can it be adapted to any locale without redesign?
Checks	Screen readers, keyboard use, contrast, captions	Translations, date/number/currency formats, culture	Unicode support, externalised text, flexible layouts, right-to-left support
Example	Alt text on images	German version fits longer words	Adding Arabic needs no code change
Relation to UI design	“Application accessibility” design issue	“Localization features”	“Internationalization” design issue (Unicode)

QUICK RECAP

- Web systems must work across devices, browsers and networks while staying fast, secure and usable.
- Twelve types: functional, cross-browser, cross-platform, performance, security, usability, compatibility, database, API, accessibility, localisation and internationalisation, regression.
- Cross-browser varies the browser; cross-platform varies the OS and device.

Master Comparison Tables

This appendix collects, in one place, every comparison that examiners commonly ask for. Each table is self-contained; where a fuller version appears in a chapter, the chapter reference is given.

Table A.1. Design versus coding.

Aspect	Design	Coding
Purpose	Plan structure, data, interfaces, behaviour	Implement the plan
Abstraction	High: models, diagrams, pseudocode	Low: executable statements
Input / output	Requirements model → design models	Design model → source code
Quality check	Technical reviews	Unit tests, code review
Principle	“Design is not coding”	“Coding is not design”

Table A.2. Abstraction versus modularity.

Aspect	Abstraction	Modularity
Idea	Hide detail, show essentials	Divide the system into modules
Question answered	<i>What</i> does it do (not how)?	<i>Into which parts</i> is it divided?
Forms	Procedural, data, control	Modules, components, subsystems
Example	<code>calculateInterest()</code> hides the formula	<code>add()</code> , <code>divide()</code> as separate functions
Evaluated by	Level of detail exposed	Decomposability, composability, understandability, continuity, protection

Table A.3. Modularity versus separation of concerns.

Aspect	Separation of concerns	Modularity
Nature	Principle (divide and conquer)	Structural technique realising the principle
Unit	A concern: feature or behaviour	A module: named, separately addressable component
Output	Smaller sub-problems	Independent modules integrated into a system
Risk	None inherent	Under- or overmodularity

Table A.4. Cohesion versus coupling.

Aspect	Cohesion	Coupling
Measures	Relatedness <i>within</i> a module	Interdependence <i>between</i> modules
Desired level	High	Low
Best type	Functional	Data
Worst type	Coincidental	Content
Scale (worst to best)	Coincidental, logical, temporal, communicational, functional	Content, common, external, control, stamp, data
Effect of good value	Easy to maintain, test, extend	Changes and errors stay local

Table A.5. High versus low cohesion.

Aspect	High cohesion	Low cohesion
Elements	Strongly related; one task	Loosely related or unrelated
Example	<code>calculate_average()</code>	Print report + clear screen + play music
Maintenance	Easy; change is localised	Hard; changes affect unrelated tasks
Testing	Clear expected behaviour	Many unrelated behaviours to test
Reuse	High	Low

Table A.6. High versus low coupling.

Aspect	High (tight) coupling	Low (loose) coupling
Dependency	Strong: shared globals, internals, flags	Weak: only needed data passed
Example	A changes B. <code>internal_data</code>	<code>greet_user(name, age)</code>
Change impact	Ripples across modules	Confined to one module
Testing	Modules hard to test alone	Modules testable in isolation
Desirability	Avoid	Aim for

Table A.7. Horizontal versus vertical partitioning.

Aspect	Horizontal	Vertical (factoring)
Division	Branches/layers per major function (input, process, output; presentation, logic, data)	Control on top, workers below; functional subsystems
Merits	Easy testing and maintenance; fewer side effects; extensible	Worker changes have minimal impact; scalable
Demerits	More data passed; complex flow control	Control-module changes affect all subordinates

Table A.8. Transform versus transaction mapping.

Aspect	Transform mapping	Transaction mapping
Flow	Linear, sequential	Branches by transaction type
Centre	Transform centre	Transaction centre (dispatcher)
Structure	Main over input, transform, output controllers	Main over reception branch and dispatcher
Example	Payroll	ATM

Table A.9. Refactoring versus debugging.

Aspect	Refactoring	Debugging
Goal	Improve internal structure	Remove a defect
External behaviour	Unchanged	Changed (corrected)
Trigger	Low cohesion, redundancy, complexity, technical debt	A failure observed in testing or use
Verification	Regression tests must still pass	Retest of the failed case, then regression

Table A.10. White-box versus black-box testing.

Aspect	White box	Black box
Also called	Clear, open, glass box	Behavioural testing
Basis	Code and internal logic	Specification
Knowledge	Code access required	None of internals
Techniques	Basis path, coverage, graph matrices, condition testing, mutation	ECP, BVA, decision tables, state transitions
Finds	Logic errors, bad paths, dead code	Missing/incorrect functions, interface and behaviour errors

Table A.11. Manual versus automated testing.

Aspect	Manual	Automated
Execution	By a person	By scripts/tools (Selenium, JUnit)
Best for	Exploratory, usability, one-off tests	Regression, load, repetitive tests
Cost	Low setup, high per run	High setup, low per run
Consistency	Subject to human error	Repeatable

Table A.12. Functional versus non-functional testing.

Aspect	Functional	Non-functional
Question	What does it do?	How well does it do it?
Based on	Functional requirements	Quality attributes
Examples	Unit, integration, system, UAT	Performance, security, usability

Table A.13. Statement versus branch versus path coverage.

Aspect	Statement	Branch	Path
Requirement	Every statement once	Every decision outcome (T and F) once	Every (basis) path once
Formula	executed stmts / total	executed branches / total	executed paths / total
Strength	Weakest	Medium; subsumes statement	Strongest; subsumes branch
if without else	One true test may suffice	True and false tests needed	All combinations with other decisions

Table A.14. Equivalence partitioning versus boundary value analysis.

Aspect	ECP	BVA
Chooses	One value per class	Values at and around boundaries
Password 6–12	3, 6 (or 9), 16	5, 6, 7, 11, 12, 13
Finds	Class-level handling errors	Off-by-one and relational errors

Table A.15. Decision table versus state transition testing.

Aspect	Decision table	State transition
Use when	Many condition combinations	Behaviour depends on history
Model	Conditions, actions, rules (2^n)	States, events, transitions
Test per	Rule	Transition or sequence

Table A.16. Regression testing versus retesting.

Aspect	Regression	Retesting
Purpose	Nothing else broke	The specific defect is fixed
Tests	Previously passed tests	Previously failed tests
Scope	Wide	Narrow
Automation	Ideal	Usually manual

Table A.17. Class testing versus integration testing (OO).

Aspect	Class testing	Integration testing
Unit	One class	Collaborating classes
Focus	Methods, attributes, state	Messages and collaboration
Example	Account sequences	Member–Loan–Book scenario

Table A.18. Cross-browser versus cross-platform testing.

Aspect	Cross-browser	Cross-platform
Varies	Browser and version	OS and device
Example defect	CSS feature unsupported in one browser	Layout broken on a small Android screen

Table A.19. Accessibility versus localisation versus internationalisation.

Aspect	Accessibility	Localisation	Internationalisation
Concern	Users with disabilities	One target locale	Any locale, without redesign
Example	Screen reader, keyboard use	Hindi translation, INR currency	Unicode, externalised strings

Formula Handbook

Every formula used in Modules 4 and 5 is listed here with its variables, when to use it, a worked example and the most common mistake.

B.1 Cyclomatic complexity (edges and nodes)

$$M = E - N + 2P$$

Variables. E edges, N nodes, P connected components ($P = 1$ for one program).

When to use. Any CFG is given or drawn.

Example. $E = 10, N = 9, P = 1: M = 10 - 9 + 2 = 3$.

Common mistake. Using $P =$ predicates; forgetting loop back edges.

B.2 Cyclomatic complexity (decisions)

$$M = D + 1$$

Variables. D predicate nodes; a k -way decision counts $k - 1$.

When to use. Quick check, or when only code is given.

Example. Nested if-else with two conditions: $M = 2 + 1 = 3$.

Common mistake. Counting a switch with 4 cases as 1 decision.

B.3 Cyclomatic complexity (regions)

$$M = R$$

Variables. R regions of the planar graph, including the outer region.

When to use. Verification from a drawn CFG.

Example. If-else CFG: 1 enclosed + 1 outer = 2.

Common mistake. Forgetting the outer region.

B.4 Cyclomatic complexity from a connection matrix

$$M = \sum_{c_i > 0} (c_i - 1) + 1$$

Variables. c_i = number of 1s in row i .

When to use. Graph/connection-matrix questions.

Example. Rows 3, 1, 1, 0: $(2) + (0) + (0) + 1 = 3$.

Common mistake. Subtracting 1 from empty rows; filling the matrix column-wise.

B.5 Statement coverage

$$\frac{\text{executed statements}}{\text{total statements}} \times 100\%$$

Variables. Counts of executable statements.

When to use. Evaluating a test set's statement coverage.

Example. 4 of 6 statements executed: 66.67%.

Common mistake. Counting non-executable lines such as `else` or braces.

B.6 Branch coverage

$$\frac{\text{executed branches}}{\text{total branches}} \times 100\%$$

Variables. Each two-way decision has 2 branches.

When to use. Evaluating decision coverage.

Example. 2 decisions, tests cover 3 outcomes: $3/4 = 75\%$.

Common mistake. Using the number of decisions as the denominator.

B.7 Path coverage

$$\frac{\text{executed paths}}{\text{total feasible paths}} \times 100\%$$

Variables. Paths from entry to exit; basis paths in practice.

When to use. Thorough testing of complex logic.

Example. 2 of 3 feasible paths: 66.67%.

Common mistake. Counting infeasible paths in the denominator without comment.

B.8 Mutation score

$$\frac{\text{killed mutants}}{\text{total mutants} - \text{equivalent mutants}} \times 100\%$$

Variables. Killed: detected by tests; equivalent: behave identically to original.

When to use. Evaluating test-suite quality.

Example. 35 killed, 50 total, 5 equivalent: $35/45 = 77.78\%$.

Common mistake. Dividing by the number of surviving mutants; treating survivors as good.

B.9 Decision-table rules

$$\text{Rules} = 2^n \quad (\text{or } v_1 \times v_2 \times \dots)$$

Variables. n Boolean conditions; v_i values of condition i .

When to use. Building a complete decision table.

Example. 3 Boolean conditions: $2^3 = 8$ rules.

Common mistake. Merging rules without marking “don’t care”.

B.10 Boundary-value test count (supplementary)

$$T = 4n + 1 \text{ (normal)}, \quad T = 6n + 1 \text{ (robust)}$$

Variables. n input variables.

When to use. Estimating the size of a BVA suite.

Example. 2 variables: $4(2) + 1 = 9$ normal, $6(2) + 1 = 13$ robust.

Common mistake. Applying it to the course’s 6-value password selection, which is a robustness selection without a nominal value.

B.11 Fitts’s law (supplementary)

$$T = a + b \log_2 \left(\frac{D}{W} + 1 \right)$$

Variables. T selection time; D distance to target; W target width; a, b constants.

When to use. Explaining the Fitts’s-law UI principle.

Example. $D = 800$, $W = 40$: index = $\log_2 21 \approx 4.39$ bits.

Common mistake. Thinking a smaller target is faster; it is slower.

B.12 Control-hierarchy check

$$\sum \text{fan-out} = \sum \text{fan-in} = \text{number of arrows}$$

Variables. Fan-out: modules called; fan-in: callers.

When to use. Verifying a structure-chart answer.

Example. Fan-outs 3, 2, 2, 1 and fan-ins 1, 1, 1, 1, 2, 2: both sum to 8.

Common mistake. Reversing fan-in and fan-out.

Definitions Bank

Concise definitions suitable for 2-mark answers, grouped by module and listed alphabetically within each group.

Module 4: Software Design

Abstraction

Hiding implementation details and exposing only the essential features of a system.

Architectural design

Design of the relationships between the major structural elements of the software.

Architecture

The structure of software components and their interactions; the framework for detailed design.

Cohesion

The degree to which the elements inside a module belong together.

Coincidental cohesion

Unrelated functions grouped randomly in one module; the lowest cohesion.

Common coupling

Several modules share the same global data.

Communicational cohesion

Functions grouped because they operate on the same data.

Component-level design

Conversion of architectural elements into procedural descriptions of components.

Content coupling

One module modifies the internal data or logic of another; the strongest coupling.

Control abstraction

Hiding control-flow mechanics (loops, iterators, menu drivers).

Control coupling

One module controls another's behaviour by passing a flag.

Control hierarchy

Organisation of modules showing how control flows from higher to lower modules.

Coupling

The degree of interdependence between modules.

Data abstraction

Hiding the implementation of data structures behind an interface.

Data coupling

Modules share only the necessary elementary data; the weakest coupling.

Depth

Number of levels of control in a hierarchy.

Design pattern

A proven, named solution to a recurring design problem.

External coupling

Modules depend on external systems (databases, files, services).

Fan-in

Number of modules that directly control a given module.

Fan-out

Number of modules directly controlled by a given module.

Fitts's law

Selection time grows with distance and shrinks with target size.

Functional cohesion

A module performs a single, well-defined task; the highest cohesion.

Horizontal partitioning

Division of the structure into branches or layers for major functions.

Implementation model

The system's look and feel together with its supporting documentation.

Internationalisation

Designing software with a locale-independent core adaptable to many locales.

Localisation

Adapting internationalised software to a specific locale.

Logical cohesion

Logically similar functions grouped; one selected by an external flag.

Mental model

The user's perception of how the system works.

Modularity

Dividing a system into independent modules that combine into a whole.

Overmodularity

Too many small modules, raising integration cost.

Procedural abstraction

A named procedure performing a task without exposing its implementation.

Refactoring

Improving internal structure without changing external behaviour.

Refinement

Top-down elaboration of detail, step by step, until code can be written.

Separation of concerns

Solving a complex problem by dividing it into independently solvable concerns.

Software design

Iterative translation of requirements into a blueprint for construction.

Stamp coupling

A whole data structure is passed when only part is needed.

Technical debt

Future rework cost incurred by expedient but poor design or code.

Temporal cohesion

Functions grouped because they execute at the same time.

Transaction mapping

Mapping a DFD with transaction flow to a dispatcher-based program structure.

Transform mapping

Mapping a DFD with transform flow to input, transform and output branches.

Undermodularity

Too few, overly large modules.

User model

Profile of the end users: demographics, abilities, goals.

Vertical partitioning

Factoring: control modules at the top, worker modules at the bottom.

Width

Overall span of control: the largest number of modules on any level.

Module 5: Software Testing

API testing

Verifying requests, responses, status codes and error handling of services.

Basis path testing

Testing every independent path of the CFG at least once.

Black-box testing

Testing functionality from the specification without knowledge of internal code.

Boundary value analysis

Testing values just below, on and just above input boundaries.

Branch coverage

Percentage of decision outcomes executed by the tests.

Connection matrix

A graph matrix whose entries are link weights (1 for a connection).

Control flow graph

Graph of a program's logic: nodes are statements, edges are control transfers.

Cross-browser testing

Verifying behaviour across browsers and versions.

Cross-platform testing

Verifying behaviour across operating systems and devices.

Cyclomatic complexity

Number of independent paths in a CFG; $M = E - N + 2P$.

Decision table testing

Testing every rule (combination of conditions) of a decision table.

Equivalence class

A set of inputs expected to be processed the same way.

Equivalence class partitioning

Testing one representative from each equivalence class.

Equivalent mutant

A mutant behaviourally identical to the original program.

Graph matrix

Square $N \times N$ matrix recording links between flow-graph nodes.

Independent path

A path that introduces at least one edge not in earlier paths.

Killed mutant

A mutant detected by at least one failing test.

Mutation score

Percentage of non-equivalent mutants killed.

Mutation testing

Evaluating a test suite by seeding small faults (mutants) in the code.

Object-oriented testing

Testing classes, objects and their interactions, including inheritance and polymorphism.

Path coverage

Percentage of program paths executed.

Polymorphism testing

Testing each run-time binding of an overridden method.

Regression testing

Re-running earlier tests to ensure changes did not break existing functions.

Retesting

Re-running a failed test to confirm that a defect is fixed.

Software testing

Executing a program with the intent of finding errors and verifying requirements.

State transition testing

Testing transitions between system states caused by events.

Statement coverage

Percentage of executable statements executed.

Surviving mutant

A mutant not detected by any test.

User acceptance testing

Testing by end users to confirm readiness for deployment.

White-box testing

Testing based on the internal structure and code; clear/glass/open box.

Important Diagrams

Examiners expect certain diagrams to be drawn from memory. Table D.1 lists them, where each appears in this book, and the elements that must be present to earn full marks.

Table D.1. Diagrams to be able to draw.

Diagram	Figure	Must show
Requirements model to design model	1.2, p. 2	Four analysis element groups; pyramid with data/class at the base and component-level at the apex
Design process	1.3, p. 7	Interface, architecture, detailed design; generate–document–evaluate; final review
Modularity and cost	2.1, p. 15	Falling cost per module, rising integration cost, total curve, optimum M
Cohesion and coupling scales	3.1, 3.2	Correct order and direction (worst to best)
Control hierarchy	4.3, p. 30	Levels, labelled modules; depth, width, fan-in, fan-out annotated
Horizontal and vertical partitioning	4.4, 4.5	Layers/branches; control on top and workers below
Transform mapping (payroll)	4.8–4.10	DFD with incoming and outgoing boundaries; first- and second-level factoring
Transaction mapping (ATM)	4.11, 4.12	Transaction centre; dispatcher; one module per transaction
UI design models	5.1, p. 44	User, design, mental, implementation models
UI design process	5.2, p. 45	Spiral with analysis/modelling, design, construction, validation
Testing strategy	6.1, p. 51	Unit, integration, validation, system tests against code, design, requirements, system engineering
Types of testing	6.2, 6.3	Manual/automated, functional/non-functional, white/black; static versus dynamic
CFG notation	8.1, p. 61	Sequence, if, if–else, while, until, case
CFG worked examples	Chapter 8	Numbered nodes, labelled T/F edges, predicate nodes marked
Graph and connection matrices	Chapter 9	Node labels on both axes, link letters, row counts
Equivalence classes	10.1, p. 77	Valid region between two invalid regions with boundary values
Boundary values	10.2, p. 79	$x_{\min -}$, $x_{\min}$, $x_{\min +}$, x_{nom} , $x_{\max -}$, $x_{\max}$, $x_{\max +}$
State transition diagrams	10.3, 10.4	States, labelled event arrows, final states
Regression cycle	11.1, p. 87	Identify, select, execute, analyse, repeat

Web system tiers 13.1, p. 96 Browser, server, API, database with the testing types for each

EXAM TIP – Drawing technique

Use a ruler for structure charts; number every CFG node and label every decision edge T/F; write the figure title under the diagram; keep the diagram in the answer's flow, not at the end. A small, correct, labelled diagram earns more than a large unlabelled one.

Examination Answer Bank

Model answers are given at three lengths. A 2-mark answer is a precise definition with one supporting fact; a 5-mark answer adds explanation, a classification or small diagram, and an example; a 10-mark answer follows the structure Introduction → Definition → Explanation → Diagram/Table → Example → Advantages/Limitations → Conclusion.

E.1 2-Mark Answers

E.1 Define software design and state its position in the process. (2)

Software design is the iterative process of translating analysed requirements into a blueprint for constructing software. It follows requirements analysis and is the last modelling activity before construction (code generation and testing), and it is applied in every process model.

E.2 Name the four design models. (2)

Data/class design, architectural design, interface design and component-level design.

E.3 Distinguish fan-in and fan-out. (2)

Fan-out is the number of modules directly controlled by a module; fan-in is the number of modules that directly control it. High fan-in indicates reuse of a module.

E.4 What is refactoring? (2)

Refactoring is the reorganisation of a design or code to improve its internal structure without changing its external behaviour. It removes redundancy and low cohesion and reduces technical debt.

E.5 State the golden rules of UI design. (2)

Place the user in control; reduce the user's memory load; make the interface consistent.

E.6 Define cyclomatic complexity and give its formula. (2)

Cyclomatic complexity is the number of linearly independent paths in a program's control flow graph and hence the minimum number of basis-path test cases. $M = E - N + 2P$, where E is edges, N nodes and P connected components; equivalently $M = D + 1$.

E.7 Define mutation score. (2)

Mutation score is the percentage of (non-equivalent) mutants killed by a test suite: $\text{killed} / (\text{total} - \text{equivalent}) \times 100\%$. A higher score indicates a more effective test suite.

E.8 Differentiate regression testing and retesting. (2)

Retesting re-runs a failed test to confirm that a specific defect has been fixed. Regression testing re-runs previously passed tests to confirm that changes have not broken existing functionality.

E.2 5-Mark Answers**E.9 Explain the types of abstraction with examples. (5)**

Abstraction hides implementation details and exposes only essential features, reducing complexity by letting developers focus on what a system does rather than how. **Procedural abstraction** is a named procedure that performs a task without revealing its implementation, e.g. `calculateInterest(principal, rate, time)`. **Data abstraction** defines a data structure whose implementation is hidden behind operations, e.g. a `Calculator` class with a private `result` accessed only through `getResult()`. **Control abstraction** hides control-flow mechanics, e.g. a for-each loop or a menu-driven `runCalculator()` function that conceals the loop and `switch`. Abstraction is complemented by refinement, which reveals detail step by step.

E.10 Explain the criteria for evaluating modularity. (5)

Decomposability: the problem can be divided systematically into subproblems, e.g. banking into accounts, loans and transactions. **Composability**: existing modules can be assembled into new systems, e.g. a reusable payment gateway. **Understandability**: each module can be understood alone, e.g. authentication independent of the shopping cart. **Continuity**: small requirement changes affect individual modules only, e.g. tax-rule changes affect only the tax module. **Protection**: an error in one module does not spread, e.g. a seat-selection crash does not affect payment. The number of modules should be near the optimum M , avoiding under- and overmodularity.

E.11 Explain the four UI design models. (5)

The **user model** profiles the end users (demographics, abilities, goals) and classifies them as novices, knowledgeable intermittent users and knowledgeable frequent users. The **design model** is created by software engineers to satisfy the user model. The **mental model** is the user's perception of how the system works. The **implementation model** is the system's look and feel together with its documentation. The goal is to make the implementation model coincide with the mental model; the key principle is "know the user, know the tasks".

E.12 Explain boundary value analysis with the password example. (5)

BVA tests values at the edges of equivalence classes because errors cluster there. For a password of 6–12 characters, the boundaries are 6 and 12. Test values: 5 (invalid, just below), 6 (valid, on), 7 (valid, just above), 11 (valid, just below upper), 12 (valid, on), 13 (invalid, just above). Expected: 5 and 13 rejected, the rest accepted. These values detect off-by-one errors such as `len > 6` in place of `len >= 6`, which a mid-range value like 9 would miss.

E.13 Explain object-oriented testing and its challenges. (5)

OO testing verifies classes, objects and their interactions, because encapsulation, inheritance and polymorphism make procedural techniques insufficient. It comprises class testing (methods and state of one class), integration testing (collaboration between objects), inheritance testing (derived classes inherit and override correctly), polymorphism testing (each dynamic binding) and scenario-based testing (real use cases). Challenges: encapsulation limits visibility; base-class changes require retesting derived classes; dynamic binding makes the executed method hard to predict; internal state makes isolation difficult.

E.3 10-Mark Answers**E.14 Explain cohesion and coupling with their types. Why is “high cohesion, low coupling” desirable? (10)**

Introduction. Component-level design is evaluated through two complementary measures of functional independence: cohesion and coupling.

Definitions. Cohesion is the degree to which the elements inside a module belong together. Coupling is the degree of interdependence between modules.

Types of cohesion (lowest to highest). *Coincidental:* unrelated tasks grouped randomly (print report, clear screen, play music). *Logical:* similar tasks selected by a flag (one output handler for screen, printer, file). *Temporal:* tasks run at the same time (start-up initialisation). *Communicational:* tasks share the same data (get, log, display temperature). *Functional:* one well-defined task (compute an average).

Types of coupling (strongest to weakest). *Content:* one module modifies another's internals. *Common:* shared global data. *External:* dependence on external systems such as a third-party API. *Control:* a flag passed to control another module. *Stamp:* a whole record passed when a field is needed. *Data:* only required elementary data passed.

Diagram/table. Draw the two scales (Figures 3.1 and 3.2) or Table A.4.

Example. A `ReportManager` that fetches data, computes statistics and e-mails reports has low cohesion; splitting it into three functionally cohesive modules, passing only the data each needs, achieves high cohesion and data coupling.

Advantages of high cohesion and low coupling. Changes stay local (continuity); errors do not propagate (protection); modules are testable in isolation; modules can be reused (composability); teams work in parallel; each module is understandable alone.

Conclusion. High cohesion and low coupling together constitute functional independence, the central goal of modular design.

E.15 Explain transform mapping with the payroll example. (10)

Introduction. Architectural mapping converts a DFD into a program structure in six steps: identify flow type, define boundaries, map, define control hierarchy, refine, and elaborate.

Definition. Transform mapping applies to transform flow, in which data enters, is converted to an internal form, is processed at a transform centre, and leaves in external form along a linear path.

Steps. (1) Confirm transform flow. (2) Mark incoming and outgoing boundaries. (3) Identify the transform centre. (4) First-level factoring: main controller over input, transform and

output controllers. (5) Second-level factoring: each bubble becomes a module under its controller. (6) Refine for cohesion and coupling.

Example and diagrams. Payroll DFD: Collect Work Hours → Validate Data → Calculate Pay → Generate Payslip → Update Records, with Employee details fetched from a store. Incoming boundary after Validate Data, outgoing boundary before Generate Payslip. Input domain: collect and validate; transform centre: calculate pay; output domain: payslip and records. Draw Figures 4.8 and 4.10.

Refinement. Each worker is functionally cohesive and data coupled; small input modules may be merged; Calculate Pay may be factored into gross pay, deductions and net pay.

Advantages/limitations. Produces a factored, maintainable structure; boundary placement is a judgement, and different designers may place boundaries slightly differently.

Conclusion. Transform mapping systematically converts linear data flow into a hierarchical, functionally independent architecture.

E.16 Explain the golden rules of user interface design. (10)

Introduction. Interface design creates the communication medium between people and software; three golden rules guide it.

Rule 1: Place the user in control. Define interaction modes that do not force unnecessary actions; provide flexible interaction (keyboard, mouse, touch, voice); allow interruption and undo; streamline interaction for skilled users (macros); hide technical internals; allow direct manipulation of on-screen objects.

Rule 2: Reduce the user's memory load. Rely on recognition through visual cues; set meaningful defaults with reset; define intuitive shortcuts (Alt+P); use real-world metaphors (chequebook for bill payment); disclose information progressively (hierarchical menus).

Rule 3: Make the interface consistent. Organise visual information by uniform rules; standardise input mechanisms and navigation; provide context through titles, icons and colours; maintain consistency across a product family; respect established conventions (Alt+S to save).

Table. Tables 5.1–5.3 may be reproduced in condensed form.

Example. A word processor that lets users toggle spell-check mode, remembers recent files, and keeps the same toolbar in every window applies all three rules.

Conclusion. The rules shift the burden from the user to the system and underpin all later interface principles.

E.17 Explain basis path testing with an example. (10)

Introduction. Basis path testing is a white-box technique that derives test cases from the program's control flow graph.

Definition. It identifies a basis set of independent paths and executes each at least once; the number of such paths equals the cyclomatic complexity M .

Procedure. Draw the CFG; compute $M = E - N + 2P$ (verify with $D + 1$ and regions); list independent paths, each adding a new edge; design a test case for each path; execute and compare with expected output.

Example. For `if (a<b) F1(); else if (a<c) F2(); else F3();`: nodes 1 ($a<b$), 2 (F1), 3 ($a<c$), 4 (F2), 5 (F3), 6 (end); edges 1–2, 1–3, 3–4, 3–5, 2–6, 4–6, 5–6. $E = 7$, $N = 6$: $M = 7 - 6 + 2 = 3$; $D = 2$: $M = 3$. Paths: 1–2–6 ($a = 1, b = 5$), 1–3–4–6 ($a = 5, b = 2, c = 9$), 1–3–5–6 ($a = 5, b = 2, c = 3$), with expected outputs F1, F2, F3.

Advantages. Guarantees statement and branch coverage; gives a precise number of tests; exposes complex modules. **Limitations.** Cannot find missing functionality; loops create infeasible or numerous paths; compound conditions must be split.

Conclusion. Basis path testing gives a systematic, measurable minimum set of white-box tests.

E.18 Explain black-box testing techniques with examples. (10)

Introduction. Black-box (behavioural) testing designs tests from the specification, without knowledge of the code.

Equivalence class partitioning. Divide inputs into valid and invalid classes; test one value from each. Password 6–12: classes < 6, 6–12, > 12; tests lengths 3, 6, 16.

Boundary value analysis. Test just below, on and just above each boundary: 5, 6, 7, 11, 12, 13.

Decision table testing. List conditions, actions and 2^n rules. Login: username T/F $\times$ password T/F gives 4 rules; only T/T shows the home page; the others show an error.

State transition testing. Model states, events and transitions. PIN entry: Start $\rightarrow$ 1st try $\rightarrow$ 2nd try $\rightarrow$ 3rd try; a correct PIN from any try grants access; a third incorrect PIN blocks the account. Test each transition.

Table. Summarise with Table A.14 and Table A.15.

Advantages/limitations. Tester needs no code; tests reflect user requirements; but internal paths may remain untested and specifications may be incomplete.

Conclusion. The four techniques, applied together, give compact, requirement-driven test suites.

E.19 Explain mutation testing and compare it with regression testing. (10)

Introduction. Both techniques concern change: regression testing checks that code changes have not broken the software; mutation testing checks whether tests would notice a change.

Mutation testing. Small faults are seeded by mutation operators ($+\rightarrow-$, $>\rightarrow=>$, statement deletion). The suite is run on each mutant; a failing suite kills the mutant, a passing suite lets it survive. Mutation score = $\text{killed} / (\text{total} - \text{equivalent}) \times 100\%$. Example: 35 killed of 50 with 5 equivalent gives 77.78%.

Regression testing. After bug fixes, new features, refactoring and before release, relevant earlier tests are re-run: identify affected areas, select tests, execute (often automated), analyse.

Comparison. Mutation testing evaluates test quality by changing the code deliberately; regression testing evaluates the software after real changes. Mutation is white-box and computationally expensive; regression is usually black-box and routinely automated.

Conclusion. A suite with a high mutation score makes an effective regression suite; the techniques are complementary.

Practice Problems

The practice bank is graded from recall to analysis. Answers to all numerical problems in Section D are given in Appendix H; the other sections are answered by the chapter references shown in brackets.

Section A — 2-Mark Questions

- A1. State the three characteristics of a good design. [1.4]
- A2. Why is design described as “not coding”? [1.5]
- A3. Give one example each of procedural, data and control abstraction. [2.1]
- A4. What is a design pattern? State two decisions it helps a designer make. [2.2]
- A5. Define modular composability with an example. [2.4]
- A6. Distinguish undermodularity from overmodularity. [2.4]
- A7. What is the difference between refinement and abstraction? [2.5]
- A8. Why does refactoring reduce technical debt? [2.6]
- A9. Identify the cohesion of a module that initialises variables, opens files and clears the screen. [3.1]
- A10. Why is stamp coupling worse than data coupling? [3.2]
- A11. Define depth and width of a control hierarchy. [4.3]
- A12. State one merit and one demerit of horizontal partitioning. [4.4]
- A13. What is a flow boundary? [4.7]
- A14. What is a transaction centre? [4.8]
- A15. Classify users in the user model. [5.2]
- A16. Why is variability of system response time important? [5.4]
- A17. State the Fitts’s-law principle. [5.6]
- A18. Distinguish static and dynamic testing. [6.5]
- A19. Why is white-box testing called glass-box testing? [7.1]
- A20. Why does 100% statement coverage not imply 100% branch coverage? [7.2]
- A21. What is an independent path? [8.2]
- A22. What does a row in a connection matrix represent? [9.1]
- A23. Define an invalid equivalence class. [10.2]
- A24. List the four parts of a decision table. [10.4]
- A25. Define an equivalent mutant. [11.2]
- A26. Name four kinds of object-oriented testing. [12.2]
- A27. Differentiate cross-browser and cross-platform testing. [13.2]

Section B — 5-Mark Questions

- B1. Explain how the requirements model is translated into the four design models, with a diagram.
- B2. Explain any five design principles with examples.
- B3. Explain modularity using the calculator example and list its benefits.
- B4. Explain the architectural models with one example each.
- B5. Explain control hierarchy with a structure chart and compute its measures.
- B6. Explain horizontal and vertical partitioning with the library system.
- B7. Explain the UI design process spiral.
- B8. Explain the design issues of help facilities and error handling.
- B9. Explain the testing strategy diagram.
- B10. Explain the errors in conditions detected by control-structure testing.
- B11. Explain equivalence partitioning with an example of your choice.
- B12. Explain state transition testing with the PIN example.
- B13. Explain the steps and challenges of regression testing.
- B14. Explain the challenges and techniques of OO testing.
- B15. Explain any five web-testing types with realistic examples.

Section C — 10-Mark Questions

- C1. Discuss the types of cohesion and coupling with code examples and justify “high cohesion, low coupling”.
- C2. Explain architectural mapping; perform transform mapping for a payroll system.
- C3. Explain transaction mapping with the ATM example and refine the resulting structure.
- C4. Explain the golden rules and the sixteen UI principles, with examples.
- C5. Explain basis path testing; draw the CFG, compute cyclomatic complexity by three methods and design test cases for a program with a loop and a nested decision.
- C6. Explain graph and connection matrices and compute cyclomatic complexity from a matrix.
- C7. Explain all four black-box techniques with worked examples.
- C8. Explain mutation testing, derive mutation scores and relate it to boundary testing.

Section D — Numerical and Problem-Solving Questions

- D1. A CFG has 14 edges and 12 nodes. Find M .
- D2. A program has five binary decisions. Find the minimum number of basis-path test cases.
- D3. A function contains one `if` and one `switch` with five cases and a default. Find M .
- D4. A program consists of two independent functions whose combined graph has 15 edges and 13 nodes. Find M .
- D5. 200 mutants are generated; 150 are killed and 20 are equivalent. Find the mutation score.
- D6. An order quantity must be 1–99. List the BVA test values and their expected results.

- D7. How many rules does a complete decision table with four Boolean conditions have?
- D8. A connection matrix has row counts 2, 1, 3, 1, 2, 0. Find M .
- D9. A test run executes 18 of 24 statements. Find statement coverage.
- D10. A program has five binary decisions; tests exercise 7 outcomes. Find branch coverage.
- D11. Root R calls A and B ; A calls C , D and E ; B calls E and F ; E calls G . Find depth, width, and the fan-in and fan-out of every module.
- D12. How many normal and robust BVA test cases are needed for two input variables?
- D13. Draw the CFG, find M and design tests for the factorial loop: `read n; fact=1; i=1; while i<=n: fact=fact*i; i=i+1; print fact.`
- D14. Using the PIN state diagram, list the minimum event sequences that cover every transition.

Section E — Scenario-Based Questions

- E1. A developer merges report printing, e-mail sending and database backup into one “utility” module because all three are run at midnight. Identify the cohesion, explain the risk and propose a redesign.
- E2. A mobile banking app shows “Error 500” when a transfer fails, and loses the entered details. Identify the violated UI guidelines and rewrite the message.
- E3. After a tax-rule fix, invoices print with incorrect totals although invoice code was not touched. Name the testing activity that would have caught this and describe how you would select tests.
- E4. A test suite achieves 100% statement coverage but a defect in an untaken `else-less if` reaches production. Explain why and which coverage would have helped.
- E5. A web shop works in Chrome but the checkout button is hidden on Safari for iPhone. Classify the defect and plan the testing needed.
- E6. A class `Shape` gains a new subclass `Triangle`. Which OO testing activities must be repeated and why?
- E7. A kiosk offers Issue, Return, Renew and Search. Which mapping applies? Draw the structure.
- E8. A mutation run reports 40 surviving mutants of the form `>` replaced by `>=`. What does this tell you about the tests?

Section F — Diagram-Based Questions

- F1. Draw the design pyramid and map the analysis elements onto it.
- F2. Draw the design-process flow and explain the iteration points.
- F3. Draw the modularity–cost graph and mark under- and overmodular regions.
- F4. Draw a structure chart for an online examination system and annotate depth, width, fan-in and fan-out.
- F5. Draw the flow-graph notation for sequence, if–else, while, until and case.
- F6. Draw the testing-strategy spiral and explain each level.
- F7. Draw a state transition diagram for a traffic light with a pedestrian button.

F8. Draw the tiers of a web application and indicate which testing type targets each.

Mock Examinations

Each paper carries 50 marks and is designed for 90 minutes. Answer keys are in Appendix H; attempt each paper under timed conditions before consulting them.

G.1 Mock Examination 1

Time: 90 minutes

Maximum marks: 50

Part A — Answer all questions ($5 \times 2 = 10$)

1. Define coupling and list its six types from strongest to weakest.
2. Explain modular continuity with an example.
3. What is a transaction centre? Give an example.
4. For an age field with valid range 18–60, list the BVA values around the lower boundary with expected results.
5. What is a killed mutant?

Part B — Answer all questions ($4 \times 5 = 20$)

6. For the program below, draw the CFG, compute M by two methods, list the independent paths and give test cases.

```

read n
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
print fact

```

7. A flow graph has links $1 \rightarrow 2$ (a), $1 \rightarrow 3$ (b), $2 \rightarrow 4$ (c), $3 \rightarrow 4$ (d), $3 \rightarrow 5$ (e), $4 \rightarrow 6$ (f), $5 \rightarrow 6$ (g). Draw the graph matrix and connection matrix, compute the cyclomatic complexity and list all paths.
8. A test suite is run against 80 mutants; 56 are killed and 10 are equivalent. Compute the mutation score. How many more mutants must be killed to reach 90%?
9. An online application form: (i) clears every field when one entry is invalid; (ii) displays “Invalid input”; (iii) takes between 1 and 8 seconds unpredictably to respond. Identify the UI design issues and principles violated and propose corrections.

Part C — Answer both questions ($2 \times 10 = 20$)

10. A result-processing system reads student marks, validates them, computes totals and grades, formats a mark sheet and prints it. Identify the flow type, mark the boundaries, and derive the program structure by the appropriate mapping. Refine the result.
11. A bank approves personal loans using three conditions: C1 the applicant is salaried; C2 the credit score is at least 700; C3 the applicant has an existing loan. The loan is approved if C1 and C2 hold and C3 does not; it is referred to a manager if C2 holds and exactly one of "C1 is false" or "C3 is true" holds; otherwise it is rejected. Construct the complete decision table, derive test cases, and add boundary tests for the credit score.

G.2 Mock Examination 2**Time: 90 minutes****Maximum marks: 50****Part A — Answer all questions ($5 \times 2 = 10$)**

1. Define stepwise refinement.
2. What is the width of a control hierarchy?
3. State four characteristics of an effective error message.
4. Define equivalence class.
5. List four types of errors in conditions.

Part B — Answer all questions ($4 \times 5 = 20$)

6. Draw the CFG for the program and compute its cyclomatic complexity by all three methods; list the basis paths with test data.

```

1 read a, b, c
2 if a > b:
3     if a > c: big = a
4     else:    big = c
5 else:
6     if b > c: big = b
7     else:    big = c
8 print big

```

7. For the code below compute statement and branch coverage for the test set $\{x = 5\}$, and then for $\{x = 5, x = -20\}$.

```

1 x = read()           # S1
2 if x > 0:             # S2
3     y = x            # S3
4 else:
5     y = -x           # S4
6 if y > 10:           # S5
7     print("big")     # S6
8 print(y)             # S7

```

8. Distinguish regression testing from retesting. A defect in the shipping-cost calculation has been fixed; describe the testing to be performed next.
9. The connection matrix of a flow graph has 1s at (1,2), (2,3), (2,4), (3,5), (4,5), (5,2) and (5,6). Draw the flow graph and compute the cyclomatic complexity using the matrix; verify with $E - N + 2$.

Part C — Answer both questions ($2 \times 10 = 20$)

10. The following modules exist in a hostel management system. Identify the cohesion or coupling in each, and redesign the system for high cohesion and low coupling: (a) `misc()` that prints mess bills, resets passwords and archives logs; (b) `allot(room_record)` that uses only the room number; (c) `process(mode)` where `mode` selects `allot`, `vacate` or `transfer`; (d) modules that read and write a global `current_semester`; (e) `fee_due(days, rate)`.
11. An e-banking web portal is to be released. Explain the testing types you would apply, with a concrete example for each of at least eight types, and plan the browser–OS configurations for three browsers on two operating systems. Also explain how you would test the inheritance of `SavingsAccount` from `Account` in its back end.

Answer Keys

H.1 Mock Examination 1 — Answer Key

1. Coupling is the degree of interdependence between modules. Strongest to weakest: content, common, external, control, stamp, data. [1 + 1]
2. Modular continuity: a small change in requirements affects only individual modules, not the whole system. Example: a change in tax rules affects only the Tax module of an invoicing system. [1 + 1]
3. A transaction centre is the process in a transaction-flow DFD that evaluates an incoming transaction and routes it to one of several action paths, e.g. the Menu Selection Process of an ATM routing to Withdraw, Deposit or Balance Enquiry. [1 + 1]
4. 17 (invalid, rejected), 18 (valid, accepted), 19 (valid, accepted). [2]
5. A mutant for which at least one test produces a result different from the original program, so the test suite fails and the seeded fault is detected. [2]
6. Nodes 1–7 as numbered; node 4 ($i \leq n$) is the predicate. Edges: $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $4 \rightarrow 5$ (T), $5 \rightarrow 6$, $6 \rightarrow 4$ (back edge), $4 \rightarrow 7$ (F): $E = 7$, $N = 7$.

$$M = E - N + 2 = 7 - 7 + 2 = 2; \quad M = D + 1 = 1 + 1 = 2.$$

Paths: P1 1–2–3–4–7, test $n = 0$, output 1. P2 1–2–3–4–5–6–4–7, test $n = 3$, output 6. [CFG 1, M 2, paths and tests 2]

7. Graph matrix: row 1: a (col 2), b (col 3); row 2: c (col 4); row 3: d (col 4), e (col 5); row 4: f (col 6); row 5: g (col 6); row 6 empty.

	1	2	3	4	5	6	1s	–1
1		1	1				2	1
2				1			1	0
3				1	1		2	1
4						1	1	0
5						1	1	0
6							0	ignore

$M = (1 + 0 + 1 + 0 + 0) + 1 = 3$; check $E - N + 2 = 7 - 6 + 2 = 3$. Paths: 1–2–4–6, 1–3–4–6, 1–3–5–6. [matrices 2, M 2, paths 1]

8. Relevant mutants = $80 - 10 = 70$. Score = $\frac{56}{70} \times 100 = 80\%$. For 90%: $0.9 \times 70 = 63$ killed, i.e. $63 - 56 = 7$ more mutants. [3 + 2]

9. (i) Violates work-product integrity and the golden rule “place the user in control” (actions should be undoable, work preserved): keep valid entries and highlight only the faulty field. (ii) Violates error-handling guidelines (understandable language, actionable advice): e.g. “Phone

number must have 10 digits; you entered 9.” (iii) Response-time variability: aim for a consistent response and use latency reduction (progress indicator). [5]

10. Flow type: transform flow (linear). Input domain: Read Marks, Validate Marks; incoming boundary after validation. Transform centre: Compute Total and Grade. Output domain: Format Mark Sheet, Print Mark Sheet; outgoing boundary before formatting. First-level factoring: *Result System* over Input, Transform and Output controllers. Second-level: Read Marks and Validate Marks under Input; Compute Total and Compute Grade under Transform; Format and Print under Output. Refinement: separate total and grade computation for functional cohesion; pass only marks arrays and totals (data coupling); consider merging Read and Validate if trivial. [flow and boundaries 3, factoring diagrams 5, refinement 2]

11. $2^3 = 8$ rules.

	R1	R2	R3	R4	R5	R6	R7	R8
C1 Salaried	T	T	T	T	F	F	F	F
C2 Score ≥ 700	T	T	F	F	T	T	F	F
C3 Existing loan	T	F	T	F	T	F	T	F
Approve		X						
Refer	X					X		
Reject			X	X	X		X	X

Reasoning: R1 has C2 and only “C3 true” $\Rightarrow$ refer; R6 has C2 and only “C1 false” $\Rightarrow$ refer; R5 has both $\Rightarrow$ reject. Rules R3, R4, R7, R8 (C2 false) may be merged as one “reject” rule with C1, C3 = don’t care. Test cases: one per rule, e.g. R2: salaried, score 750, no loan $\rightarrow$ approve. Boundary tests on C2: scores 699 (C2 false), 700 and 701 (C2 true) with C1 = T, C3 = F, expected reject, approve, approve. [table 5, tests 3, BVA 2]

H.2 Mock Examination 2 — Answer Key

1. A top-down strategy in which a high-level function is elaborated step by step into more detailed procedural steps until it can be coded. [2]
2. The overall span of control: the largest number of modules at any one level of the hierarchy. [2]
3. Understandable language; actionable advice for recovery; statement of negative consequences; visual or auditory cue; never blames the user (any four). [2]
4. A set of inputs expected to be processed in the same way, so that testing one member represents the whole set. [2]
5. Boolean operator, Boolean variable, parenthesis, relational operator, arithmetic expression errors (any four). [2]
6. Nodes: 1 read; 2 $a > b$; 3 $a > c$; 4 $big = a$; 5 $big = c$; 6 $b > c$; 7 $big = b$; 8 $big = c$; 9 print. Edges: 1–2, 2–3, 2–6, 3–4, 3–5, 6–7, 6–8, 4–9, 5–9, 7–9, 8–9: $E = 11$, $N = 9$.

$$M = 11 - 9 + 2 = 4; \quad D + 1 = 3 + 1 = 4; \quad R = 3 \text{ enclosed} + 1 \text{ outer} = 4.$$

Path	Sequence	(a, b, c)	Output
P1	1-2-3-4-9	(5, 3, 1)	5
P2	1-2-3-5-9	(5, 3, 9)	9
P3	1-2-6-7-9	(3, 7, 1)	7
P4	1-2-6-8-9	(3, 7, 9)	9

[CFG 1, three methods 2, paths 2]

7. Executable statements S1–S7 (7); branches: S2-T, S2-F, S5-T, S5-F (4). Test {5}: executes S1, S2, S3, S5, S7 ($y = 5$, not > 10). Statement coverage = $5/7 \times 100 = 71.43\%$; branch coverage = $2/4 \times 100 = 50\%$ (S2-T, S5-F). Test {5, -20}: $x = -20$ executes S1, S2, S4, S5, S6, S7 ($y = 20$). Union covers all 7 statements (100%) and all 4 branches (100%). [5]

8. Retesting re-runs the previously failing shipping-cost test(s) to confirm the fix; regression testing re-runs previously passing tests to ensure nothing else broke. Next: retest the defect; perform impact analysis (checkout, invoice, order summary use shipping cost); select those tests plus a smoke sample; run them, preferably automated; analyse and report. [2 + 3]

9. Edges: 1–2, 2–3, 2–4, 3–5, 4–5, 5–2 (back edge), 5–6. Row counts: 1, 2, 1, 1, 2, 0. $M = (0 + 1 + 0 + 0 + 1) + 1 = 3$. Check: $E = 7$, $N = 6$: $7 - 6 + 2 = 3$. [graph 2, matrix 2, check 1]

10. (a) Temporal (or coincidental) cohesion (unrelated tasks grouped by timing): split into `MessBilling`, `PasswordService`, `LogArchiver`. (b) Stamp coupling: `pass room_no` only (data coupling). (c) Control coupling, and logical cohesion inside `process`: replace with `allot()`, `vacate()`, `transfer()`. (d) Common coupling: pass the semester as a parameter or encapsulate it in a `SemesterService` with accessor methods. (e) Functional cohesion with data coupling: already ideal. Conclude with the benefits of functional independence. [identification 5, redesign 4, conclusion 1]

11. Types with examples: functional (fund transfer creates debit and credit); cross-browser (Chrome, Firefox, Edge); cross-platform (Windows and Android); performance (month-end salary-day load); security (SQL injection, session timeout, HTTPS, OTP); usability (new user pays a bill unaided); database (transfer is atomic); API (balance API returns 401 for expired token); accessibility (screen reader on OTP page); localisation/internationalisation (Tamil and Hindi, INR formatting); regression (after new UPI feature). Configurations: $3 \times 2 = 6$. Inheritance: re-run `Account` tests on `SavingsAccount`, then test overridden `withdraw` at the minimum-balance boundary (balance 1000, withdraw 500 allowed, 501 rejected) and polymorphic calls through an `Account` reference. [types 6, configurations 1, OO 3]

H.3 Answers to Section D (Practice Problems)

D1. $M = 14 - 12 + 2 = 4$.

D2. $M = 5 + 1 = 6$ test cases.

D3. `if`: 1 decision; 6-way switch: $6 - 1 = 5$. $D = 6$, $M = 7$.

D4. $M = 15 - 13 + 2(2) = 6$.

D5. $150 / (200 - 20) \times 100 = 150 / 180 \times 100 = 83.33\%$.

D6. 0 invalid; 1, 2 valid; 98, 99 valid; 100 invalid.

D7. $2^4 = 16$.

- D8.** $(1 + 0 + 2 + 0 + 1) + 1 = 5$.
- D9.** $18/24 \times 100 = 75\%$.
- D10.** $7/10 \times 100 = 70\%$.
- D11.** Levels {R}, {A, B}, {C, D, E, F}, {G}: depth 4, width 4. Fan-out: R 2, A 3, B 2, E 1, others 0. Fan-in: E 2, R 0, all others 1. Both sums = 8.
- D12.** $4(2) + 1 = 9$ normal; $6(2) + 1 = 13$ robust.
- D13.** $E = 7, N = 7, M = 2$; tests $n = 0$ (output 1) and $n = 3$ (output 6).
- D14.** Four sequences: correct; wrong–correct; wrong–wrong–correct; wrong–wrong–wrong (blocked). Together they cover all seven transitions.

Final Revision Handbook

I.1 Mnemonics and Memory Hooks

Table I.1. Mnemonics.

Topic	Mnemonic	Expansion
Design models (base to apex)	“Developers Always Improve Code”	Data/class, Architectural, Interface, Component
Modularity criteria	DCUCP	Decomposability, Composability, Understandability, Continuity, Protection
Cohesion (worst→best)	“Cats Like Tuna Canned Fresh”	Coincidental, Logical, Temporal, Communicational, Functional
Coupling (worst→best)	“Can Computers Ever Control Software Data”	Content, Common, External, Control, Stamp, Data
Architecture models	“Some Frameworks Do Process Functions”	Structural, Framework, Dynamic, Process, Functional
Golden rules	“Control, Memory, Consistency” (CMC)	User in control; reduce memory load; consistent interface
UI process	“All Designers Construct Validly”	Analysis/modelling, Design, Construction, Validation
Cyclomatic methods	“Every Decision Recounted”	$E - N + 2P, D + 1, R$
Black-box techniques	“Every Boundary Deserves Scrutiny”	ECP, BVA, Decision table, State transition
Condition errors	“Big Bears Prefer Raw Apples”	Boolean operator, Boolean variable, Parenthesis, Relational, Arithmetic

I.2 Do Not Confuse These

- **Cohesion** (inside one module) vs **coupling** (between modules).
- **Communicational cohesion** (functions in one module share data) vs **common coupling** (different modules share global data).
- **Logical cohesion** (inside the callee) vs **control coupling** (the flag between caller and callee).
- **Fan-in** (callers) vs **fan-out** (callees).
- **Transform centre** (processing) vs **transaction centre** (routing).
- **Refactoring** (same behaviour) vs **debugging** (corrected behaviour).
- **Mental model** (user’s perception) vs **implementation model** (system’s look, feel and documentation).
- P in $E - N + 2P$ (components) vs predicate count (write D).
- **Killed** mutant (good for tests) vs **surviving** mutant (weak tests).

- **Regression** testing (nothing else broke) vs **retesting** (this defect is fixed).
- **Localisation** (one locale) vs **internationalisation** (any locale); both differ from **accessibility**.

I.3 Common Examination Traps

1. Reversing the cohesion or coupling order.
2. Omitting flow boundaries or controller modules in transform mapping.
3. Missing a loop's back edge in a CFG.
4. Counting a k -way switch as one decision.
5. Treating a compound condition as one predicate.
6. Subtracting 1 from empty rows in a connection matrix, or forgetting the final +1.
7. Using the number of decisions as the denominator of branch coverage.
8. In BVA, testing only 6 and 12 without 5 and 13.
9. Forgetting to exclude equivalent mutants from the mutation-score denominator.
10. Writing test cases without expected outputs.

I.4 Final-Day Revision Sheet

Module 4. Design follows analysis, precedes construction, applies to every process. Four models: data/class, architectural, interface, component. Good design: explicit + implicit requirements, readable, complete picture. Ten principles: tunnel vision, traceable, reuse, intellectual distance, uniformity, change, graceful degradation, not coding, assess quality, review.

Abstraction: procedural, data, control. Architecture properties: structural, extra-functional, families. Models: structural, framework, dynamic, process, functional. Control hierarchy: depth, width, fan-in, fan-out, super/subordinate. Partitioning: horizontal (branches/layers) vs vertical (factoring).

Patterns: proven named solutions. Separation of concerns: divide and conquer. Modularity: DCUCP; optimum M . Cohesion C-L-T-C-F; coupling C-C-E-C-S-D; aim: high cohesion, low coupling. Refinement: top-down. Refactoring: structure, not behaviour.

Mapping steps: flow type, boundaries, map, hierarchy, refine, elaborate. Transform: payroll, boundaries, first- and second-level factoring. Transaction: ATM, dispatcher.

UI: golden rules; four models; three user types; spiral process; response time (length, variability); help; errors; labels; accessibility; internationalisation (Unicode); sixteen principles.

Module 5. Testing: execute to find errors. Objectives: bugs, requirements, quality, satisfaction, compliance. Strategy: unit → integration → validation → system.

White box: glass box; statement $\subset$ branch $\subset$ path coverage. Condition errors: B, B, P, R, A.

Basis path: CFG → M → paths → tests. $M = E - N + 2P = D + 1 = R$.

Graph matrix: $N \times N$, row = from. Connection matrix: $M = \sum (c_i - 1) + 1$.

Black box: ECP (3 classes for a range), BVA (5, 6, 7, 11, 12, 13), decision table (2^n rules), state transition (states, events, transitions).

Regression: after fixes, features, refactoring, before release; identify, select, execute, analyse. Mutation: create, execute, analyse; score = killed / (total – equivalent).

OO: class, integration, inheritance, polymorphism, scenario; challenges: encapsulation, inheritance, polymorphism, state.

Web: functional, cross-browser, cross-platform, performance, security, usability, compatibility, database, API, accessibility, L10n/I18n, regression.

I.5 One-Page Formula Sheet

Quantity	Formula
Cyclomatic complexity	$M = E - N + 2P$
Cyclomatic complexity	$M = D + 1$ (k -way decision counts $k - 1$)
Cyclomatic complexity	$M = R$ (regions, including outer)
From connection matrix	$M = \sum_{c_i > 0} (c_i - 1) + 1$
Statement coverage	$\frac{\text{executed statements}}{\text{total statements}} \times 100\%$
Branch coverage	$\frac{\text{executed branches}}{\text{total branches}} \times 100\%$
Path coverage	$\frac{\text{executed paths}}{\text{total feasible paths}} \times 100\%$
Mutation score	$\frac{\text{killed}}{\text{total} - \text{equivalent}} \times 100\%$
Decision-table rules	2^n (Boolean conditions)
BVA test count (suppl.)	$4n + 1$ normal; $6n + 1$ robust
Fitts's law (suppl.)	$T = a + b \log_2(D/W + 1)$
Structure-chart check	$\sum \text{fan-out} = \sum \text{fan-in}$

I.6 One-Page Ultra-Fast Revision

Topic	Essentials
Design role	After analysis, before construction; every process model; blueprint
Design models	Data/class, architectural, interface, component-level
Abstraction	Procedural, data, control
Architecture	Structural, framework, dynamic, process, functional models
Control hierarchy	Depth (levels), width (max per level), fan-out, fan-in
Partitioning	Horizontal: branches/layers; vertical: factoring, control on top
Modularity	DCUCP; optimum between under- and overmodularity
Cohesion	Coincidental < logical < temporal < communicational < functional
Coupling	Content > common > external > control > stamp > data
Mapping	Transform (payroll, boundaries, factoring); transaction (ATM, dispatcher)
Refactoring	Structure up, behaviour same, technical debt down
UI	3 golden rules; 4 models; spiral; response time, help, errors, labels, accessibility, I18n; 16 principles
Testing	Find errors; unit → integration → validation → system
White box	Glass box; statement $\subset$ branch $\subset$ path; condition errors
Basis path	$M = E - N + 2P = D + 1 = R$; M tests
Matrices	Row = from; $\sum (c_i - 1) + 1$
Black box	ECP, BVA (5, 6, 7, 11, 12, 13), decision tables (2^n), state transitions
Regression	After every change; select, execute, analyse; automate
Mutation	Killed good, survived bad; score = killed / (total – equivalent)
OO testing	Class, integration, inheritance, polymorphism, scenario
Web testing	12 types; cross-browser vs cross-platform

Index

Accessibility testing, 97
Anticipation, 48
API testing, 97
Architectural design, 2
Arithmetic expression error, 58
Automated testing, 52

Black-box testing, 52
Boolean operator error, 58
Boolean variable error, 58

Class testing, 92
coincidental cohesion, 20
common coupling, 23
Communication, 48
communicational cohesion, 21
Component-level design, 2
Consistency, 48
content coupling, 23
Control abstraction, 10
control coupling, 24
Controlled autonomy, 48
Cross-browser testing, 96
Cross-platform testing, 97

Data abstraction, 10
data coupling, 25
Data/class design, 2
Database testing, 97
Depth, 31
Design model, 44
design model, 2
Dynamic model, 29

Efficiency, 48
Equivalent mutant, 88
external coupling, 24

Fan-in, 31
Fan-out, 31
Fitts's law, 48
Flexibility, 48
Focus, 48
Framework model, 29
functional cohesion, 21
functional independence, 19
Functional model, 29

Functional testing, 51, 52

Human interface objects, 48

Implementation model, 44
Inheritance testing, 92
Integration testing, 51
Interface design, 2

Killed mutant, 88

Latency reduction, 48
Learnability, 48
logical cohesion, 20

Maintain work-product integrity, 48
Maintenance testing, 52
Manual testing, 52
Mental model, 44
Metaphors, 48
Modular composability, 14
Modular continuity, 14
Modular decomposability, 14
Modular protection, 14
Modular understandability, 14
Mutant, 88
Mutation operator, 88

Non-functional testing, 52

Overmodularity, 15

Parenthesis error, 58
Performance testing, 51
Polymorphism testing, 92
Procedural abstraction, 10
Process model, 29

Readability, 48
Regression testing, 51
Relational operator error, 58
Requirement-based testing, 51

Scenario-based testing, 92
Security testing, 51
stamp coupling, 24
Structural model, 29
Subordinate module, 31

Superordinate module, 31
Surviving mutant, 88
System testing, 51

technical reviews, 4
temporal cohesion, 20
Track state, 48

Undermodularity, 15
Usability testing, 52
User acceptance testing (UAT), 51
User model, 44

Visible navigation, 48

White-box testing, 52
Width, 31